Proyecto_Terminal_1

July 22, 2025

[26]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy.signal import find_peaks

# --- 1. CONFIGURACIÓN Y CARGA DE DATOS ---
CONFIG = {
    'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv',
    'ventana_suavizado': 7,
    'dias_recuperacion': 14,
    'dpi_graficas': 300,
}
EVENTOS_CLAVE = {
    'Primer Caso': '2020-02-28', 'Sana Distancia': '2020-03-23',
    'Semáforo COVID-19': '2020-06-01', 'Inicio Vacunación': '2020-12-24',
}
COLORES = {
    'casos_crudos': '#c7c7c7', 'casos_suavizados': '#0d3b66', 'activos':␣
 ↪'#7a5195',
    'distribucion': '#f9a825', 'olas': ['#2a9d8f', '#e9c46a', '#f4a261',␣
 ↪'#e76f51', '#d62828', '#8e9aaf'],
    'tabla_header': '#0d3b66'
}

# --- Carga y Preprocesamiento ---
print("Cargando y preprocesando datos...")
df_raw = pd.read_csv(CONFIG['archivo_datos'])
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
poblacion_mexico = df_nacional['poblacion'].iloc[0]
casos_diarios = df_nacional.iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_nacional.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()
datos['infectados_activos'] = datos['casos_suavizados'].
 ↪rolling(CONFIG['dias_recuperacion'], min_periods=1).sum()
```

```python
# --- 2. DEFINICIÓN DE OLAS Y MÉTRICAS ---
print("Definiendo olas y calculando todas las métricas...")

# Algoritmo "Valle a Valle" para definir Olas
indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
nombres_olas = ['Ola 1 (Original)', 'Ola 2 (Alpha)', 'Ola 3 (Delta)', 'Ola 4␣
 ↪(Ómicron)', 'Ola 5 (Ómicron Subs.)', 'Ola 6']
for i in range(len(indices_valles_completos) - 1):
    if i >= len(nombres_olas): break
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
    datos_ola_slice = datos['casos_suavizados'].iloc[inicio_idx:fin_idx]
    if datos_ola_slice.empty: continue
    idx_pico_local = datos_ola_slice.idxmax()
    altura_pico = datos_ola_slice.max()
    if altura_pico > 1000:
        olas_definidas[nombres_olas[len(olas_definidas)]] = {
            'inicio': datos.index[inicio_idx], 'fin': datos.index[fin_idx],
            'pico_fecha': idx_pico_local, 'pico_valor': altura_pico
        }

# Dataframe para la tabla de resumen por ola
tabla_resumen_data = []
for nombre, info in olas_definidas.items():
    datos_ola = datos.loc[info['inicio']:info['fin']]
    tabla_resumen_data.append({
        'Ola': nombre, 'Inicio': info['inicio'].strftime('%Y-%m-%d'), 'Fin':␣
 ↪info['fin'].strftime('%Y-%m-%d'),
        'Duración (días)': (info['fin'] - info['inicio']).days, 'Fecha Pico':␣
 ↪info['pico_fecha'].strftime('%Y-%m-%d'),
        'Casos en Pico': f"{int(info['pico_valor']):,}", 'Total Casos':␣
 ↪f"{int(datos_ola['casos_diarios'].sum()):,}",
        'Promedio Diario': f"{int(datos_ola['casos_diarios'].mean()):,}"
    })
tabla_resumen_df = pd.DataFrame(tabla_resumen_data)

# Dataframe para la tabla de métricas globales
pico_diario_fecha = datos['casos_suavizados'].idxmax()
pico_diario_valor = datos['casos_suavizados'].max()
pico_activos_fecha = datos['infectados_activos'].idxmax()
pico_activos_valor = datos['infectados_activos'].max()
```

```
metricas_globales_data = {
    'Métrica': ['Período de Análisis', 'Duración Total (días)', 'Total de Casos␣
 ↪Confirmados', 'Promedio de Casos Diarios',
                'Mediana de Casos Diarios', 'Pico Máx. de Casos Diarios␣
 ↪(suavizado)', 'Fecha del Pico de Casos Diarios',
                'Pico Máx. de Casos Activos (estimado)', 'Fecha del Pico de␣
 ↪Casos Activos'],
    'Valor': [f"{datos.index.min().strftime('%Y-%m-%d')} al {datos.index.max().
 ↪strftime('%Y-%m-%d')}",
              f"{(datos.index.max() - datos.index.min()).days:,}",␣
 ↪f"{int(datos['casos_diarios'].sum()):,}",
              f"{datos['casos_diarios'].mean():,.2f}",␣
 ↪f"{datos['casos_diarios'].median():,.2f}",
              f"{int(pico_diario_valor):,}", pico_diario_fecha.
 ↪strftime('%Y-%m-%d'),
              f"{int(pico_activos_valor):,}", pico_activos_fecha.
 ↪strftime('%Y-%m-%d')]
}
metricas_globales_df = pd.DataFrame(metricas_globales_data)

# --- 3. GENERACIÓN DE TODAS LAS SALIDAS ---
print("\n" + "="*80)
print("Tabla 1: Resumen de Métricas por Ola")
print("="*80)
print(tabla_resumen_df.to_string(index=False))
print("\n" + "="*80)
print("Tabla 2: Métricas Epidemiológicas Globales")
print("="*80)
print(metricas_globales_df.to_string(index=False))
print("="*80 + "\n")
print("Generando todos los archivos de imagen...")

# --- Gráfica/Tabla 0a: Tabla de Resumen por Ola como Imagen ---
fig, ax = plt.subplots(figsize=(12, 4)); ax.axis('tight'); ax.axis('off')
the_table = ax.table(cellText=tabla_resumen_df.values,␣
 ↪colLabels=tabla_resumen_df.columns, loc='center', cellLoc='center')
the_table.auto_set_font_size(False); the_table.set_fontsize(10); the_table.
 ↪scale(1.2, 1.2)
for (i, j), cell in the_table.get_celld().items():
    if i == 0: cell.set_text_props(weight='bold', color='white'); cell.
 ↪set_facecolor(COLORES['tabla_header'])
plt.savefig('tabla_1_resumen_olas.png', dpi=CONFIG['dpi_graficas'],␣
 ↪bbox_inches='tight', pad_inches=0.05)
print(" -> Tabla de Resumen por Ola guardada como 'tabla_1_resumen_olas.png'")

# --- Gráfica/Tabla 0b: Tabla de Métricas Globales como Imagen ---
```

```
fig, ax = plt.subplots(figsize=(8, 5)); ax.axis('tight'); ax.axis('off')
the_table = ax.table(cellText=metricas_globales_df.values,␣
 ↪colLabels=metricas_globales_df.columns, loc='center', cellLoc='left',␣
 ↪colWidths=[0.6, 0.4])
the_table.auto_set_font_size(False); the_table.set_fontsize(12); the_table.
 ↪scale(1.2, 1.5)
for (i, j), cell in the_table.get_celld().items():
    if i == 0: cell.set_text_props(weight='bold', color='white'); cell.
 ↪set_facecolor(COLORES['tabla_header'])
    if j == 1: cell.set_text_props(ha='right')
plt.title('Tabla de Métricas Epidemiológicas Globales', fontsize=16,␣
 ↪weight='bold', pad=20)
plt.savefig('tabla_2_metricas_globales.png', dpi=CONFIG['dpi_graficas'],␣
 ↪bbox_inches='tight', pad_inches=0.1)
print(" -> Tabla de Métricas Globales guardada como 'tabla_2_metricas_globales.
 ↪png'")

# --- Gráfica 1: Distribución de Frecuencias ---
fig, ax = plt.subplots(figsize=(12, 7))
ax.hist(datos['casos_suavizados'].dropna(), bins=50,␣
 ↪color=COLORES['distribucion'], edgecolor='black', alpha=0.7)
ax.set_title('Distribución de Frecuencias de Casos Diarios (Suavizados)',␣
 ↪fontsize=16, weight='bold')
ax.set_xlabel('Número de Casos Diarios'); ax.set_ylabel('Frecuencia (Días)')
ax.grid(axis='y', linestyle='--', alpha=0.7); ax.set_yscale('log')
ax.get_xaxis().set_major_formatter(plt.FuncFormatter(lambda x, p:␣
 ↪format(int(x), ',')))
plt.savefig('grafica_1_distribucion_frecuencias_final.png',␣
 ↪dpi=CONFIG['dpi_graficas'])
print(" -> Gráfica de Distribución guardada como␣
 ↪'grafica_1_distribucion_frecuencias_final.png'")

# --- Gráfica 2: Comparativa de Datos Crudos vs. Suavizados (1x2) ---
fig, (ax_raw, ax_smooth) = plt.subplots(1, 2, figsize=(20, 8), sharey=True)
fig.suptitle('Efecto del Suavizado en la Serie Temporal de Casos Diarios',␣
 ↪fontsize=18, weight='bold')
ax_raw.set_title('Datos Crudos Reportados', fontsize=14); ax_raw.plot(datos.
 ↪index, datos['casos_diarios'], 'o', markersize=2,␣
 ↪color=COLORES['casos_crudos'], alpha=0.5)
ax_raw.set_ylabel('Número de Casos Diarios'); ax_raw.grid(True, linestyle='--',␣
 ↪alpha=0.6)
ax_smooth.set_title('Tendencia (Promedio Móvil 7 días)', fontsize=14);␣
 ↪ax_smooth.plot(datos.index, datos['casos_suavizados'],␣
 ↪color=COLORES['casos_suavizados'], linewidth=2.5)
ax_smooth.grid(True, linestyle='--', alpha=0.6)
for ax in [ax_raw, ax_smooth]:
```

```
    ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p:␣
 ↪format(int(x), ',')))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y')); ax.xaxis.
 ↪set_major_locator(mdates.YearLocator())
plt.savefig('grafica_2_comparativa_suavizado.png', dpi=CONFIG['dpi_graficas'])
print(" -> Gráfica Comparativa 1x2 guardada como␣
 ↪'grafica_2_comparativa_suavizado.png'")

# --- Gráfica 3: Dashboard Epidemiológico 2x2 ---
fig, axes = plt.subplots(2, 2, figsize=(20, 14), sharex=True)
fig.suptitle('Análisis de la Dinámica de COVID-19 en México (Definición Valle a␣
 ↪Valle)', fontsize=22, weight='bold')
((ax1, ax2), (ax3, ax4)) = axes
ax1.set_title('Casos Diarios (Lineal)', fontsize=14)
ax1.plot(datos.index, datos['casos_diarios'], 'o', markersize=2,␣
 ↪color=COLORES['casos_crudos'], alpha=0.3, label='Casos reportados')
ax1.plot(datos.index, datos['casos_suavizados'],␣
 ↪color=COLORES['casos_suavizados'], linewidth=2, label='Tendencia')
ax1.set_ylabel('Número de Casos'); ax1.legend()
ax2.set_title('Casos Diarios (Log)', fontsize=14); ax2.plot(datos.index,␣
 ↪datos['casos_suavizados'], color=COLORES['casos_suavizados'], linewidth=2)
ax2.set_yscale('log'); ax2.set_ylabel('Número de Casos (Log)')
ax3.set_title('Infectados Activos (Lineal)', fontsize=14); ax3.plot(datos.
 ↪index, datos['infectados_activos'], color=COLORES['activos'], linewidth=2)
ax3.fill_between(datos.index, datos['infectados_activos'],␣
 ↪color=COLORES['activos'], alpha=0.2); ax3.set_ylabel('Infectados Activos')
ax4.set_title('Infectados Activos (Log)', fontsize=14); ax4.plot(datos.index,␣
 ↪datos['infectados_activos'], color=COLORES['activos'], linewidth=2)
ax4.set_yscale('log'); ax4.set_ylabel('Infectados Activos (Log)')
for ax in axes.flatten():
    ax.grid(True, which='both', linestyle='--', alpha=0.6)
    ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p:␣
 ↪format(int(x), ',')))
    for i, (nombre, info) in enumerate(olas_definidas.items()):
        ax.axvspan(info['inicio'], info['fin'], color=COLORES['olas'][i %␣
 ↪len(COLORES['olas'])], alpha=0.15, zorder=0)
for evento, fecha in EVENTOS_CLAVE.items():
    ax1.axvline(pd.to_datetime(fecha), color='black', linestyle=':',␣
 ↪linewidth=1.5)
    ax1.text(pd.to_datetime(fecha) + pd.Timedelta(days=5), ax1.get_ylim()[1]*0.
 ↪6, evento, rotation=90, va='center', fontsize=10,␣
 ↪backgroundcolor='#ffffffaa')
plt.xlabel('Fecha'); ax3.xaxis.set_major_formatter(mdates.DateFormatter('%b␣
 ↪%Y'))
ax3.xaxis.set_major_locator(mdates.MonthLocator(interval=6))
plt.tight_layout(rect=[0, 0, 1, 0.96])
```

```
plt.savefig('grafica_3_dashboard_epidemiologico_final.png',␣
 ↪dpi=CONFIG['dpi_graficas'])
print(" -> Gráfica Dashboard 2x2 (Final) guardada como␣
 ↪'grafica_3_dashboard_epidemiologico_final.png'")

plt.show()
```

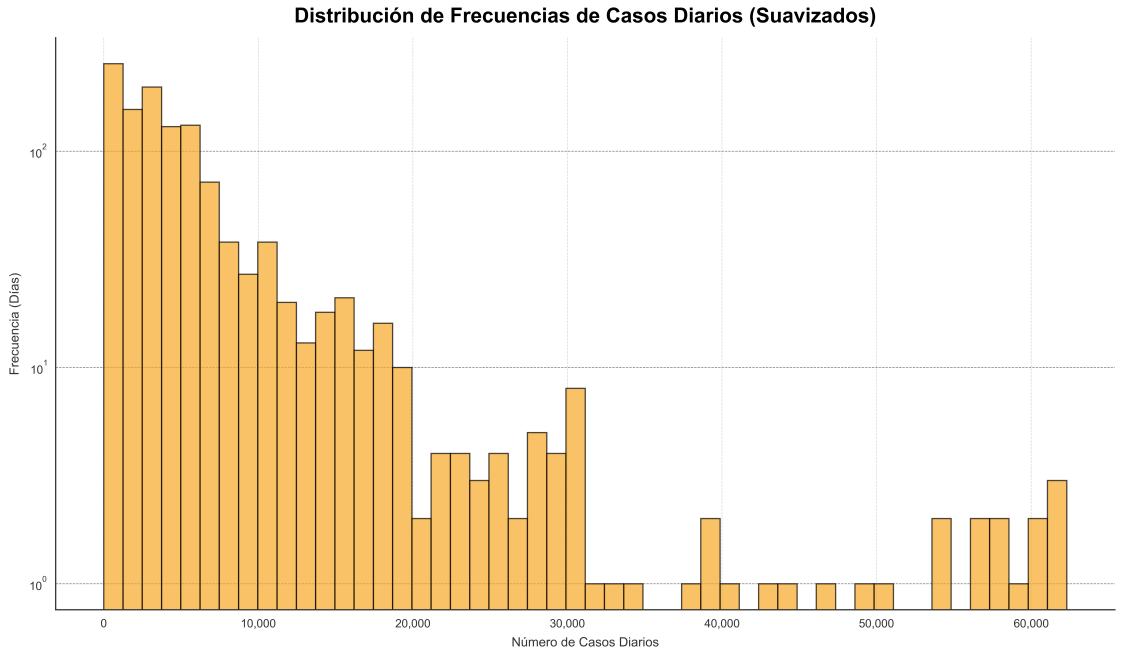
```
Cargando y preprocesando datos…
Definiendo olas y calculando todas las métricas…

================================================================================
Tabla 1: Resumen de Métricas por Ola
================================================================================
                  Ola     Inicio        Fin  Duración (días) Fecha Pico Casos en
Pico Total Casos Promedio Diario
     Ola 1 (Original) 2020-02-26 2020-09-13             200 2020-07-18
7,275     733,718           3,650
        Ola 2 (Alpha) 2020-09-13 2020-12-23             101 2020-12-18
11,088     708,399           6,945
        Ola 3 (Delta) 2020-12-23 2021-05-13             141 2021-01-10
16,010     980,207           6,902
      Ola 4 (Ómicron) 2021-05-13 2021-12-14             215 2021-08-10
19,305   1,567,627           7,257
Ola 5 (Ómicron Subs.) 2021-12-14 2022-04-14             121 2022-01-15
62,299   1,798,477          14,741
               Ola 6 2022-04-14 2022-10-30             199 2022-07-10
30,824   1,364,627           6,823

================================================================================
Tabla 2: Métricas Epidemiológicas Globales
================================================================================
                              Métrica                    Valor
                  Período de Análisis 2020-02-26 al 2023-06-24
                Duración Total (días)                   1,214
           Total de Casos Confirmados               7,633,355
            Promedio de Casos Diarios                6,282.60
             Mediana de Casos Diarios                3,461.00
Pico Máx. de Casos Diarios (suavizado)                  62,299
       Fecha del Pico de Casos Diarios              2022-01-15
 Pico Máx. de Casos Activos (estimado)                 803,713
       Fecha del Pico de Casos Activos              2022-01-22
================================================================================

Generando todos los archivos de imagen…
 -> Tabla de Resumen por Ola guardada como 'tabla_1_resumen_olas.png'
 -> Tabla de Métricas Globales guardada como 'tabla_2_metricas_globales.png'
 -> Gráfica de Distribución guardada como
```

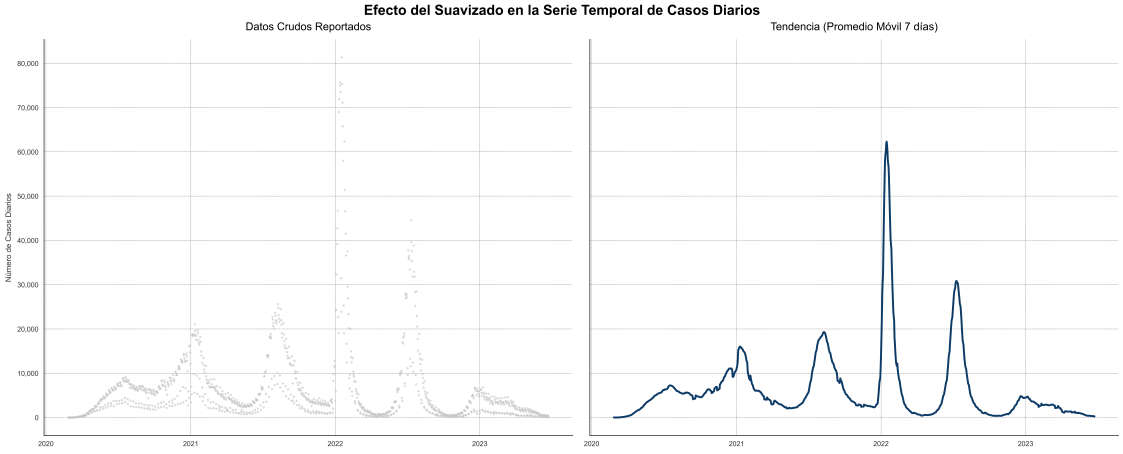
```
'grafica_1_distribucion_frecuencias_final.png'
 -> Gráfica Comparativa 1x2 guardada como 'grafica_2_comparativa_suavizado.png'
 -> Gráfica Dashboard 2x2 (Final) guardada como
'grafica_3_dashboard_epidemiologico_final.png'
```

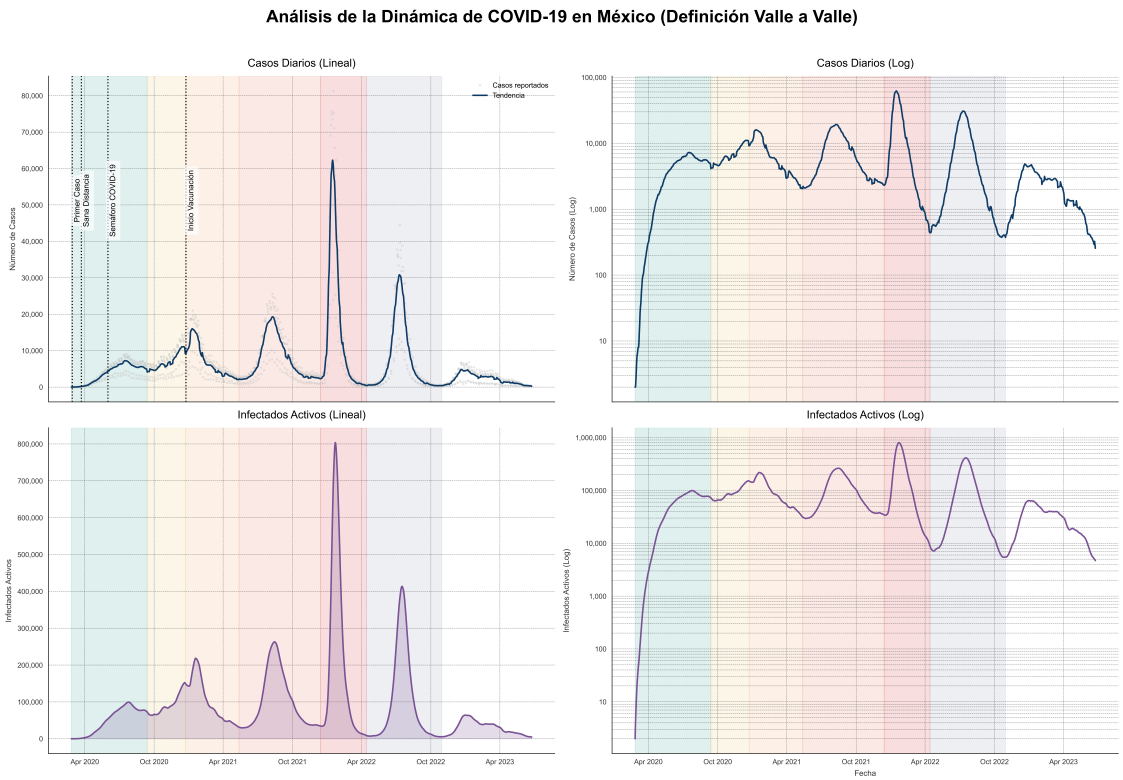
Ola	Inicio	Fin	Duración (días)	Fecha Pico	Casos en Pico	Total Casos	Promedio Diario
Ola 1 (Original)	2020-02-26	2020-09-13	200	2020-07-18	7,275	733,718	3,650
Ola 2 (Alpha)	2020-09-13	2020-12-23	101	2020-12-18	11,088	708,399	6,945
Ola 3 (Delta)	2020-12-23	2021-05-13	141	2021-01-10	16,010	980,207	6,902
Ola 4 (Ómicron)	2021-05-13	2021-12-14	215	2021-08-10	19,305	1,567,627	7,257
Ola 5 (Ómicron Subs.)	2021-12-14	2022-04-14	121	2022-01-15	62,299	1,798,477	14,741
Ola 6	2022-04-14	2022-10-30	199	2022-07-10	30,824	1,364,627	6,823

Tabla de Métricas Epidemiológicas Globales

Métrica	Valor
Período de Análisis	2020-02-26 al 2023-06-24
Duración Total (días)	1,214
Total de Casos Confirmados	7,633,355
Promedio de Casos Diarios	6,282.60
Mediana de Casos Diarios	3,461.00
Pico Máx. de Casos Diarios (suavizado)	62,299
Fecha del Pico de Casos Diarios	2022-01-15
Pico Máx. de Casos Activos (estimado)	803,713
Fecha del Pico de Casos Activos	2022-01-22


Distribución de Frecuencias de Casos Diarios (Suavizados)
Frecuencia (Días)
10^2
10^1
10^0
0
10,000
20,000
30,000
40,000
50,000
60,000
Número de Casos Diarios


Efecto del Suavizado en la Serie Temporal de Casos Diarios
Datos Crudos Reportados
Tendencia (Promedio Móvil 7 días)
Número de Casos Diarios
80,000
70,000
60,000
50,000
40,000
30,000
20,000
10,000
0
2020
2021
2022
2023



[37]:
```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.patheffects as path_effects

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO PARA PUBLICACIÓN CIENTÍFICA ---
# ==============================================================================

# 1. Paleta de Colores
PALETA = {
    'S': '#6495ED',  # UAM Azul Claro
    'I': '#D55E00',  # UNAM Naranja Quemado
    'R': '#009E73',  # Verde Sereno
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
    'Infeccion': '#C0392B',
    'Recuperacion': '#2980B9'
}

# 2. Configuración Global de Matplotlib
plt.rcParams.update({
    'font.family': 'sans-serif',
```

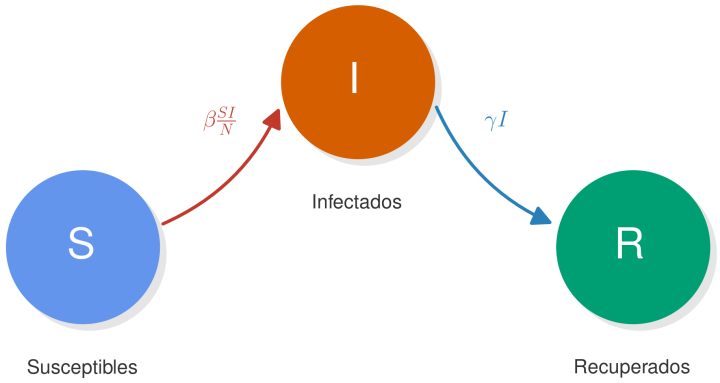
```
    'font.sans-serif': ['Arial', 'Helvetica', 'DejaVu Sans'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
    'font.size': 12, 'axes.labelsize': 12, 'axes.titlesize': 14,
    'figure.titlesize': 16, 'text.color': PALETA['Texto'],
    'savefig.format': 'pdf', 'savefig.bbox': 'tight', 'figure.dpi': 300
})

# ==============================================================================
# --- GENERACIÓN DEL DIAGRAMA ELEGANTE ---
# ==============================================================================

# --- 1. CONFIGURACIÓN DE LA FIGURA ---
fig, ax = plt.subplots(figsize=(10, 7))
ax.set_aspect('equal')
ax.set_xlim(0, 10)
ax.set_ylim(0, 7)
ax.axis('off') # Ocultar ejes y marco
fig.patch.set_facecolor(PALETA['Fondo'])
ax.set_facecolor(PALETA['Fondo'])

# --- 2. TÍTULOS Y DESCRIPCIÓN ---
fig.suptitle(r'\textbf{Modelo de Compartimentos SIR}', y=0.94)
ax.set_title('Flujo de la epidemia entre estados', style='italic', fontsize=12,␣
 ↪pad=20)

# --- 3. DIBUJAR COMPARTIMENTOS ---
comp_style = dict(radius=0.7, ec='none', zorder=2)
shadow_style = dict(radius=0.7, fc='black', alpha=0.1, zorder=1, transform=ax.
 ↪transData)
text_style = dict(ha='center', va='center', fontsize=26, weight='bold',␣
 ↪color='white', zorder=3)
label_style = dict(ha='center', va='center', fontsize=12, zorder=3)

# --- AJUSTE: Composición triangular para un look más dinámico ---
positions = {'S': (2.5, 3), 'I': (5, 4.5), 'R': (7.5, 3)}
labels = {'S': 'Susceptibles', 'I': 'Infectados', 'R': 'Recuperados'}

for comp, pos in positions.items():
    ax.add_patch(mpatches.Circle((pos[0]+0.06, pos[1]-0.06), **shadow_style))
    ax.add_patch(mpatches.Circle(pos, fc=PALETA[comp], **comp_style))
    ax.text(pos[0], pos[1], comp, **text_style)
    ax.text(pos[0], pos[1] - 1.1, labels[comp], **label_style)

# --- 4. DIBUJAR FLUJOS CURVOS Y TASAS "FLOTANTES" ---
arrow_style = dict(arrowstyle='-|>', mutation_scale=25, lw=2.0,␣
 ↪connectionstyle="arc3,rad=0.2")
```

```
rate_text_style = dict(ha='center', va='center', fontsize=14)

# Efecto de halo para el texto de las tasas
halo_effect = [path_effects.withStroke(linewidth=3, foreground=PALETA['Fondo'])]

# Flecha de S a I (Infección)
ax.add_patch(mpatches.FancyArrowPatch((3.2, 3.2), (4.3, 4.3),␣
 ↪color=PALETA['Infeccion'], **arrow_style))
ax.text(3.75, 4.15, r'$\beta \frac{SI}{N}$', color=PALETA['Infeccion'],␣
 ↪path_effects=halo_effect, **rate_text_style)

# Flecha de I a R (Recuperación)
ax.add_patch(mpatches.FancyArrowPatch((5.7, 4.3), (6.8, 3.2),␣
 ↪color=PALETA['Recuperacion'], **arrow_style))
ax.text(6.25, 4.15, r'$\gamma I$', color=PALETA['Recuperacion'],␣
 ↪path_effects=halo_effect, **rate_text_style)

# --- 5. DEFINICIÓN DE PARÁMETROS REFINADA ---
params_text = (r'\textbf{Parámetros del Modelo}' + '\n' + r'$\beta$: Tasa de␣
 ↪transmisión' + '\n' + r'$\gamma$: Tasa de recuperación')
ax.text(0.5, 0.1, params_text, transform=ax.transAxes, ha='center',␣
 ↪va='center', fontsize=11,
        bbox=dict(boxstyle="round,pad=0.5", fc=PALETA['Fondo'], alpha=0.7,␣
 ↪ec='none'))

# --- 6. GUARDAR FIGURA ---
plt.savefig('Diagrama_Flujo_SIR_Elegante.pdf')
plt.savefig('Diagrama_Flujo_SIR_Elegante.png', dpi=300)
print(" Diagrama de Flujo SIR elegante guardado como␣
 ↪'Diagrama_Flujo_SIR_Elegante.pdf' y .png")
plt.show()
```

```
 Diagrama de Flujo SIR elegante guardado como 'Diagrama_Flujo_SIR_Elegante.pdf'
y .png
```

Flujo de la epidemia entre estados

Modelo de Compartimentos SIR



Parámetros del Modelo
β: Tasa de transmisión
γ: Tasa de recuperación

[78]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
import matplotlib.gridspec as gridspec

# ======================================================================
# ESTILO PARA PUBLICACIÓN CIENTÍFICA
# ======================================================================

def setup_publication_style():
    plt.rcParams.update({
        'font.family': 'serif',
        'font.serif': ['Computer Modern Roman', 'Times New Roman'],
        'text.usetex': True,
        'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 11,
        'axes.labelsize': 12,
        'axes.titlesize': 14,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
```

```
        'legend.fontsize': 9,
        'figure.titlesize': 16,
        'lines.linewidth': 1.8,
        'axes.linewidth': 0.8,
        'grid.color': '#cccccc',
        'grid.linestyle': ':',
        'grid.linewidth': 0.6,
        'axes.spines.right': False,
        'axes.spines.top': False,
        'xtick.direction': 'in',
        'ytick.direction': 'in',
        'legend.frameon': True,
        'legend.framealpha': 0.9,
        'legend.fancybox': True,
        'legend.facecolor': 'white',
        'legend.edgecolor': '#dddddd',
        'figure.constrained_layout.use': True,  # Habilitar layout automático
        'figure.constrained_layout.h_pad': 0.2, # Aumentado ligeramente para␣
 ↪más espacio vertical
        'figure.constrained_layout.w_pad': 0.2, # Aumentado ligeramente para␣
 ↪más espacio horizontal
        'figure.constrained_layout.hspace': 0.15, # Aumentado para espacio␣
 ↪entre filas
        'figure.constrained_layout.wspace': 0.15  # Aumentado para espacio␣
 ↪entre columnas
    })

# Activar estilo
setup_publication_style()

# ======================================================================
# PARÁMETROS Y FUNCIONES DEL MODELO
# ======================================================================
GAMMA = 1.0 / 10.0
POP   = 1000.0
PARAMETER_SETS = [
    {'name': 'R0=0.8', 'R0': 0.8}, {'name': 'R0=1.0', 'R0': 1.0},
    {'name': 'R0=1.5', 'R0': 1.5}, {'name': 'R0=2.5', 'R0': 2.5},
    {'name': 'R0=3.5', 'R0': 3.5},
]
cmap = plt.get_cmap('cividis')
norm = plt.Normalize(vmin=0.8, vmax=3.5)

def compute_eigenvalue(s_frac, R0):
    return GAMMA * (R0 * s_frac - 1)

def sir_model(t, y, R0):
```

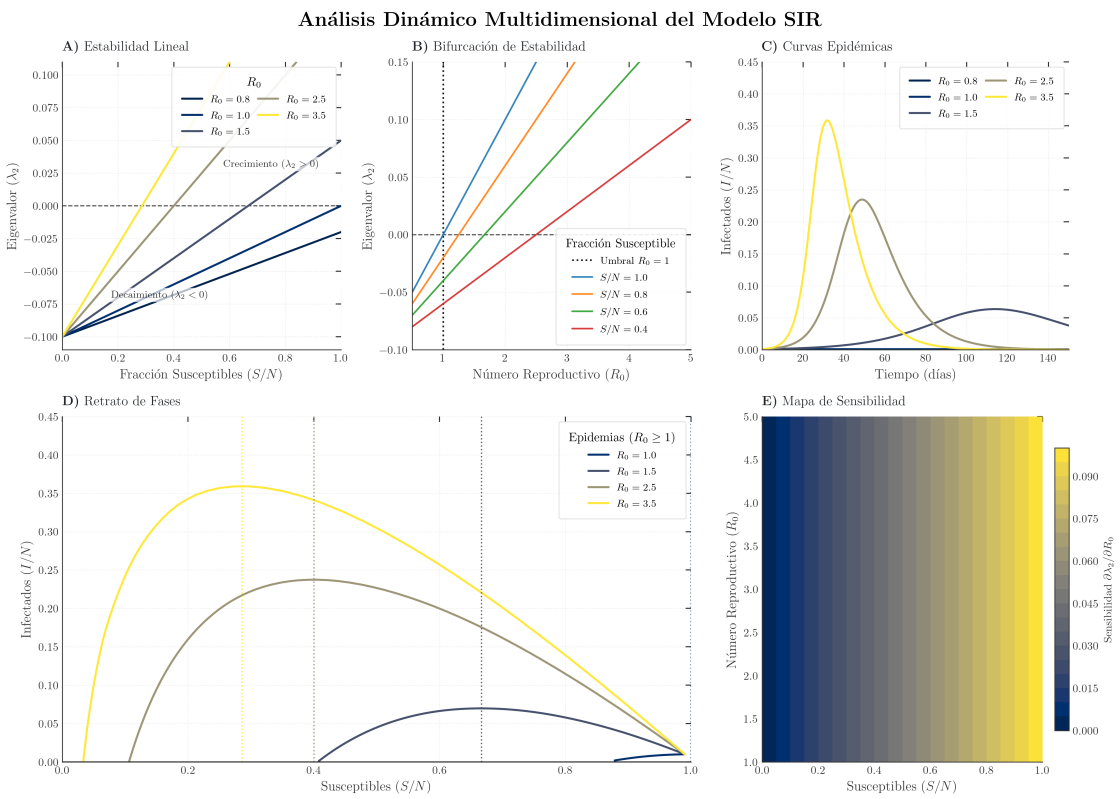
```
    S, I = y
    beta = R0 * GAMMA
    dS_dt = -beta * S * I / POP
    dI_dt = beta * S * I / POP - GAMMA * I
    return [dS_dt, dI_dt]

# ================================================================
# FUNCIONES DE GRAFICADO MEJORADAS
# ================================================================
def plot_stability(ax):
    s_vals = np.linspace(0, 1, 200)
    ax.axhline(0, color='k', linestyle='--', lw=1, alpha=0.7)
    for p in PARAMETER_SETS:
        lam = compute_eigenvalue(s_vals, p['R0'])
        ax.plot(s_vals, lam, color=cmap(norm(p['R0'])),
                label=f"$R_0={p['R0']:.1f}$")

    # Anotaciones mejoradas con bbox para mayor claridad
    # Se ajusta la posición de 'Crecimiento' para evitar superposición con la␣
 ↪leyenda
    ax.annotate(r'Crecimiento ($\lambda_2 > 0$)',
                xy=(0.75, 0.03), ha='center', fontsize=9, alpha=0.8,
                bbox=dict(facecolor='white', alpha=0.7, edgecolor='none',␣
 ↪boxstyle='round,pad=0.2'))
    ax.annotate(r'Decaimiento ($\lambda_2 < 0$)',
                xy=(0.35, -0.07), ha='center', fontsize=9, alpha=0.8,
                bbox=dict(facecolor='white', alpha=0.7, edgecolor='none',␣
 ↪boxstyle='round,pad=0.2'))

    ax.set(xlabel=r'Fracción Susceptibles ($S/N$)', ylabel=r'Eigenvalor␣
 ↪($\lambda_2$)')
    ax.set_xlim(0, 1)
    ax.set_ylim(-0.11, 0.11)

    # Leyenda compacta
    ax.legend(title=r'$R_0$', loc='upper right', ncol=2, columnspacing=0.8)

def plot_bifurcation(ax):
    R0s = np.linspace(0.5, 5, 200)
    s_fracs = [1.0, 0.8, 0.6, 0.4]
    ax.axhline(0, color='k', linestyle='--', lw=1, alpha=0.7)
    ax.axvline(1, color='k', linestyle=':', lw=1.5, alpha=0.9, label=r'Umbral␣
 ↪$R_0=1$')

    for s in s_fracs:
        lam = compute_eigenvalue(s, R0s)
        ax.plot(R0s, lam, lw=1.5, alpha=0.9, label=f'$S/N={s:.1f}$')
```

```
    ax.set(xlabel=r'Número Reproductivo ($R_0$)', ylabel=r'Eigenvalor␣
 ↪($\lambda_2$)')
    ax.set_xlim(0.5, 5)
    ax.set_ylim(-0.1, 0.15)

    # Leyenda mejor posicionada
    ax.legend(title='Fracción Susceptible', loc='lower right', ncol=1)

def plot_time_series(ax):
    t = np.linspace(0, 150, 300)
    for p in PARAMETER_SETS:
        sol = solve_ivp(sir_model, [0, 150], [POP-1, 1], args=(p['R0'],),␣
 ↪t_eval=t)
        ax.plot(sol.t, sol.y[1]/POP, color=cmap(norm(p['R0'])))

    ax.set(xlabel='Tiempo (días)', ylabel='Infectados ($I/N$)')
    ax.set_xlim(0, 150)
    ax.set_ylim(0, 0.45)

    # Leyenda compacta en esquina
    handles = [plt.Line2D([0], [0], color=cmap(norm(p['R0'])),
                          label=f"$R_0={p['R0']:.1f}$") for p in PARAMETER_SETS]
    ax.legend(handles=handles, loc='upper right', ncol=2, columnspacing=0.8)

def plot_sensitivity(ax):
    Sg, Rg = np.meshgrid(np.linspace(0, 1, 100), np.linspace(1, 5, 100))
    sens = GAMMA * Sg
    cont = ax.contourf(Sg, Rg, sens, levels=20, cmap='cividis')
    ax.set(xlabel=r'Susceptibles ($S/N$)', ylabel=r'Número Reproductivo␣
 ↪($R_0$)')

    # Barra de color más compacta
    cbar = plt.colorbar(cont, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label(r'Sensibilidad $\partial\lambda_2 / \partial R_0$',␣
 ↪fontsize=10)

def plot_phase_portrait(ax):
    y0 = [POP - 10, 10]
    epidemic_lines = []
    r0_values = []

    for p in PARAMETER_SETS:
        if p['R0'] >= 1.0:
            sol = solve_ivp(sir_model, [0, 200], y0, args=(p['R0'],),␣
 ↪dense_output=True)
            t = np.linspace(0, sol.t[-1], 300)
```

```
            s_path, i_path = sol.sol(t)
            line, = ax.plot(s_path/POP, i_path/POP, color=cmap(norm(p['R0'])))
            epidemic_lines.append(line)
            r0_values.append(p['R0'])

    # Nulclinas con etiquetas
    s_null_vals = np.array([1/p['R0'] for p in PARAMETER_SETS if p['R0'] >= 1.
 ↪0])
    for s_null in s_null_vals:
        ax.axvline(s_null, color=cmap(norm(1/s_null)), linestyle=':', lw=1.2,␣
 ↪alpha=0.9)

    ax.axhline(0, color='k', lw=1)
    ax.set(xlabel=r'Susceptibles ($S/N$)', ylabel=r'Infectados ($I/N$)')
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 0.45)

    # Leyenda para epidemias
    if epidemic_lines:
        labels = [f"$R_0={r:.1f}$" for r in r0_values]
        ax.legend(epidemic_lines, labels, title=r'Epidemias ($R_0 \geq 1$)',
                  loc='upper right', ncol=1)

# ======================================================================
# CREACIÓN DE FIGURA CON DISEÑO OPTIMIZADO
# ======================================================================
fig = plt.figure(figsize=(14, 10)) # constrained_layout=True se maneja en␣
 ↪rcParams

# GridSpec optimizado con relaciones de altura
gs = gridspec.GridSpec(
    2, 3, figure=fig,
    height_ratios=[1, 1.2],  # Más espacio para la fila inferior
    width_ratios=[1, 1, 1.1],  # Más espacio para el último panel
)

ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[0, 2])
ax4 = fig.add_subplot(gs[1, 0:2])  # Panel grande para retrato de fases
ax5 = fig.add_subplot(gs[1, 2])

axes = [ax1, ax2, ax3, ax4, ax5]
titles = [
    r'\textbf{A)} Estabilidad Lineal',
    r'\textbf{B)} Bifurcación de Estabilidad',
    r'\textbf{C)} Curvas Epidémicas',
```

```
    r'\textbf{D)} Retrato de Fases',
    r'\textbf{E)} Mapa de Sensibilidad'
]

for ax, title in zip(axes, titles):
    ax.set_title(title, loc='left', fontsize=12, pad=10)

# Llamadas a las funciones de graficado
plot_stability(ax1)
plot_bifurcation(ax2)
plot_time_series(ax3)
plot_phase_portrait(ax4)
plot_sensitivity(ax5)

# Título principal con espacio adicional
fig.suptitle(r'\textbf{Análisis Dinámico Multidimensional del Modelo SIR}',
             y=0.98, fontsize=18)

# Guardar sin fig.tight_layout() ya que constrained_layout está activado en␣
 ↪rcParams
fig.savefig('Figura_Analisis_Dinamico_Completa.pdf')
fig.savefig('Figura_Analisis_Dinamico_Completa.png', dpi=300)

print("  Figura completa de 5 paneles guardada con diseño optimizado.")
plt.show()
```

```
 Figura completa de 5 paneles guardada con diseño optimizado.
```



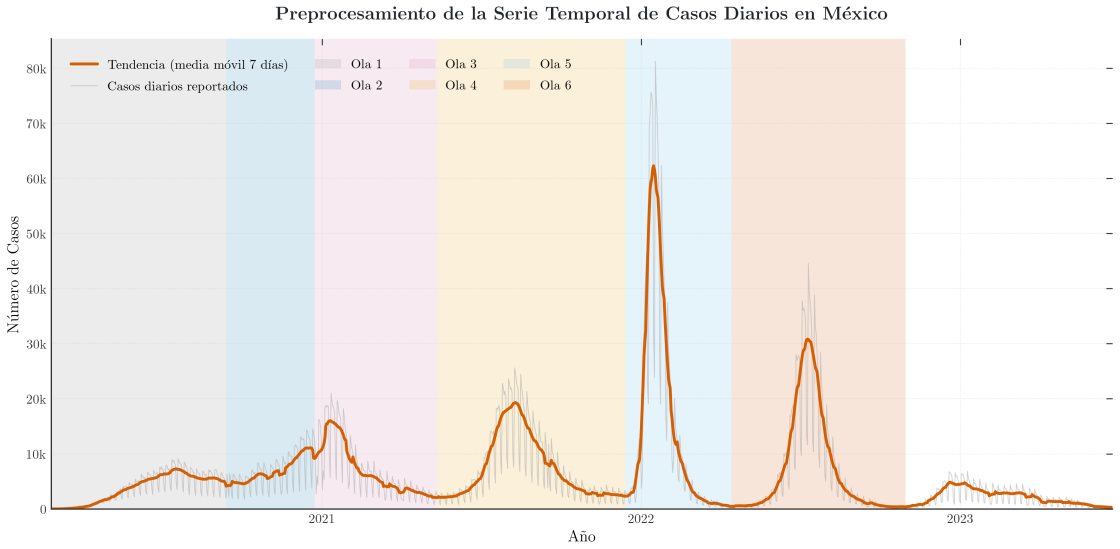
[70]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy.signal import find_peaks

# ================================================================================
# --- INICIO: PLANTILLA DE ESTILO PARA PUBLICACIÓN CIENTÍFICA ---
# ================================================================================

# 1. Paleta de Colores Oficial
PALETA = {
    'S': '#6495ED', 'I': '#D55E00', 'R': '#009E73',
    'Ola 1': '#808080',
    'Ola 2': '#0072B2',
    'Ola 3': '#CC79A7',
    'Ola 4': '#E69F00',
    'Ola 5': '#56B4E9',  # <-- AÑADIDO: Celeste para Ola 5
    'Ola 6': '#D35400',  # <-- AÑADIDO: Naranja calabaza para Ola 6
    'Ejes': '#333333', 'Texto': '#000000', 'Fondo': '#FFFFFF',
    'Contexto': '#708090', 'Dato Crudo': '#B0B0B0'
}
```

```python
# 2. Configuración Global de Matplotlib (rcParams)
plt.rcParams.update({
    'font.family': 'serif',
    'font.serif': ['Computer Modern Roman', 'Times New Roman'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'legend.fontsize': 10,
    'lines.linewidth': 2.0,
    'axes.linewidth': 1.0,
    'grid.color': '#cccccc',
    'grid.linestyle': ':',
    'axes.spines.right': False,
    'axes.spines.top': False,
    'xtick.direction': 'in',
    'ytick.direction': 'in',
    'legend.frameon': False,
    'figure.dpi': 300,
    'savefig.dpi': 300,
    'savefig.bbox': 'tight'
})

# ==============================================================================
# --- FIN: PLANTILLA DE ESTILO ---
# ==============================================================================


# --- 1. CARGA Y PREPROCESAMIENTO DE DATOS ---
print("Cargando y preprocesando datos...")
df_raw = pd.read_csv('Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv')
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
casos_diarios = df_nacional.iloc[:, 3:].T
casos_diarios.columns = ['casos_diarios']
casos_diarios.index = pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
casos_diarios['casos_suavizados'] = casos_diarios['casos_diarios'].rolling(
    window=7, center=True, min_periods=1
).mean()


# --- 2. ANÁLISIS ADICIONAL: DETECCIÓN DE OLAS ---
print("Detectando olas epidemiológicas para añadir contexto...")
```

```
indices_valles, _ = find_peaks(-casos_diarios['casos_suavizados'],␣
 ↪distance=100, prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(casos_diarios) - 1])))
olas_definidas = {}
# <-- MODIFICADO: Se extiende la lista para incluir las 6 olas
nombres_olas = ['Ola 1', 'Ola 2', 'Ola 3', 'Ola 4', 'Ola 5', 'Ola 6']
for i in range(len(indices_valles_completos) - 1):
    if len(olas_definidas) >= len(nombres_olas):
        break
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i + 1]
    datos_ola_slice = casos_diarios['casos_suavizados'].iloc[inicio_idx:fin_idx]
    if datos_ola_slice.empty or datos_ola_slice.max() < 1000:
        continue

    current_ola_name = nombres_olas[len(olas_definidas)]
    olas_definidas[current_ola_name] = {
        'inicio': casos_diarios.index[inicio_idx],
        'fin': casos_diarios.index[fin_idx],
        'color': PALETA[current_ola_name]
    }


# --- 3. CREACIÓN DE LA FIGURA CON CONTEXTO COMPLETO ---
print("Generando figura final...")
fig, ax = plt.subplots(figsize=(12, 6))

# Capa 1: Sombreado de las olas epidemiológicas para dar contexto
for nombre, info in olas_definidas.items():
    ax.axvspan(info['inicio'], info['fin'], color=info['color'], alpha=0.15,␣
 ↪zorder=0, ec='none', label=nombre)

# Capa 2: Graficar los datos diarios originales
ax.plot(casos_diarios.index, casos_diarios['casos_diarios'],
        color=PALETA['Dato Crudo'], alpha=0.6, linewidth=0.6,
        label='Casos diarios reportados')

# Capa 3: Graficar la tendencia suavizada
ax.plot(casos_diarios.index, casos_diarios['casos_suavizados'],
        color=PALETA['I'],  # Color 'Infectados' de nuestra paleta
        linewidth=2.2,
        label=r'Tendencia (media móvil 7 días)')

# --- Títulos, etiquetas y formato refinado ---
ax.set_title(r'\textbf{Preprocesamiento de la Serie Temporal de Casos Diarios␣
 ↪en México}', pad=15)
```

```
ax.set_xlabel('Año')
ax.set_ylabel('Número de Casos')
ax.grid(True, axis='y')

# Leyenda con orden personalizado y más columnas
handles, labels = ax.get_legend_handles_labels()
# Orden: Tendencia, Datos Crudos, Olas...
order = [labels.index(r'Tendencia (media móvil 7 días)'), labels.index('Casos␣
 ↪diarios reportados')] + \
        [i for i, s in enumerate(labels) if 'Ola' in s]
ax.legend([handles[i] for i in order], [labels[i] for i in order], loc='upper␣
 ↪left', ncol=4)  # <-- MODIFICADO: 4 columnas para mejor ajuste

# Formato de ejes
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/
 ↪1000):,}k' if x > 0 else '0'))
ax.set_xlim([casos_diarios.index.min(), casos_diarios.index.max()])
ax.set_ylim(bottom=0)

# --- Guardar la figura ---
fig.savefig('Figura_Suavizado_Con_Contexto_Completo.pdf')
fig.savefig('Figura_Suavizado_Con_Contexto_Completo.png', dpi=300)

print(" Figura de suavizado con contexto de 6 olas guardada.")
plt.show()
```

```
Cargando y preprocesando datos…
Detectando olas epidemiológicas para añadir contexto…
Generando figura final…
 Figura de suavizado con contexto de 6 olas guardada.
```



[85]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.ticker as mticker
import matplotlib.patches as mpatches

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO PARA PUBLICACIÓN CIENTÍFICA ---
# ==============================================================================

def setup_publication_style():
    """Configura el estilo de Matplotlib para una estética elegante y␣
 ↪profesional."""
    plt.rcParams.update({
        'font.family': 'serif',
        'font.serif': ['Computer Modern Roman', 'Times New Roman'],
        'text.usetex': True,
        'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 10, 'axes.labelsize': 11, 'axes.titlesize': 13,
        'xtick.labelsize': 9, 'ytick.labelsize': 9, 'legend.fontsize': 9,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.6,
        'axes.spines.right': False, 'axes.spines.top': False,
        'xtick.direction': 'in', 'ytick.direction': 'in',
        'savefig.dpi': 300, 'savefig.format': 'pdf', 'savefig.bbox': 'tight'
    })
```

```
setup_publication_style()

# ======================================================================
# --- PARÁMETROS, DATOS Y DEFINICIÓN DE OLAS ---
# ======================================================================
CONFIG = {'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv', 'ventana_suavizado': 7}
PALETTE = {'casos_reportados': '#0072B2', 'casos_reales': '#D55E00',␣
 ↪'diferencia': '#E69F00',
           'olas': ['#2a9d8f', '#e9c46a', '#f4a261', '#e76f51', '#d62828',␣
 ↪'#8e9aaf']}

print("Cargando y preprocesando datos...")
df_raw = pd.read_csv(CONFIG['archivo_datos'])
casos_diarios = df_raw[df_raw['nombre'] == 'Nacional'].iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_raw.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()

print("Definiendo olas con el método 'Valle a Valle'...")
from scipy.signal import find_peaks
indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
nombres_olas = ['Ola 1 (Original)', 'Ola 2 (Alpha)', 'Ola 3 (Delta)', 'Ola 4␣
 ↪(Ómicron)', 'Ola 5 (Ómicron Subs.)', 'Ola 6']
for i in range(len(indices_valles_completos) - 1):
    if i >= len(nombres_olas): break
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
    datos_ola_slice = datos['casos_suavizados'].iloc[inicio_idx:fin_idx]
    if not datos_ola_slice.empty and datos_ola_slice.max() > 1000:
        olas_definidas[nombres_olas[len(olas_definidas)]] = {'inicio': datos.
 ↪index[inicio_idx], 'fin': datos.index[fin_idx]}

# ======================================================================
# --- APLICACIÓN DEL AJUSTE POR SUBREGISTRO ---
# ======================================================================
print("Aplicando factores de detección validados...")
factores_deteccion = {
    'Ola 1 (Original)': 0.024, 'Ola 2 (Alpha)': 0.024, 'Ola 3 (Delta)': 0.050,
    'Ola 4 (Ómicron)': 0.059, 'Ola 5 (Ómicron Subs.)': 0.059, 'Ola 6': 0.059
}
```

```
datos['casos_ajustados'] = np.nan
for nombre_ola, info in olas_definidas.items():
    fd = factores_deteccion[nombre_ola]
    datos.loc[info['inicio']:info['fin'], 'casos_ajustados'] = datos.
 ↪loc[info['inicio']:info['fin'], 'casos_suavizados'] / fd
datos['casos_ajustados'] = datos['casos_ajustados'].ffill()

# ======================================================================
# --- FUNCIÓN AUXILIAR PARA DIBUJAR LÍNEAS CON "CORTE DE COLOR" ---
# ======================================================================
def plot_line_by_wave(ax, data, column_name, color_palette):
    """Dibuja una línea en segmentos, coloreando cada segmento según la ola."""
    for i, (nombre_ola, info) in enumerate(olas_definidas.items()):
        wave_data = data.loc[info['inicio']:info['fin']]
        ax.plot(wave_data.index, wave_data[column_name], color=color_palette[i␣
 ↪% len(color_palette)])

# ======================================================================
# --- GENERACIÓN DE GRÁFICAS ---
# ======================================================================

# --- Gráfica 1: Versión Logarítmica ---
fig_log, (ax1_log, ax2_log) = plt.subplots(2, 1, figsize=(10, 8),␣
 ↪gridspec_kw={'height_ratios': [3, 1]})
fig_log.suptitle(r'\textbf{Figura 3.2a: Ajuste por Subregistro (Escala␣
 ↪Logarítmica)}', y=0.99)
ax1_log.set_title(r'\textbf{A)} Comparativa de Casos Reportados vs. Estimación␣
 ↪de Infecciones Reales', loc='left')

# --- APLICACIÓN DEL CORTE DE COLOR ---
plot_line_by_wave(ax1_log, datos, 'casos_suavizados', PALETTE['olas'])
ax1_log.plot(datos.index, datos['casos_ajustados'],␣
 ↪color=PALETTE['casos_reales'], lw=1.5, ls='--', label='Infecciones Reales␣
 ↪(estimadas)')
ax1_log.set_yscale('log')
ax1_log.set_ylabel('Número de Casos Diarios (Log)'); ax1_log.grid(True)
ax1_log.set_xlim(datos.index.min(), datos.index.max())
plt.setp(ax1_log.get_xticklabels(), visible=False)
# Leyenda Manual para claridad
legend_elements = [plt.Line2D([0], [0], color=PALETTE['casos_reales'], lw=1.5,␣
 ↪ls='--', label='Infecciones Reales (estimadas)'),
                   mpatches.Patch(color='gray', alpha=0.5, label='Casos␣
 ↪Confirmados (coloreado por ola)')]
ax1_log.legend(handles=legend_elements)
```

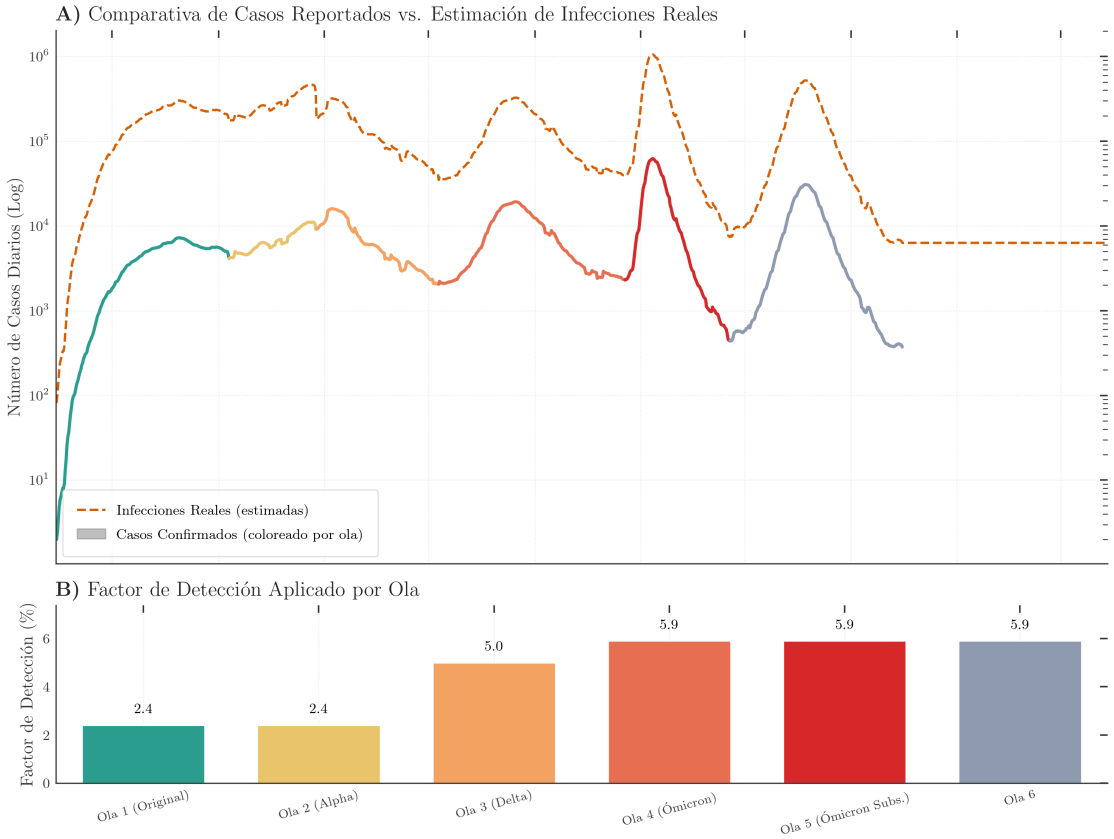
```
ax2_log.set_title(r'\textbf{B)} Factor de Detección Aplicado por Ola',␣
 ↪loc='left')
nombres_olas_lista = list(olas_definidas.keys())
valores_fd = [factores_deteccion[n] * 100 for n in nombres_olas_lista]
colores_barras = [PALETTE['olas'][i % len(PALETTE['olas'])] for i in␣
 ↪range(len(nombres_olas_lista))]
ax2_log.bar(nombres_olas_lista, valores_fd, color=colores_barras,␣
 ↪edgecolor='white', width=0.7)
for i, val in enumerate(valores_fd):
    ax2_log.text(i, val + 0.5, f'{val:.1f}%', ha='center', fontsize=9,␣
 ↪weight='bold')
ax2_log.set_ylabel(r'Factor de Detección (\%)'); ax2_log.tick_params(axis='x',␣
 ↪labelrotation=15, labelsize=8)
ax2_log.grid(axis='y'); ax2_log.set_ylim(0, max(valores_fd) * 1.25)
ax2_log.set_xlim(-0.5, len(nombres_olas_lista) - 0.5)
plt.subplots_adjust(left=0.1, right=0.95, top=0.9, bottom=0.15, hspace=0.1)
fig_log.savefig('Figura_3_2_Ajuste_Subregistro_Log.pdf')
print("\n  Figura de Ajuste por Subregistro (Logarítmica y con corte de color)␣
 ↪guardada.")
plt.show()


# --- Gráfica 2: Versión Lineal ---
fig_lin, (ax1_lin, ax2_lin) = plt.subplots(2, 1, figsize=(10, 8),␣
 ↪gridspec_kw={'height_ratios': [3, 1]})
fig_lin.suptitle(r'\textbf{Figura 3.2b: Ajuste por Subregistro (Escala␣
 ↪Lineal)}', y=0.99)
ax1_lin.set_title(r'\textbf{A)} Comparativa de Casos Reportados vs. Estimación␣
 ↪de Infecciones Reales', loc='left')

# --- APLICACIÓN DEL CORTE DE COLOR ---
plot_line_by_wave(ax1_lin, datos, 'casos_suavizados', PALETTE['olas'])
ax1_lin.plot(datos.index, datos['casos_ajustados'],␣
 ↪color=PALETTE['casos_reales'], lw=1.5, ls='--', label='Infecciones Reales␣
 ↪(estimadas)')
ax1_lin.set_ylabel('Número de Casos Diarios'); ax1_lin.grid(True)
ax1_lin.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/
 ↪1000)}k' if x > 0 else 0))
ax1_lin.set_xlim(datos.index.min(), datos.index.max())
plt.setp(ax1_lin.get_xticklabels(), visible=False)
ax1_lin.legend(handles=legend_elements) # Reutilizar la leyenda manual

ax2_lin.set_title(r'\textbf{B)} Factor de Detección Aplicado por Ola',␣
 ↪loc='left')
ax2_lin.bar(nombres_olas_lista, valores_fd, color=colores_barras,␣
 ↪edgecolor='white', width=0.7)
```

```
for i, val in enumerate(valores_fd):
    ax2_lin.text(i, val + 0.5, f'{val:.1f}%', ha='center', fontsize=9,␣
 ↪weight='bold')
ax2_lin.set_ylabel(r'Factor de Detección (\%)'); ax2_lin.tick_params(axis='x',␣
 ↪labelrotation=15, labelsize=8)
ax2_lin.grid(axis='y'); ax2_lin.set_ylim(0, max(valores_fd) * 1.25)
ax2_lin.set_xlim(-0.5, len(nombres_olas_lista) - 0.5)
plt.subplots_adjust(left=0.1, right=0.95, top=0.9, bottom=0.15, hspace=0.1)
fig_lin.savefig('Figura_3_2_Ajuste_Subregistro_Lineal.pdf')
print("\n  Figura de Ajuste por Subregistro (Lineal y con corte de color)␣
 ↪guardada.")
plt.show()
```

```
Cargando y preprocesando datos…
Definiendo olas con el método 'Valle a Valle'…
Aplicando factores de detección validados…
```

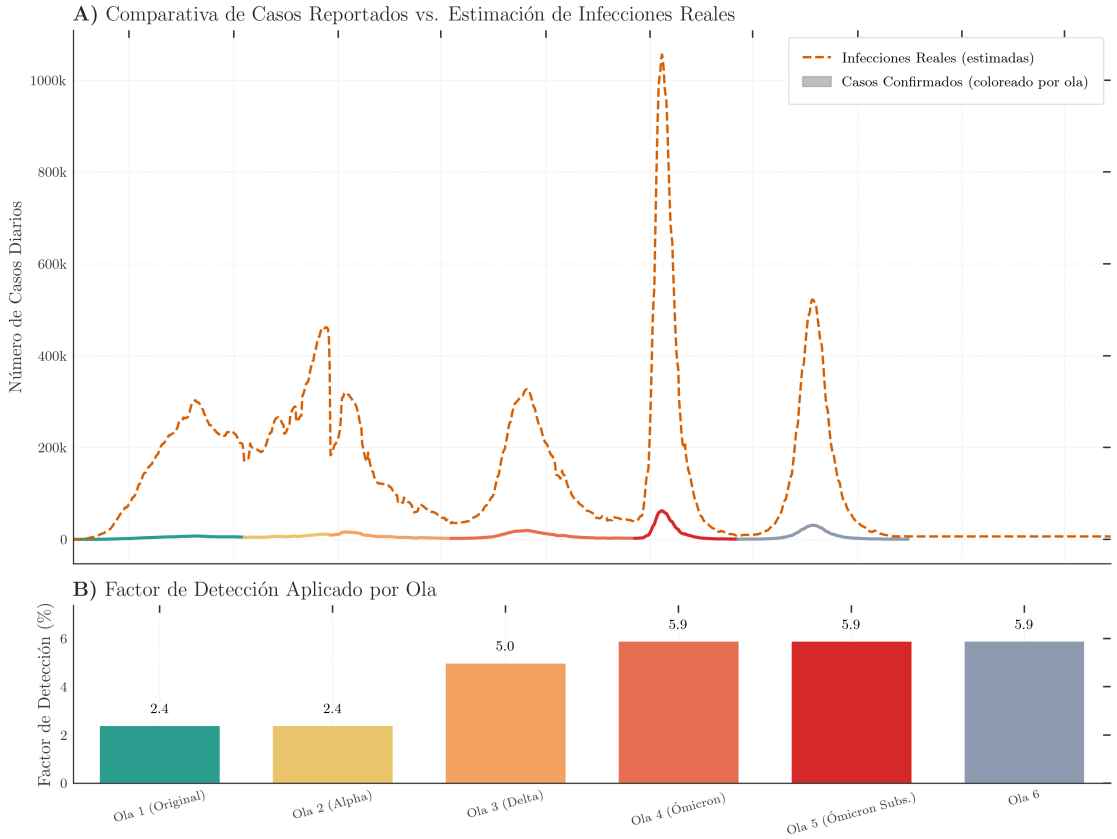
```
  Figura de Ajuste por Subregistro (Logarítmica y con corte de color) guardada.
```

Figura 3.2a: Ajuste por Subregistro (Escala Logarítmica)



```
Figura de Ajuste por Subregistro (Lineal y con corte de color) guardada.
```

Figura 3.2b: Ajuste por Subregistro (Escala Lineal)



[90]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from textwrap import wrap

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO PARA PUBLICACIÓN CIENTÍFICA ---
# ==============================================================================

def setup_publication_style():
    """Configura el estilo de Matplotlib para una estética elegante y␣
 ↪profesional."""
    plt.rcParams.update({
        'font.family': 'serif',
        'font.serif': ['Computer Modern Roman', 'Times New Roman'],
        'text.usetex': True,
```

```
        'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}\usepackage{graphicx}',
        'font.size': 8,  # Ajustado para una tabla más densa
        'figure.titlesize': 16,
        'savefig.dpi': 300,
        'savefig.format': 'pdf',
        'savefig.bbox': 'tight'
    })

setup_publication_style()
PALETTE = {'header': '#0d3b66', 'olas': ['#2a9d8f', '#e9c46a', '#f4a261',␣
 ↪'#e76f51', '#d62828', '#8e9aaf']}

# ======================================================================
# --- DATOS DE LOS PARÁMETROS POR OLA CON FUENTES INTEGRADAS ---
# ======================================================================

data = {
    'Ola': [r'Ola 1', r'Ola 2', r'Ola 3', r'Ola 4', r'Ola 5', r'Ola 6'],
    'Variante Dominante': [
        'Linajes Ancestrales', 'Alpha, Gamma, B.1.1.519', 'Delta (B.1.617.2)',
        'Ómicron (BA.1)', 'Ómicron (BA.4/BA.5)', 'Ómicron (BQ.1, XBB.1.5)'
    ],
    'Periodo Infeccioso (D)': [
        r'10 días' + '\n' + r'\textit{\tiny (Gries, et al., 2021)}',
        r'11 días' + '\n' + r'\textit{\tiny (Rodríguez-Morales, et al., 2021)}',
        r'9 días' + '\n' + r'\textit{\tiny (Singh, et al., 2021)}',
        r'7 días' + '\n' + r'\textit{\tiny (Lee, et al., 2022)}',
        r'6 días' + '\n' + r'\textit{\tiny (Iwata, et al., 2023)}',
        r'6 días' + '\n' + r'\textit{\tiny (Rosen, A., 2023)}'
    ],
    'Fraccion de Susceptibles S0/N': [
        r'1.000' + '\n' + r'\textit{\tiny (Población sin inmunidad)}',
        r'0.740' + '\n' + r'\textit{\tiny (Post-Ola 1, pre-vacunación)}',
        r'0.471' + '\n' + r'\textit{\tiny (Inmunidad mixta, post-vac.)}',
        r'0.701' + '\n' + r'\textit{\tiny (Evasión inmune de Ómicron)}',
        r'0.416' + '\n' + r'\textit{\tiny (Re-susceptibilidad a BA.5)}',
        r'0.315' + '\n' + r'\textit{\tiny (Inmunidad acumulada Ómicron)}'
    ],
    'Factor de Deteccion': [
        r'2.4\%' + '\n' + r'\textit{\tiny (ENSANUT 2020)}',
        r'2.4\%' + '\n' + r'\textit{\tiny (Extrapolación pre-ENSANUT 2021)}',
        r'5.0\%' + '\n' + r'\textit{\tiny (ENSANUT 2021)}',
        r'5.9\%' + '\n' + r'\textit{\tiny (Transición a ENSANUT 2022)}',
        r'5.9\%' + '\n' + r'\textit{\tiny (ENSANUT 2022)}',
        r'5.9\%' + '\n' + r'\textit{\tiny (Extrapolación post-ENSANUT 2022)}'
    ]
```

```
}

df = pd.DataFrame(data)

# ================================================================
# --- GENERACIÓN DE LA TABLA COMO IMAGEN ---
# ================================================================

fig = plt.figure(figsize=(12, 4.5))
ax = plt.subplot(111)
ax.axis('tight')
ax.axis('off')

# Definir las cabeceras con los nombres solicitados
col_labels = [
    r'\textbf{Ola Epidémica}', r'\textbf{Variante Dominante}',␣
 ↪r'\textbf{Periodo Infeccioso (D)}',
    r'\textbf{Fracción Susceptible ($S_0/N$)}', r'\textbf{Factor de Detección␣
 ↪(FD)}'
]

the_table = ax.table(
    cellText=df.values,
    colLabels=col_labels,
    loc='center',
    cellLoc='center',
    colWidths=[0.1, 0.2, 0.18, 0.22, 0.22]
)

# Estilo de la tabla
the_table.auto_set_font_size(False)
the_table.set_fontsize(8)
the_table.scale(1, 2.5)

# Estilo de la cabecera y celdas
for (i, j), cell in the_table.get_celld().items():
    if i == 0: # Cabecera
        cell.set_text_props(weight='bold', color='white')
        cell.set_facecolor(PALETTE['header'])
    else: # Filas de datos
        # --- AÑADIDO: Señalización de Olas con Color ---
        if j == 0:
            cell.get_text().set_color(PALETTE['olas'][i-1])
            cell.get_text().set_weight('bold')

fig.suptitle(r'\textbf{Tabla 3.X: Parámetros Epidemiológicos por Ola para el␣
 ↪Modelo SIR}', y=1.02, fontsize=16)
```

```
plt.savefig('Tabla_Parametros_R0_Completa.pdf')
plt.savefig('Tabla_Parametros_R0_Completa.png', dpi=300)

print(" Tabla de parámetros completa y detallada guardada.")
plt.show()
```

```
Tabla de parámetros completa y detallada guardada.
```

Tabla 3.X: Parámetros Epidemiológicos por Ola para el Modelo SIR

Ola Epidémica	Variante Dominante	Periodo Infeccioso (D)	Fracción Susceptible (S_0/N)	Factor de Detección (FD)
Ola 1	Linajes Ancestrales	10 días *(Gries, et al., 2021)*	1.000 *(Población sin inmunidad)*	2.4% *(ENSANUT 2020)*
Ola 2	Alpha, Gamma, B.1.1.519	11 días *(Rodríguez-Morales, et al., 2021)*	0.740 *(Post-Ola 1, pre-vacunación)*	2.4% *(Extrapolación pre-ENSANUT 2021)*
Ola 3	Delta (B.1.617.2)	9 días *(Singh, et al., 2021)*	0.471 *(Inmunidad mixta, post-vac.)*	5.0% *(ENSANUT 2021)*
Ola 4	Ómicron (BA.1)	7 días *(Lee, et al., 2022)*	0.701 *(Evasión inmune de Ómicron)*	5.9% *(Transición a ENSANUT 2022)*
Ola 5	Ómicron (BA.4/BA.5)	6 días *(Ivata, et al., 2023)*	0.416 *(Re-susceptibilidad a BA.5)*	5.9% *(ENSANUT 2022)*
Ola 6	Ómicron (BQ.1, XBB.1.5)	6 días *(Rosen, A., 2023)*	0.315 *(Inmunidad acumulada Ómicron)*	5.9% *(Extrapolación post-ENSANUT 2022)*

[92]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats, optimize

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO Y PARÁMETROS VALIDADOS ---
# ==============================================================================
def setup_publication_style():
    plt.rcParams.update({
        'font.family': 'serif', 'font.serif': ['Computer Modern Roman', 'Times␣
 ↪New Roman'],
        'text.usetex': True, 'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 10, 'axes.labelsize': 12, 'axes.titlesize': 14,
        'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 10,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.6,
        'axes.spines.right': False, 'axes.spines.top': False,
        'xtick.direction': 'in', 'ytick.direction': 'in',
        'savefig.dpi': 300, 'savefig.format': 'pdf', 'savefig.bbox': 'tight'
    })
setup_publication_style()
```

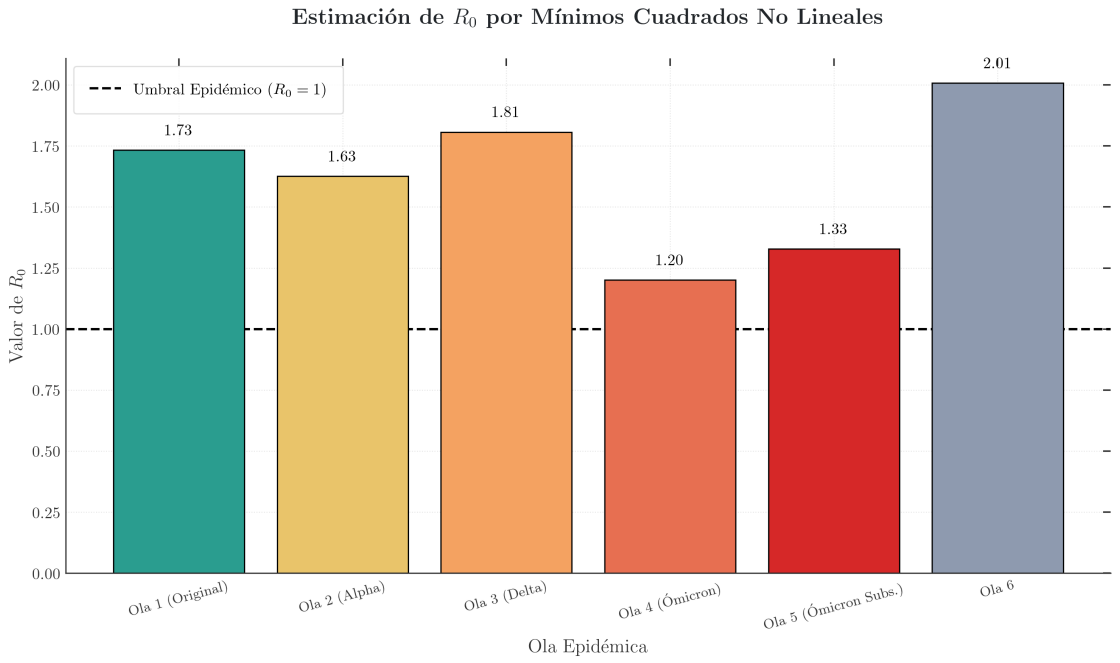
```
# Parámetros validados de tu investigación
PARAMETROS_OLAS = {
    'Ola 1 (Original)': {'D': 10, 'S0/N': 1.000, 'FD': 0.024, 'color':␣
 ↪'#2a9d8f'},
    'Ola 2 (Alpha)':    {'D': 11, 'S0/N': 0.740, 'FD': 0.024, 'color':␣
 ↪'#e9c46a'},
    'Ola 3 (Delta)':    {'D': 9,  'S0/N': 0.471, 'FD': 0.050, 'color':␣
 ↪'#f4a261'},
    'Ola 4 (Ómicron)':  {'D': 7,  'S0/N': 0.701, 'FD': 0.059, 'color':␣
 ↪'#e76f51'},
    'Ola 5 (Ómicron Subs.)': {'D': 6, 'S0/N': 0.416, 'FD': 0.059, 'color':␣
 ↪'#d62828'},
    'Ola 6':            {'D': 6,  'S0/N': 0.315, 'FD': 0.059, 'color':␣
 ↪'#8e9aaf'}
}
CONFIG = {'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv', 'ventana_suavizado': 7}

# ======================================================================
# --- CARGA DE DATOS Y DEFINICIÓN DE OLAS ---
# ======================================================================
print("Cargando datos y definiendo olas...")
df_raw = pd.read_csv(CONFIG['archivo_datos'])
casos_diarios = df_raw[df_raw['nombre'] == 'Nacional'].iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_raw.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()

from scipy.signal import find_peaks
indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
# --- CORRECCIÓN: Se añade una comprobación para no exceder el número de olas␣
 ↪con parámetros definidos ---
nombres_olas_lista = list(PARAMETROS_OLAS.keys())
for i in range(len(indices_valles_completos) - 1):
    if i >= len(nombres_olas_lista):
        break  # Detener el bucle si hemos procesado todas las olas definidas

    nombre_ola = nombres_olas_lista[i]
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
```

```
    olas_definidas[nombre_ola] = {'inicio': datos.index[inicio_idx], 'fin':␣
 ↪datos.index[fin_idx]}

# ================================================================
# --- FUNCIONES DE AJUSTE (NLS) ---
# ================================================================
def modelo_sir_exp(t, r_eff, i0, t_inf):
    gamma = 1 / t_inf
    beta = r_eff * gamma
    return i0 * np.exp((beta - gamma) * t)

def ajustar_modelo_nls(t, casos, t_inf):
    if np.any(casos <= 0) or len(casos) < 2: return np.nan
    try:
        params, _ = optimize.curve_fit(
            lambda t_fit, r_eff, i0: modelo_sir_exp(t_fit, r_eff, i0, t_inf),
            t, casos, p0=[2.0, casos[0]], bounds=([0.5, 1], [20.0, casos.max()␣
 ↪* 5])
        )
        return params[0]
    except (RuntimeError, ValueError):
        return np.nan

def encontrar_fase_exponencial(casos, window_size=21):
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos))
    max_r2 = -1; best_start = 0
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        _, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2:
            max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

# ================================================================
# --- CÁLCULO PRINCIPAL DE R0 POR OLA (NLS) ---
# ================================================================
print("\nCalculando R0 para cada ola con Mínimos Cuadrados No Lineales...")
resultados_r0_nls = []

for nombre_ola, info in olas_definidas.items():
    casos_ola = datos.loc[info['inicio']:info['fin']]['casos_suavizados']
    params = PARAMETROS_OLAS[nombre_ola]

    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    t_exp = np.arange(len(casos_exp))
```

```
    casos_corr = casos_exp / params['FD']

    reff_nls = ajustar_modelo_nls(t_exp, casos_corr.values, params['D'])

    R0 = reff_nls / params['S0/N']

    resultados_r0_nls.append({'Ola': nombre_ola, 'R0_NLS': R0})
    print(f" -> {nombre_ola}: R0 = {R0:.3f}")

# ======================================================================
# --- PRESENTACIÓN DE RESULTADOS ---
# ======================================================================
df_resultados = pd.DataFrame(resultados_r0_nls)

print("\n" + "="*80 + "\nTabla de Resultados: R0 por Mínimos Cuadrados No␣
 ↪Lineales\n" + "="*80)
print(df_resultados.to_string(index=False))

# --- Gráfica de Resultados ---
fig, ax = plt.subplots(figsize=(10, 6))
colors = [PARAMETROS_OLAS[n]['color'] for n in df_resultados['Ola']]
bars = ax.bar(df_resultados['Ola'], df_resultados['R0_NLS'], color=colors,␣
 ↪edgecolor='black', lw=0.8, zorder=2)

ax.axhline(1.0, color='black', linestyle='--', lw=1.5, label='Umbral Epidémico␣
 ↪($R_0=1$)', zorder=1)
ax.set_title(r'\textbf{Estimación de $R_0$ por Mínimos Cuadrados No Lineales}',␣
 ↪y=1.05)
ax.set_ylabel(r'Valor de $R_0$'); ax.set_xlabel('Ola Epidémica')
ax.tick_params(axis='x', labelrotation=15, labelsize=9)
ax.legend()
ax.grid(axis='y', linestyle=':', color='#cccccc')

for bar in bars:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2.0, yval + 0.05, f'{yval:.2f}',␣
 ↪ha='center', va='bottom', weight='bold')

plt.savefig('Figura_R0_NLS.pdf')
print("\n  Figura con los valores de R0 (NLS) guardada.")
plt.show()
```

```
Cargando datos y definiendo olas…

Calculando R0 para cada ola con Mínimos Cuadrados No Lineales…
 -> Ola 1 (Original): R0 = 1.733
```

```
 -> Ola 2 (Alpha): R0 = 1.626
 -> Ola 3 (Delta): R0 = 1.807
 -> Ola 4 (Ómicron): R0 = 1.201
 -> Ola 5 (Ómicron Subs.): R0 = 1.328
 -> Ola 6: R0 = 2.008

================================================================================
Tabla de Resultados: R0 por Mínimos Cuadrados No Lineales
================================================================================
                    Ola   R0_NLS
     Ola 1 (Original) 1.733451
        Ola 2 (Alpha) 1.626209
        Ola 3 (Delta) 1.806604
      Ola 4 (Ómicron) 1.200813
Ola 5 (Ómicron Subs.) 1.327619
                Ola 6 2.007720

  Figura con los valores de R0 (NLS) guardada.
```



[93]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats, optimize

# ================================================================================
```

```python
# --- INICIO: PLANTILLA DE ESTILO Y PARÁMETROS VALIDADOS ---
# ==============================================================================
def setup_publication_style():
    plt.rcParams.update({
        'font.family': 'serif', 'font.serif': ['Computer Modern Roman', 'Times␣
 ↪New Roman'],
        'text.usetex': True, 'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 10, 'axes.labelsize': 12, 'axes.titlesize': 14,
        'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 10,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.6,
        'axes.spines.right': False, 'axes.spines.top': False,
        'xtick.direction': 'in', 'ytick.direction': 'in',
        'savefig.dpi': 300, 'savefig.format': 'pdf', 'savefig.bbox': 'tight'
    })
setup_publication_style()

# Parámetros validados de tu investigación
PARAMETROS_OLAS = {
    'Ola 1 (Original)': {'D': 10, 'S0/N': 1.000, 'FD': 0.024, 'color':␣
 ↪'#2a9d8f'},
    'Ola 2 (Alpha)':    {'D': 11, 'S0/N': 0.740, 'FD': 0.024, 'color':␣
 ↪'#e9c46a'},
    'Ola 3 (Delta)':    {'D': 9,  'S0/N': 0.471, 'FD': 0.050, 'color':␣
 ↪'#f4a261'},
    'Ola 4 (Ómicron)':  {'D': 7,  'S0/N': 0.701, 'FD': 0.059, 'color':␣
 ↪'#e76f51'},
    'Ola 5 (Ómicron Subs.)': {'D': 6, 'S0/N': 0.416, 'FD': 0.059, 'color':␣
 ↪'#d62828'},
    'Ola 6':            {'D': 6,  'S0/N': 0.315, 'FD': 0.059, 'color':␣
 ↪'#8e9aaf'}
}
CONFIG = {'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv', 'ventana_suavizado': 7}

# ==============================================================
# --- CARGA DE DATOS Y DEFINICIÓN DE OLAS ---
# ==============================================================
print("Cargando datos y definiendo olas...")
df_raw = pd.read_csv(CONFIG['archivo_datos'])
casos_diarios = df_raw[df_raw['nombre'] == 'Nacional'].iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_raw.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()
```

```
from scipy.signal import find_peaks
indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
nombres_olas_lista = list(PARAMETROS_OLAS.keys())
for i in range(len(indices_valles_completos) - 1):
    if i >= len(nombres_olas_lista): break
    nombre_ola = nombres_olas_lista[i]
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
    olas_definidas[nombre_ola] = {'inicio': datos.index[inicio_idx], 'fin':␣
 ↪datos.index[fin_idx]}

# ======================================================================
# --- FUNCIONES DE AJUSTE (MLE) ---
# ======================================================================
def modelo_sir_exp(t, r_eff, i0, t_inf):
    gamma = 1 / t_inf
    beta = r_eff * gamma
    return i0 * np.exp((beta - gamma) * t)

def negative_log_likelihood(params, t, casos, t_inf):
    """Función de verosimilitud logarítmica negativa para una distribución de␣
 ↪Poisson."""
    r_eff, i0 = params

    # El valor esperado (lambda de Poisson) es la predicción del modelo
    valor_esperado = modelo_sir_exp(t, r_eff, i0, t_inf)

    # Evitar log(0) si la predicción es cero
    valor_esperado = np.maximum(1e-9, valor_esperado)

    # Log-verosimilitud de Poisson: sum(k*log(lambda) - lambda)
    log_likelihood = np.sum(casos * np.log(valor_esperado) - valor_esperado)

    # Devolvemos el negativo porque los optimizadores minimizan
    return -log_likelihood

def ajustar_modelo_mle(t, casos, t_inf):
    if np.any(casos <= 0) or len(casos) < 2: return np.nan

    # Adivinanza inicial para los parámetros [r_eff, i0]
    initial_guess = [2.0, casos[0]]
```

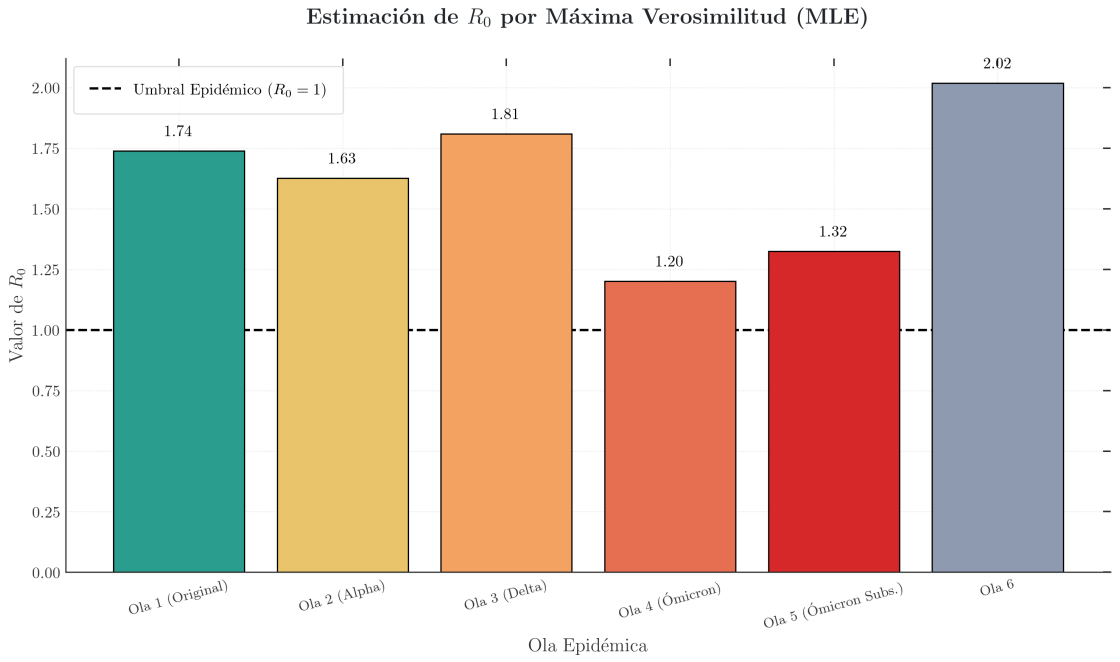
```
    # Usar scipy.optimize.minimize para encontrar los parámetros que minimizan␣
 ↪nuestra función
    result = optimize.minimize(
        fun=negative_log_likelihood,
        x0=initial_guess,
        args=(t, casos, t_inf),
        method='Nelder-Mead'
    )

    if result.success:
        return result.x[0]  # Retorna solo el R_eff optimizado
    else:
        return np.nan

def encontrar_fase_exponencial(casos, window_size=21):
    # (Función sin cambios, se mantiene la misma lógica para encontrar la fase)
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos))
    max_r2 = -1; best_start = 0
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        _, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2:
            max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

# ======================================================================
# --- CÁLCULO PRINCIPAL DE R0 POR OLA (MLE) ---
# ======================================================================
print("\nCalculando R0 para cada ola con Máxima Verosimilitud...")
resultados_r0_mle = []

for nombre_ola, info in olas_definidas.items():
    casos_ola = datos.loc[info['inicio']:info['fin']]['casos_suavizados']
    params = PARAMETROS_OLAS[nombre_ola]

    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    t_exp = np.arange(len(casos_exp))

    casos_corr = casos_exp / params['FD']

    # Asegurarse de que los casos sean enteros para el modelo de Poisson
    casos_corr_int = np.round(casos_corr.values).astype(int)

    reff_mle = ajustar_modelo_mle(t_exp, casos_corr_int, params['D'])
```

```
    R0 = reff_mle / params['S0/N']

    resultados_r0_mle.append({'Ola': nombre_ola, 'R0_MLE': R0})
    print(f" -> {nombre_ola}: R0 = {R0:.3f}")

# ======================================================================
# --- PRESENTACIÓN DE RESULTADOS ---
# ======================================================================
df_resultados = pd.DataFrame(resultados_r0_mle)

print("\n" + "="*80 + "\nTabla de Resultados: R0 por Máxima Verosimilitud\n" +␣
 ↪"="*80)
print(df_resultados.to_string(index=False))

# --- Gráfica de Resultados ---
fig, ax = plt.subplots(figsize=(10, 6))
colors = [PARAMETROS_OLAS[n]['color'] for n in df_resultados['Ola']]
bars = ax.bar(df_resultados['Ola'], df_resultados['R0_MLE'], color=colors,␣
 ↪edgecolor='black', lw=0.8, zorder=2)

ax.axhline(1.0, color='black', linestyle='--', lw=1.5, label='Umbral Epidémico␣
 ↪($R_0=1$)', zorder=1)
ax.set_title(r'\textbf{Estimación de $R_0$ por Máxima Verosimilitud (MLE)}',␣
 ↪y=1.05)
ax.set_ylabel(r'Valor de $R_0$'); ax.set_xlabel('Ola Epidémica')
ax.tick_params(axis='x', labelrotation=15, labelsize=9)
ax.legend()
ax.grid(axis='y', linestyle=':', color='#cccccc')

for bar in bars:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2.0, yval + 0.05, f'{yval:.2f}',␣
 ↪ha='center', va='bottom', weight='bold')

plt.savefig('Figura_R0_MLE.pdf')
print("\n  Figura con los valores de R0 (MLE) guardada.")
plt.show()
```

```
Cargando datos y definiendo olas…

Calculando R0 para cada ola con Máxima Verosimilitud…
 -> Ola 1 (Original): R0 = 1.739
 -> Ola 2 (Alpha): R0 = 1.626
 -> Ola 3 (Delta): R0 = 1.809
 -> Ola 4 (Ómicron): R0 = 1.200
 -> Ola 5 (Ómicron Subs.): R0 = 1.324
 -> Ola 6: R0 = 2.019
```

```
================================================================================
Tabla de Resultados: R0 por Máxima Verosimilitud
================================================================================
                    Ola   R0_MLE
     Ola 1 (Original) 1.739157
        Ola 2 (Alpha) 1.625963
        Ola 3 (Delta) 1.808760
      Ola 4 (Ómicron) 1.200482
Ola 5 (Ómicron Subs.) 1.324289
                Ola 6 2.018784
```

Figura con los valores de R0 (MLE) guardada.



[102]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats, optimize
from scipy.signal import find_peaks
from tqdm import tqdm
import warnings

# Opcional: Ignorar la advertencia de "tight_layout" de seaborn, que es␣
↪inofensiva
```

```
warnings.filterwarnings('ignore', category=UserWarning)

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO Y PARÁMETROS VALIDADOS ---
# ==============================================================================
def setup_publication_style():
    """Configura el estilo de las gráficas para una apariencia profesional."""
    plt.rcParams.update({
        'font.family': 'serif', 'font.serif': ['Computer Modern Roman', 'Times␣
 ↪New Roman'],
        'text.usetex': True, 'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 10, 'axes.labelsize': 12, 'axes.titlesize': 14,
        'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 10,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.6,
        'axes.spines.right': False, 'axes.spines.top': False,
        'xtick.direction': 'in', 'ytick.direction': 'in',
        'savefig.dpi': 300, 'savefig.format': 'pdf', 'savefig.bbox': 'tight'
    })
setup_publication_style()

# --- PARÁMETROS DEFINIDOS COMO DISTRIBUCIONES ---
def create_dist(mode, uncertainty=0.2):
    """Crea un diccionario para una distribución triangular."""
    return {'min': mode * (1 - uncertainty), 'mode': mode, 'max': mode * (1 +␣
 ↪uncertainty)}

PARAMETROS_OLAS_DIST = {
    'Ola 1 (Original)': {'D': create_dist(10), 'SO/N': create_dist(1.000, 0.
 ↪01), 'FD': create_dist(0.024), 'color': '#2a9d8f'},
    'Ola 2 (Alpha)':    {'D': create_dist(11), 'SO/N': create_dist(0.740), 'FD':
 ↪ create_dist(0.024), 'color': '#e9c46a'},
    'Ola 3 (Delta)':    {'D': create_dist(9),  'SO/N': create_dist(0.471), 'FD':
 ↪ create_dist(0.050), 'color': '#f4a261'},
    'Ola 4 (Ómicron)':  {'D': create_dist(7),  'SO/N': create_dist(0.701), 'FD':
 ↪ create_dist(0.059), 'color': '#e76f51'},
    'Ola 5 (Ómicron Subs.)': {'D': create_dist(6), 'SO/N': create_dist(0.416),␣
 ↪'FD': create_dist(0.059), 'color': '#d62828'},
    'Ola 6':            {'D': create_dist(6),  'SO/N': create_dist(0.315), 'FD':
 ↪ create_dist(0.059), 'color': '#8e9aaf'}
}
# --- NÚMERO DE SIMULACIONES AJUSTADO A 100,000 ---
CONFIG = {'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv', 'ventana_suavizado': 7, 'N_SIM': 100000}
```

```
# ======================================================================
# --- CARGA DE DATOS Y DEFINICIÓN DE OLAS ---
# ======================================================================
print("Cargando datos y definiendo olas...")
# Se asume que el archivo CSV está en el mismo directorio que el script
df_raw = pd.read_csv(CONFIG['archivo_datos'])
casos_diarios = df_raw[df_raw['nombre'] == 'Nacional'].iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_raw.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()

indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
nombres_olas_lista = list(PARAMETROS_OLAS_DIST.keys())
for i in range(len(indices_valles_completos) - 1):
    if i >= len(nombres_olas_lista): break
    nombre_ola = nombres_olas_lista[i]
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
    olas_definidas[nombre_ola] = {'inicio': datos.index[inicio_idx], 'fin':␣
 ↪datos.index[fin_idx]}

# ======================================================================
# --- FUNCIONES DE AJUSTE ---
# ======================================================================
def modelo_sir_exp(t, r_eff, i0, t_inf):
    gamma = 1 / t_inf; beta = r_eff * gamma
    return i0 * np.exp((beta - gamma) * t)

def ajustar_modelo_sir(t, casos, t_inf):
    try:
        params, _ = optimize.curve_fit(lambda t_fit, r_eff, i0:␣
 ↪modelo_sir_exp(t_fit, r_eff, i0, t_inf),
                                       t, casos, p0=[2.0, casos[0]], bounds=([0.
 ↪5, 1], [20.0, casos.max() * 5]))
        r_eff_opt, i0_opt = params
        pred = modelo_sir_exp(t, r_eff_opt, i0_opt, t_inf)
        ss_res = np.sum((casos - pred)**2); ss_tot = np.sum((casos - np.
 ↪mean(casos))**2)
        r2 = 1 - (ss_res / ss_tot) if ss_tot > 0 else 0
        return r_eff_opt, max(0, r2)
    except (RuntimeError, ValueError): return np.nan, 0
```

```
def modelo_exponencial(t, casos, t_inf):
    log_casos = np.log(casos); slope, _, r_value, _, _ = stats.linregress(t,␣
 ↪log_casos)
    return np.exp(slope * t_inf), r_value**2

def encontrar_fase_exponencial(casos, window_size=21):
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos)); max_r2 = -1; best_start = 0
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        _, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2: max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

# ======================================================================
# --- CÁLCULO PRINCIPAL DE RO (MONTE CARLO) CON BARRA DE PROGRESO ---
# ======================================================================
print(f"\nIniciando Simulación de Monte Carlo con {CONFIG['N_SIM']:,}␣
 ↪iteraciones por ola...")
resultados_distribuciones = {}

for nombre_ola, info in olas_definidas.items():
    casos_ola = datos.loc[info['inicio']:info['fin']]['casos_suavizados']
    params_dist = PARAMETROS_OLAS_DIST[nombre_ola]
    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    t_exp = np.arange(len(casos_exp))

    d_samples = np.random.triangular(params_dist['D']['min'],␣
 ↪params_dist['D']['mode'], params_dist['D']['max'], CONFIG['N_SIM'])
    fd_samples = np.random.triangular(params_dist['FD']['min'],␣
 ↪params_dist['FD']['mode'], params_dist['FD']['max'], CONFIG['N_SIM'])
    s0_n_samples = np.random.triangular(params_dist['S0/N']['min'],␣
 ↪params_dist['S0/N']['mode'], params_dist['S0/N']['max'], CONFIG['N_SIM'])

    r0_results = []
    for i in tqdm(range(CONFIG['N_SIM']), desc=f"Procesando {nombre_ola}",␣
 ↪ncols=100):
        casos_corr = casos_exp / fd_samples[i]
        reff_sir, r2_sir = ajustar_modelo_sir(t_exp, casos_corr.values,␣
 ↪d_samples[i])
        reff_exp, r2_exp = modelo_exponencial(t_exp, casos_corr.values,␣
 ↪d_samples[i])

        if np.isnan(reff_sir) or np.isnan(reff_exp) or (r2_sir + r2_exp == 0):
```

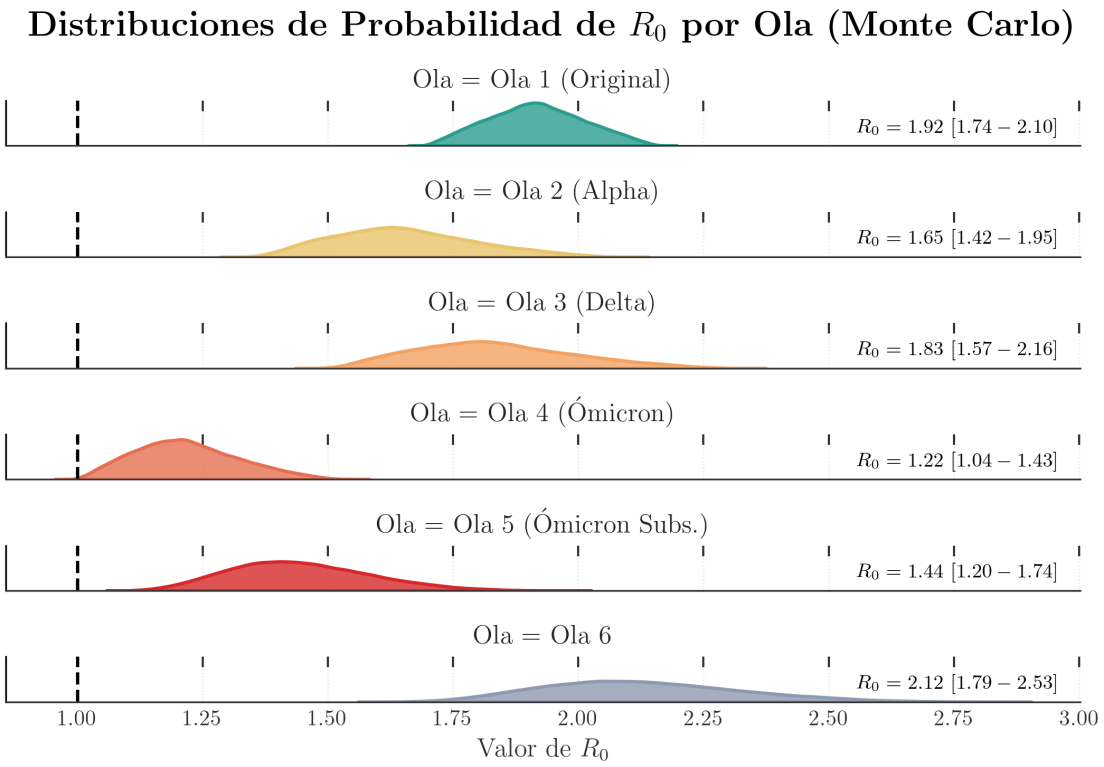
```
            reff_comb = np.nanmean([reff_sir, reff_exp])
        else:
            reff_comb = (reff_sir * r2_sir + reff_exp * r2_exp) / (r2_sir +␣
 ↪r2_exp)

        if not np.isnan(reff_comb):
            R0 = reff_comb / s0_n_samples[i]
            r0_results.append(R0)

    resultados_distribuciones[nombre_ola] = np.array(r0_results)
    print(f" -> {nombre_ola}: R0 medio final = {np.mean(r0_results):.2f}")

# ======================================================================
# --- PRESENTACIÓN DE RESULTADOS ---
# ======================================================================
stats_data = []
for nombre, dist in resultados_distribuciones.items():
    stats_data.append({
        'Ola': nombre, 'R0 Medio': np.mean(dist),
        'IC_inf': np.percentile(dist, 2.5), 'IC_sup': np.percentile(dist, 97.5),
        'Desv. Est.': np.std(dist)
    })
df_stats = pd.DataFrame(stats_data).set_index('Ola')

df_print = df_stats.copy()
df_print['R0 Medio'] = df_print['R0 Medio'].apply(lambda x: f'{x:.2f}')
df_print['Desv. Est.'] = df_print['Desv. Est.'].apply(lambda x: f'{x:.2f}')
df_print['Intervalo de Confianza 95%'] = df_print.apply(lambda row:␣
 ↪f'[{row["IC_inf"]:.2f} - {row["IC_sup"]:.2f}]', axis=1)
print("\n" + "="*80 + "\nTabla de Resultados: Estadísticas de R0 por Monte␣
 ↪Carlo\n" + "="*80)
print(df_print[['R0 Medio', 'Desv. Est.', 'Intervalo de Confianza 95%']].
 ↪to_string())

# --- Gráfica 1: Distribuciones de R0 (Ridgeline Plot con Anotaciones) ---
print("\nGenerando gráfica de distribuciones (Ridgeline Plot)...")
plot_data_long = pd.DataFrame(resultados_distribuciones).melt(var_name='Ola',␣
 ↪value_name='R0')
colores = [p['color'] for p in PARAMETROS_OLAS_DIST.values()]
orden_olas = list(PARAMETROS_OLAS_DIST.keys())
palette_dict = dict(zip(orden_olas, colores))

g = sns.FacetGrid(plot_data_long, row='Ola', hue='Ola', aspect=9, height=0.8,␣
 ↪palette=palette_dict, row_order=orden_olas)
g.map_dataframe(sns.kdeplot, x='R0', fill=True, alpha=0.8, lw=1.5)
```

```
def style_ridge(x, color, label):
    ax = plt.gca()
    ax.spines['left'].set_visible(True); ax.spines['left'].set_color('black');␣
 ↪ax.spines['left'].set_linewidth(0.8)
    ax.set_yticks([]); ax.set_ylabel('')
    ax.tick_params(axis='x', labelsize=10, direction='in')
    ax.set_ylabel(label, rotation=0, ha='right', va='center', fontsize=10,␣
 ↪weight='bold')
g.map(style_ridge, 'R0')

for i, ax in enumerate(g.axes.flat):
    nombre_ola = orden_olas[i]
    stats = df_stats.loc[nombre_ola]
    texto = fr'$R_0 = {stats["R0 Medio"]:.2f} \ [{stats["IC_inf"]:.2f} -␣
 ↪{stats["IC_sup"]:.2f}]$'
    ax.text(0.98, 0.4, texto, transform=ax.transAxes, ha='right', va='center',␣
 ↪fontsize=9)

g.refline(x=1, linestyle='--', color='k', linewidth=1.5)
g.fig.suptitle(r'\textbf{Distribuciones de Probabilidad de $R_0$ por Ola (Monte␣
 ↪Carlo)}', y=1.02, fontsize=16)
g.set_axis_labels(r'Valor de $R_0$', ''); g.tight_layout(); g.fig.
 ↪subplots_adjust(top=0.9)

plt.savefig('Figura_R0_MC_Distribuciones_Ridge.pdf')
print("\n  Figura con las distribuciones de R0 (Ridgeline Plot) guardada.")
plt.show()

# --- Gráfica 2: Resumen de R0 con Anotaciones ---
print("\nGenerando gráfica de resumen de R0...")
fig, ax = plt.subplots(figsize=(10, 6))
means = df_stats['R0 Medio']
cis = df_stats[['IC_inf', 'IC_sup']].values
errors = [[m - ci[0] for m, ci in zip(means, cis)], [ci[1] - m for m, ci in␣
 ↪zip(means, cis)]]
bars = ax.bar(df_stats.index, means, yerr=errors, color=colores,␣
 ↪edgecolor='black', lw=0.8, capsize=5)
ax.axhline(1.0, color='black', linestyle='--', lw=1.5, label='Umbral Epidémico␣
 ↪($R_0=1$)')

for i, bar in enumerate(bars):
    mean_val = means[i]
    y_val = cis[i][1]
    texto = f'{mean_val:.2f}'
    ax.text(bar.get_x() + bar.get_width() / 2.0, y_val + 0.05, texto,␣
 ↪ha='center', va='bottom', fontsize=9)
```

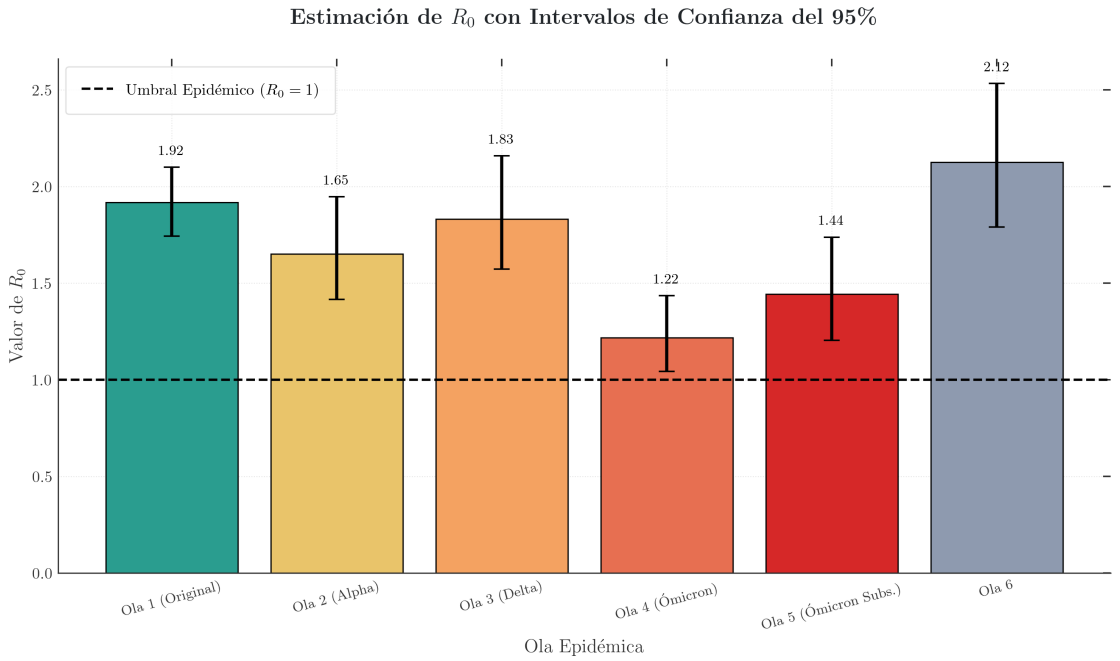
```
ax.set_title(r'\textbf{Estimación de $R_0$ con Intervalos de Confianza del␣
 ↪95\%}', y=1.05)
ax.set_ylabel(r'Valor de $R_0$'); ax.set_xlabel('Ola Epidémica')
ax.tick_params(axis='x', labelrotation=15, labelsize=9)
ax.legend()
plt.tight_layout()
plt.savefig('Figura_R0_MC_Resumen_Anotado.pdf')
print("\n  Figura de resumen de R0 (con anotaciones) guardada.")
plt.show()
```

```
Cargando datos y definiendo olas…

Iniciando Simulación de Monte Carlo con 100,000 iteraciones por ola…

Procesando Ola 1 (Original): 100%|              | 100000/100000
[01:08<00:00, 1457.24it/s]

 -> Ola 1 (Original): R0 medio final = 1.92

Procesando Ola 2 (Alpha): 100%|               | 100000/100000
[01:19<00:00, 1265.71it/s]

 -> Ola 2 (Alpha): R0 medio final = 1.65

Procesando Ola 3 (Delta): 100%|               | 100000/100000
[01:33<00:00, 1068.63it/s]

 -> Ola 3 (Delta): R0 medio final = 1.83

Procesando Ola 4 (Ómicron): 100%|              | 100000/100000
[01:35<00:00, 1048.62it/s]

 -> Ola 4 (Ómicron): R0 medio final = 1.22

Procesando Ola 5 (Ómicron Subs.): 100%|           | 100000/100000
[01:54<00:00, 870.61it/s]

 -> Ola 5 (Ómicron Subs.): R0 medio final = 1.44

Procesando Ola 6: 100%|                    | 100000/100000
[01:47<00:00, 932.72it/s]

 -> Ola 6: R0 medio final = 2.12

================================================================================
Tabla de Resultados: Estadísticas de R0 por Monte Carlo
================================================================================
                     R0 Medio Desv. Est. Intervalo de Confianza 95%
Ola
Ola 1 (Original)         1.92       0.09              [1.74 - 2.10]
Ola 2 (Alpha)            1.65       0.14              [1.42 - 1.95]
Ola 3 (Delta)            1.83       0.15              [1.57 - 2.16]
Ola 4 (Ómicron)          1.22       0.10              [1.04 - 1.43]
```

```
Ola 5 (Ómicron Subs.)     1.44        0.14              [1.20 - 1.74]
Ola 6                     2.12        0.19              [1.79 - 2.53]

Generando gráfica de distribuciones (Ridgeline Plot)…

  Figura con las distribuciones de R0 (Ridgeline Plot) guardada.
```




```
Generando gráfica de resumen de R0…

/var/folders/qk/4vl239v956x7jjxqgwkvz9g80000gn/T/ipykernel_1228/706180588.py:194
: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In
a future version, integer keys will always be treated as labels (consistent with
DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
  mean_val = means[i]


  Figura de resumen de R0 (con anotaciones) guardada.
```



[8]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats, optimize
from scipy.signal import find_peaks
from tqdm import tqdm
from matplotlib.patches import Patch

# ==============================================================================
# --- INICIO: PLANTILLA DE ESTILO Y PARÁMETROS VALIDADOS ---
# ==============================================================================
def setup_publication_style():
    """Configura el estilo de Matplotlib para gráficos de calidad de␣
 ↪publicación."""
    plt.rcParams.update({
        'font.family': 'serif', 'font.serif': ['Computer Modern Roman', 'Times␣
 ↪New Roman'],
        'text.usetex': True, 'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 10, 'axes.labelsize': 12, 'axes.titlesize': 14,
        'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 10,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.6,
        'axes.spines.right': False, 'axes.spines.top': False,
```

```
        'xtick.direction': 'in', 'ytick.direction': 'in',
        'savefig.dpi': 300, 'savefig.format': 'pdf', 'savefig.bbox': 'tight'
    })
setup_publication_style()

# --- PARÁMETROS DEFINIDOS COMO DISTRIBUCIONES ---
def create_dist(mode, uncertainty=0.2):
    """Crea un diccionario para una distribución triangular."""
    return {'min': mode * (1 - uncertainty), 'mode': mode, 'max': mode * (1 +␣
 ↪uncertainty)}

PARAMETROS_OLAS_DIST = {
    'Ola 1 (Original)': {'D': create_dist(10), 'S0/N': create_dist(1.000, 0.
 ↪01), 'FD': create_dist(0.024), 'color': '#2a9d8f'},
    'Ola 2 (Alpha)':    {'D': create_dist(11), 'S0/N': create_dist(0.740), 'FD':
 ↪ create_dist(0.024), 'color': '#e9c46a'},
    'Ola 3 (Delta)':    {'D': create_dist(9),  'S0/N': create_dist(0.471), 'FD':
 ↪ create_dist(0.050), 'color': '#f4a261'},
    'Ola 4 (Ómicron)':  {'D': create_dist(7),  'S0/N': create_dist(0.701), 'FD':
 ↪ create_dist(0.059), 'color': '#e76f51'},
    'Ola 5 (Ómicron Subs.)': {'D': create_dist(6), 'S0/N': create_dist(0.416),␣
 ↪'FD': create_dist(0.059), 'color': '#d62828'},
    'Ola 6':            {'D': create_dist(6),  'S0/N': create_dist(0.315), 'FD':
 ↪ create_dist(0.059), 'color': '#8e9aaf'}
}
CONFIG = {'archivo_datos': 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv', 'ventana_suavizado': 7, 'N_SIM': 100000}

# ======================================================================
# --- CARGA DE DATOS Y DEFINICIÓN DE OLAS ---
# ======================================================================
print("Cargando datos y definiendo olas...")
df_raw = pd.read_csv(CONFIG['archivo_datos'])
casos_diarios = df_raw[df_raw['nombre'] == 'Nacional'].iloc[0, 3:].astype(float)
datos = pd.DataFrame({'casos_diarios': casos_diarios.values}, index=pd.
 ↪to_datetime(df_raw.columns[3:], format='%d-%m-%Y'))
datos['casos_suavizados'] = datos['casos_diarios'].
 ↪rolling(CONFIG['ventana_suavizado'], center=True, min_periods=1).mean()

indices_valles, _ = find_peaks(-datos['casos_suavizados'], distance=100,␣
 ↪prominence=500)
indices_valles_completos = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(datos)-1])))
olas_definidas = {}
nombres_olas_lista = list(PARAMETROS_OLAS_DIST.keys())
for i in range(len(indices_valles_completos) - 1):
```

```
    if i >= len(nombres_olas_lista): break
    nombre_ola = nombres_olas_lista[i]
    inicio_idx, fin_idx = indices_valles_completos[i],␣
 ↪indices_valles_completos[i+1]
    olas_definidas[nombre_ola] = {'inicio': datos.index[inicio_idx], 'fin':␣
 ↪datos.index[fin_idx]}

# ======================================================================
# --- FUNCIONES DE AJUSTE ---
# ======================================================================
def modelo_sir_exp(t, r_eff, i0, t_inf):
    gamma = 1 / t_inf; beta = r_eff * gamma
    return i0 * np.exp((beta - gamma) * t)

def ajustar_modelo_sir(t, casos, t_inf):
    try:
        params, _ = optimize.curve_fit(lambda t_fit, r_eff, i0:␣
 ↪modelo_sir_exp(t_fit, r_eff, i0, t_inf),
                                       t, casos, p0=[2.0, casos[0]], bounds=([0.
 ↪5, 1], [20.0, casos.max() * 5]))
        r_eff_opt, i0_opt = params
        pred = modelo_sir_exp(t, r_eff_opt, i0_opt, t_inf)
        ss_res = np.sum((casos - pred)**2); ss_tot = np.sum((casos - np.
 ↪mean(casos))**2)
        r2 = 1 - (ss_res / ss_tot) if ss_tot > 0 else 0
        return r_eff_opt, max(0, r2)
    except (RuntimeError, ValueError): return np.nan, 0

def modelo_exponencial(t, casos, t_inf):
    if len(t) < 2 or np.any(casos <= 0): return np.nan, 0
    log_casos = np.log(casos); slope, _, r_value, _, _ = stats.linregress(t,␣
 ↪log_casos)
    if np.isnan(slope): return np.nan, 0
    return np.exp(slope * t_inf), r_value**2

def encontrar_fase_exponencial(casos, window_size=21):
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos)); max_r2 = -1; best_start = 0
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        slope, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2 and slope > 0:
            max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

# ======================================================================
```

```
# --- CÁLCULO PRINCIPAL CON MÉTODO HÍBRIDO ---
# ================================================================
print(f"\nIniciando Simulación Híbrida con {CONFIG['N_SIM']:,} iteraciones por␣
 ↪ola...")
resultados_distribuciones = {}
stats_por_ola = {}
fases_exponenciales = {}

for nombre_ola, info in tqdm(olas_definidas.items(), desc="Procesando Olas",␣
 ↪total=len(olas_definidas)):
    casos_ola = datos.loc[info['inicio']:info['fin']]['casos_suavizados']
    params_dist = PARAMETROS_OLAS_DIST[nombre_ola]

    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    fases_exponenciales[nombre_ola] = casos_exp
    t_exp = np.arange(len(casos_exp))

    d_samples = np.random.triangular(params_dist['D']['min'],␣
 ↪params_dist['D']['mode'], params_dist['D']['max'], CONFIG['N_SIM'])
    fd_samples = np.random.triangular(params_dist['FD']['min'],␣
 ↪params_dist['FD']['mode'], params_dist['FD']['max'], CONFIG['N_SIM'])
    s0_n_samples = np.random.triangular(params_dist['S0/N']['min'],␣
 ↪params_dist['S0/N']['mode'], params_dist['S0/N']['max'], CONFIG['N_SIM'])

    r0_results = []; r2_sir_list, r2_exp_list = [], []

    # --- CORRECCIÓN: Se elimina la barra de progreso interna para mayor␣
 ↪limpieza ---
    for i in range(CONFIG['N_SIM']):
        casos_corr = casos_exp / fd_samples[i]
        reff_sir, r2_sir = ajustar_modelo_sir(t_exp, casos_corr.values,␣
 ↪d_samples[i])
        reff_exp, r2_exp = modelo_exponencial(t_exp, casos_corr.values,␣
 ↪d_samples[i])

        r2_sir_list.append(r2_sir)
        r2_exp_list.append(r2_exp)

        if np.isnan(reff_sir) or np.isnan(reff_exp) or (r2_sir + r2_exp == 0):
            reff_comb = np.nanmean([reff_sir, reff_exp])
        else:
            reff_comb = (reff_sir * r2_sir + reff_exp * r2_exp) / (r2_sir +␣
 ↪r2_exp)

        if not np.isnan(reff_comb):
```

```
            R0 = reff_comb / s0_n_samples[i]
            r0_results.append(R0)

    resultados_distribuciones[nombre_ola] = np.array(r0_results)
    stats_por_ola[nombre_ola] = {
        'r2_sir_mean': np.mean(r2_sir_list),
        'r2_exp_mean': np.mean(r2_exp_list)
    }

# ======================================================================
# --- PRESENTACIÓN DE RESULTADOS ---
# ======================================================================
stats_data = []
for nombre, dist in resultados_distribuciones.items():
    params_modo = PARAMETROS_OLAS_DIST[nombre]
    stats_ajuste = stats_por_ola[nombre]
    stats_data.append({
        'Ola': nombre,
        # --- CORRECCIÓN: Se usa la sintaxis LaTeX correcta para ± ---
        r'$R_0$ ($\mu \pm \sigma$)': f'{np.mean(dist):.2f} $\\pm$ {np.std(dist):
 ↪.2f}',
        'IC 95%': f'[{np.percentile(dist, 2.5):.2f} - {np.percentile(dist, 97.
 ↪5):.2f}]',
        r'$R^2$ SIR (prom.)': f"{stats_ajuste['r2_sir_mean']:.3f}",
        r'$R^2$ Exp (prom.)': f"{stats_ajuste['r2_exp_mean']:.3f}",
        'FD (modo)': f"{params_modo['FD']['mode']:.3f}",
        r'$s_0$ (modo)': f"{params_modo['S0/N']['mode']:.3f}"
    })
df_resultados = pd.DataFrame(stats_data)

print("\n" + "="*110 + "\nTabla de Resultados: Estadísticas de R0 por Ola␣
 ↪Epidémica (Método Híbrido)\n" + "="*110)
print(df_resultados.to_string(index=False))
print("="*110)


# --- Gráfica/Tabla 0: Tabla de Resultados como Imagen ---
fig, ax = plt.subplots(figsize=(12, 4)); ax.axis('tight'); ax.axis('off')
the_table = ax.table(cellText=df_resultados.values, colLabels=df_resultados.
 ↪columns, loc='center', cellLoc='center',
                     colWidths=[0.2, 0.15, 0.2, 0.1, 0.1, 0.1, 0.1])
the_table.auto_set_font_size(False); the_table.set_fontsize(10); the_table.
 ↪scale(1.2, 1.5)
for (i, j), cell in the_table.get_celld().items():
    if i == 0: cell.set_text_props(weight='bold', color='white'); cell.
 ↪set_facecolor('#0d3b66')
```

```
plt.title(r'\textbf{Tabla de Resultados: Estimaciones de Alta Precisión de␣
 ↪$R_0$}', y=1.1, fontsize=14)
plt.savefig('Tabla_Resultados_R0_Hibrido.pdf')
print("\n Tabla de resultados de alta precisión guardada.")
plt.show()


# --- Gráfica 1: Curva Epidémica y Fases de Ajuste ---
fig, ax = plt.subplots(figsize=(12, 7))
ax.plot(datos.index, datos['casos_suavizados'], color='gray', label='Casos␣
 ↪diarios (media móvil 7 días)', alpha=0.8, lw=1.5)
for nombre, info in olas_definidas.items():
    color = PARAMETROS_OLAS_DIST[nombre]['color']
    ax.axvspan(info['inicio'], info['fin'], color=color, alpha=0.15)
    fase = fases_exponenciales.get(nombre)
    if fase is not None and not fase.empty:
        ax.plot(fase.index, fase.values, color=color, lw=3.5)
ax.set_title(r'\textbf{Curva Epidémica en México y Fases de Crecimiento para el␣
 ↪Cálculo de $R_0$}')
ax.set_ylabel('Casos Diarios Confirmados (Suavizados)'); ax.set_xlabel('Fecha')
ax.set_yscale('log'); ax.set_ylim(bottom=100); ax.grid(True, which='both',␣
 ↪linestyle=':', linewidth=0.5)
legend_elements = [plt.Line2D([0], [0], color='gray', lw=1.5, label='Casos␣
 ↪diarios')]
legend_elements += [Patch(facecolor=PARAMETROS_OLAS_DIST[nombre]['color'],␣
 ↪alpha=0.5, label=nombre) for nombre in olas_definidas.keys()]
ax.legend(handles=legend_elements, loc='upper left', fontsize=9)
plt.savefig('Figura_Curva_Epidemica_Fases_Ajuste.pdf')
print("\n Figura con la curva epidémica y fases de ajuste (corregida) guardada.
 ↪")
plt.show()


# --- Gráfica 2: Distribuciones de R0 (Gráficos de Densidad - KDE) ---
fig, axes = plt.subplots(3, 2, figsize=(10, 12), sharex=False, sharey=True)
axes = axes.flatten()
fig.suptitle(r'\textbf{Distribuciones de Probabilidad de $R_0$ (Método␣
 ↪Híbrido)}', y=0.96)
for i, (nombre, dist) in enumerate(resultados_distribuciones.items()):
    ax = axes[i]
    color = PARAMETROS_OLAS_DIST[nombre]['color']
    mean_val = np.mean(dist)
    ci_lower, ci_upper = np.percentile(dist, [2.5, 97.5])
    sns.kdeplot(x=dist, ax=ax, color=color, fill=True, alpha=0.2, lw=2.5)
    ax.axvline(mean_val, color=color, linestyle='--', lw=1.5)
    # --- CORRECCIÓN: Se escapa el % para LaTeX ---
```

```
    stats_text = (f'$\\boldsymbol{{R_0 = {mean_val:.2f}}}$\n'
                  f'IC 95\\%: [{ci_lower:.2f} - {ci_upper:.2f}]')
    ax.text(0.95, 0.95, stats_text, transform=ax.transAxes, ha='right',␣
 ↪va='top',
            fontsize=9, bbox=dict(boxstyle='round,pad=0.3', fc='white',␣
 ↪ec=color, alpha=0.8))
    ax.set_title(nombre, fontsize=12)
    ax.set_xlabel(r'Valor de $R_0$'); ax.set_ylabel('Densidad de Probabilidad')
    ax.grid(linestyle=':', color='#cccccc')
for i in range(len(resultados_distribuciones), len(axes)):
    axes[i].axis('off')
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig('Figura_R0_Hibrido_Distribuciones_KDE.pdf')
print("\n  Figura con las distribuciones de R0 (KDE) guardada.")
plt.show()


# --- Gráfica 3: Resumen de R0 con Intervalos de Confianza ---
fig, ax = plt.subplots(figsize=(10, 6))
means = [np.mean(dist) for dist in resultados_distribuciones.values()]
cis = [np.percentile(dist, [2.5, 97.5]) for dist in resultados_distribuciones.
 ↪values()]
errors = [[m - ci[0] for m, ci in zip(means, cis)], [ci[1] - m for m, ci in␣
 ↪zip(means, cis)]]
colors = [p['color'] for p in PARAMETROS_OLAS_DIST.values()]
ax.bar(df_resultados['Ola'], means, yerr=errors, color=colors,␣
 ↪edgecolor='black', lw=0.8, capsize=5, zorder=3)
ax.axhline(1.0, color='black', linestyle='--', lw=1.5, label='Umbral Epidémico␣
 ↪($R_0=1$)', zorder=2)
ax.set_title(r'\textbf{Estimación de $R_0$ con Intervalos de Confianza del 95\%␣
 ↪(Híbrido)}', y=1.05)
ax.set_ylabel(r'Valor de $R_0$'); ax.set_xlabel('Ola Epidémica')
ax.tick_params(axis='x', labelrotation=15, labelsize=9)
ax.legend()
ax.grid(axis='y', linestyle=':', color='#cccccc', zorder=0)
for i, bar in enumerate(ax.patches):
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2.0, means[i] + errors[1][i] + 0.05,␣
 ↪f'{means[i]:.2f}',
            ha='center', va='bottom', weight='bold', fontsize=9)
plt.tight_layout()
plt.savefig('Figura_R0_Hibrido_Resumen.pdf')
print("\n  Figura de resumen de R0 (Híbrido) guardada.")
plt.show()
```

```
Cargando datos y definiendo olas…
```

```
Iniciando Simulación Híbrida con 100,000 iteraciones por ola…
Procesando Olas: 100%|               | 6/6 [07:34<00:00, 75.83s/it]

================================================================================
==============================
Tabla de Resultados: Estadísticas de R0 por Ola Epidémica (Método Híbrido)
================================================================================
==============================
                  Ola $R_0$ ($\mu \pm \sigma$)        IC 95% $R^2$ SIR (prom.)
$R^2$ Exp (prom.) FD (modo) $s_0$ (modo)
     Ola 1 (Original)          1.92 $\pm$ 0.09 [1.74 - 2.10]             0.996
0.997     0.024        1.000
        Ola 2 (Alpha)          1.65 $\pm$ 0.14 [1.42 - 1.95]             0.994
0.994     0.024        0.740
        Ola 3 (Delta)          2.84 $\pm$ 0.24 [2.42 - 3.35]             0.919
0.932     0.050        0.471
      Ola 4 (Ómicron)          1.82 $\pm$ 0.15 [1.56 - 2.14]             0.993
0.993     0.059        0.701
Ola 5 (Ómicron Subs.)          5.41 $\pm$ 0.56 [4.43 - 6.60]             0.989
0.991     0.059        0.416
               Ola 6           4.82 $\pm$ 0.43 [4.09 - 5.73]             0.996
0.996     0.059        0.315
================================================================================
==============================
```

Tabla de resultados de alta precisión guardada.

Tabla de Resultados: Estimaciones de Alta Precisión de R_0

Ola	R_0 ($\mu \pm \sigma$)	IC 95	R^2 SIR (prom.)	R^2 Exp (prom.)	FD (modo)	s_0 (modo)
Ola 1 (Original)	1.92 ± 0.09	[1.74 - 2.10]	0.996	0.997	0.024	1.000
Ola 2 (Alpha)	1.65 ± 0.14	[1.42 - 1.95]	0.994	0.994	0.024	0.740
Ola 3 (Delta)	2.84 ± 0.24	[2.42 - 3.35]	0.919	0.932	0.050	0.471
Ola 4 (Ómicron)	1.82 ± 0.15	[1.56 - 2.14]	0.993	0.993	0.059	0.701
Ola 5 (Ómicron Subs.)	5.41 ± 0.56	[4.43 - 6.60]	0.989	0.991	0.059	0.416
Ola 6	4.82 ± 0.43	[4.09 - 5.73]	0.996	0.996	0.059	0.315

Figura con la curva epidémica y fases de ajuste (corregida) guardada.

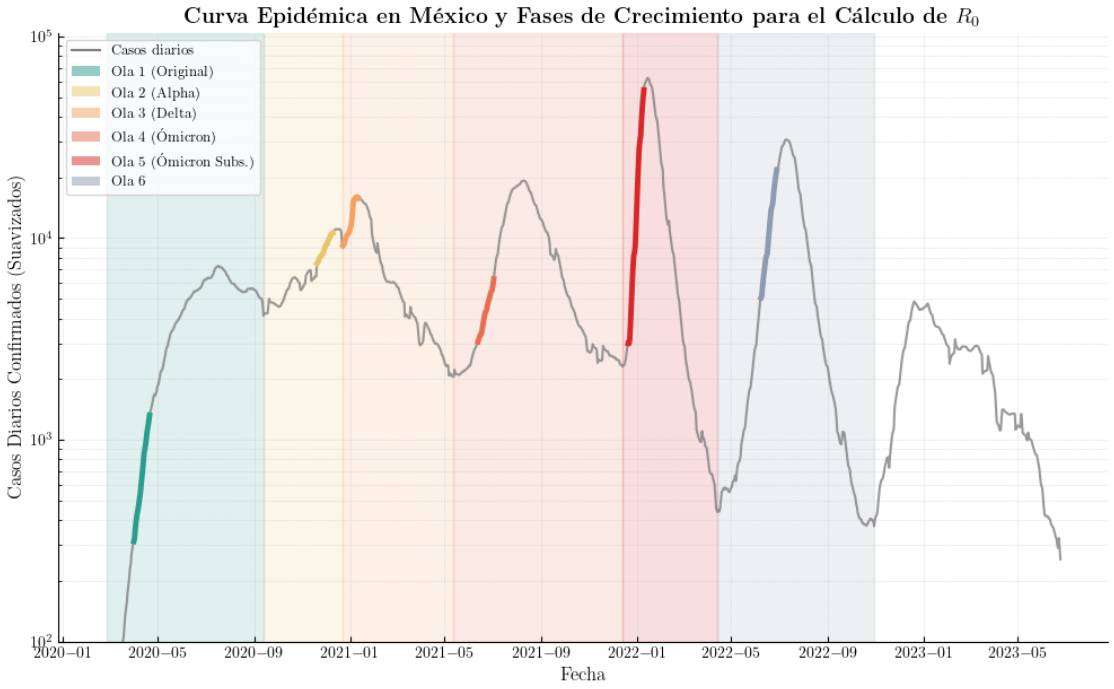


Figura con las distribuciones de R0 (KDE) guardada.

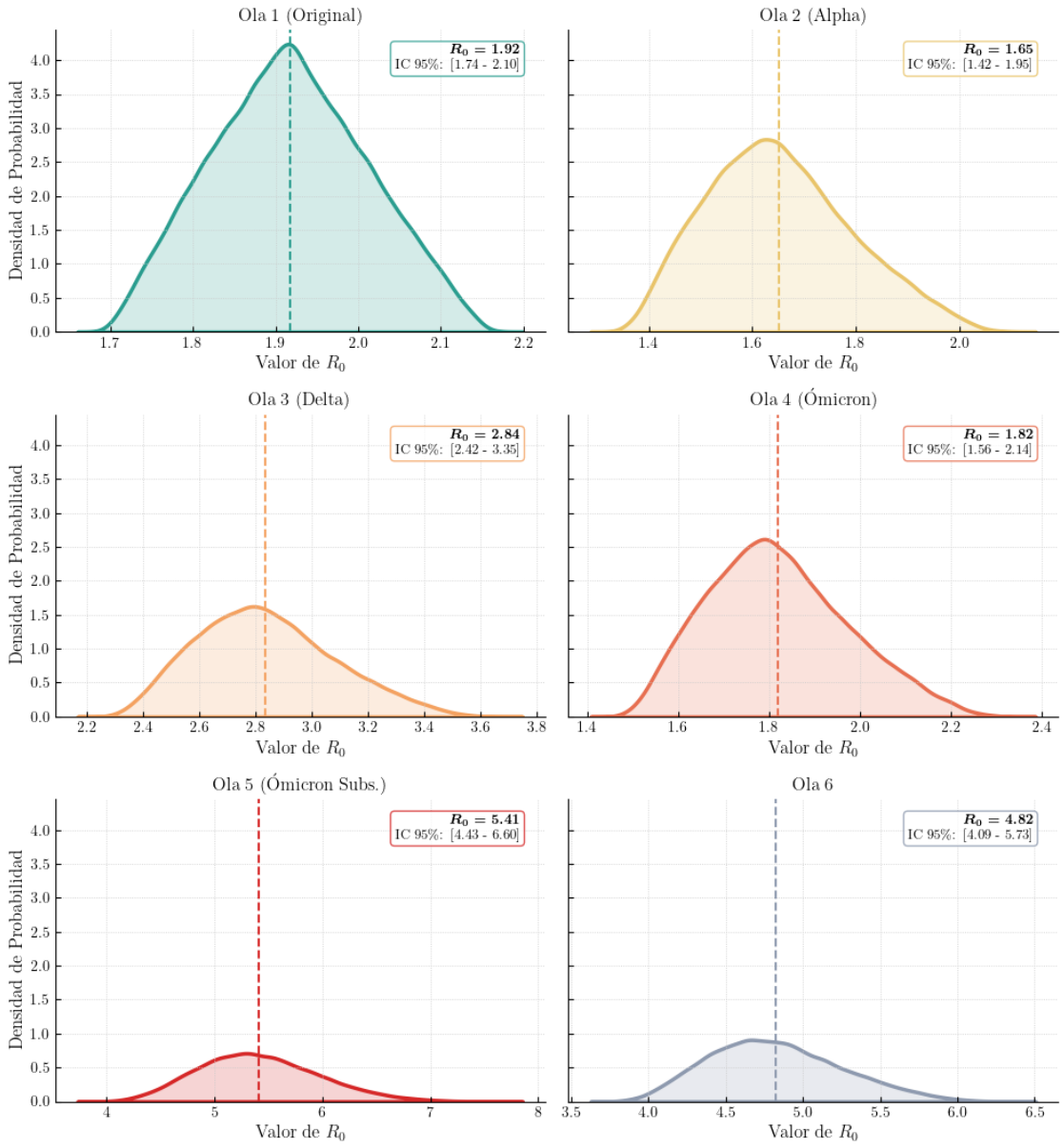
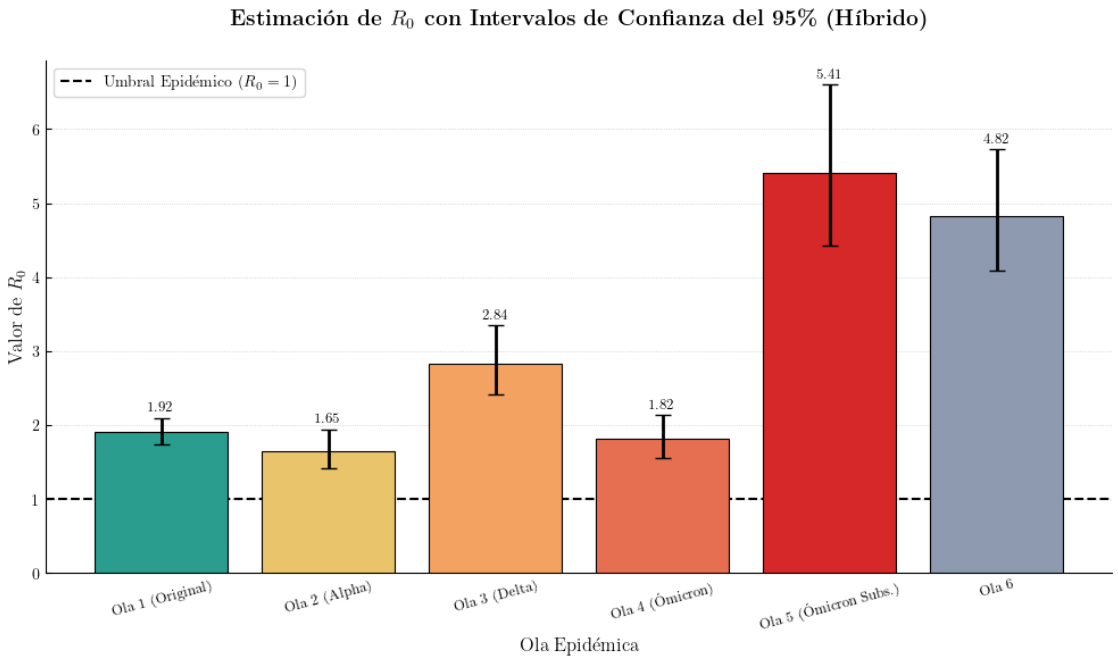


Figura de resumen de R0 (Híbrido) guardada.



[12]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
import matplotlib.gridspec as gridspec

# ============================================================================
# 1. PARÁMETROS DE LAS OLAS Y CONFIGURACIÓN
# ============================================================================
WAVE_PARAMETERS = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for wave in WAVE_PARAMETERS:
    wave['gamma'] = 1.0 / wave['D']

POPULATION = 1.0
cmap = plt.get_cmap('viridis', 6)

# ============================================================================
# 2. ESTILO Y MODELO
```

```python
# ======================================================================
def setup_publication_style():
    """Configura Matplotlib con un estilo profesional para publicaciones."""
    plt.rcParams.update({
        'font.family': 'serif', 'font.serif': ['Computer Modern Roman', 'Times␣
 ↪New Roman'],
        'text.usetex': True, 'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}',
        'font.size': 11, 'axes.labelsize': 12, 'axes.titlesize': 14,
        'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 9,
        'figure.titlesize': 16, 'lines.linewidth': 2.0, 'axes.linewidth': 0.8,
        'grid.color': '#cccccc', 'grid.linestyle': ':', 'grid.linewidth': 0.7,
        'axes.spines.right': False, 'axes.spines.top': False,
        'figure.constrained_layout.use': True
    })

def sir_model(t, y, R0, gamma, N):
    """Modelo SIR 2D (S, I)."""
    S, I = y
    beta = R0 * gamma
    dS_dt = -beta * S * I / N
    dI_dt = beta * S * I / N - gamma * I
    return [dS_dt, dI_dt]

# ======================================================================
# 3. FUNCIONES DE GRAFICADO (Sin cambios)
# ======================================================================
def plot_stability_olas(ax, wave_params):
    s_vals = np.linspace(0, 1, 200)
    ax.axhline(0, color='k', linestyle='--', lw=1, alpha=0.7)
    for i, p in enumerate(wave_params):
        lam = p['gamma'] * (p['R0'] * s_vals - 1)
        ax.plot(s_vals, lam, color=cmap(i), label=f"{p['name']} ($R_0={p['R0']:.
 ↪2f}$)")
    ax.set(xlabel=r'Fracción Susceptible ($S/N$)', ylabel=r'Eigenvalor␣
 ↪($\lambda_2$)')
    ax.legend(title='Condición de la Ola', loc='best', ncol=2)
    ax.set_xlim(0, 1)

def plot_R0_comparativa(ax, wave_params):
    names = [p['name'] for p in wave_params]
    r0s = [p['R0'] for p in wave_params]
    colors = [cmap(i) for i in range(len(wave_params))]

    bars = ax.barh(names, r0s, color=colors, edgecolor='black', linewidth=0.7,␣
 ↪zorder=3)
    ax.invert_yaxis()
```

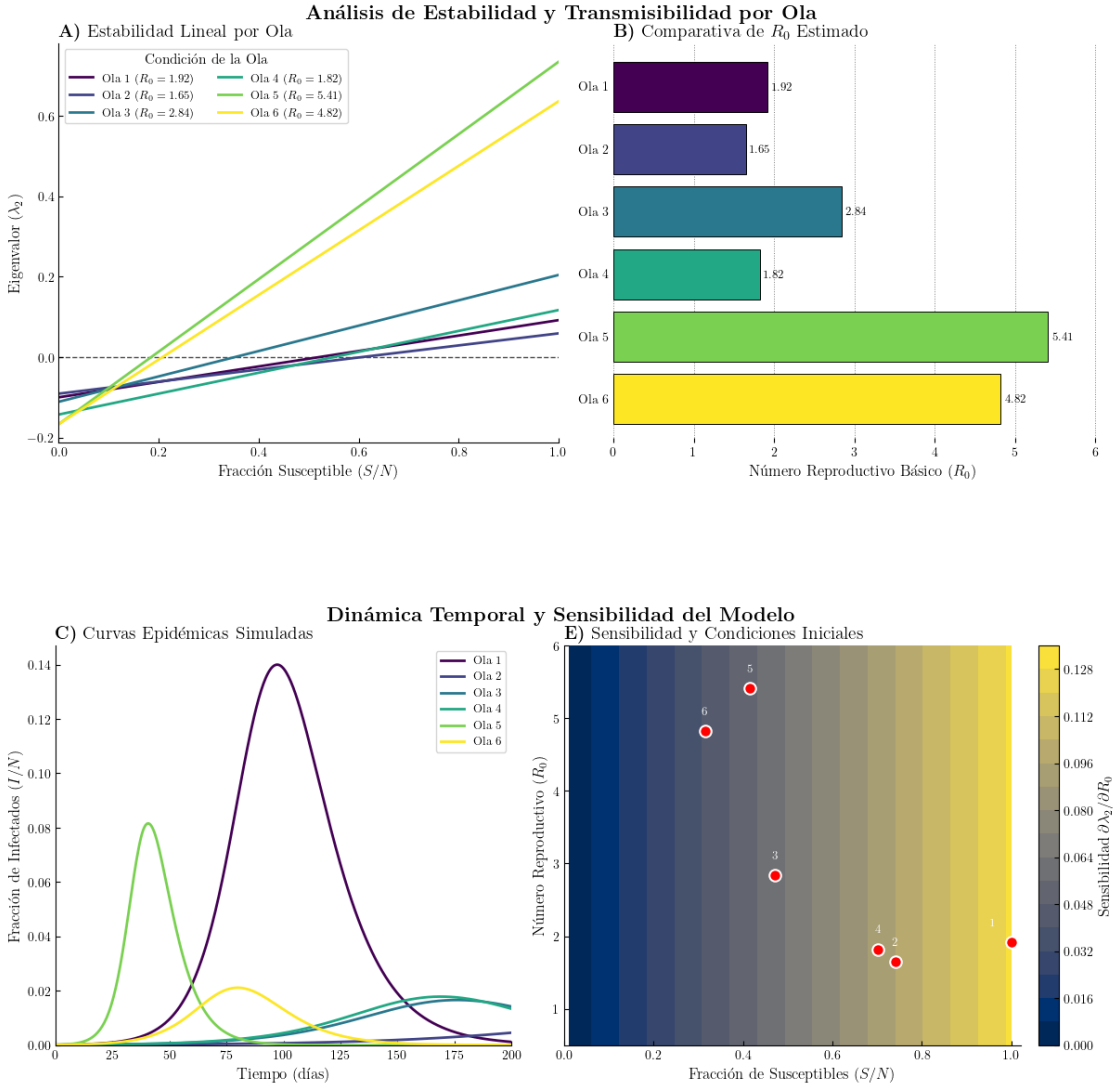
```
    ax.set(xlabel=r'Número Reproductivo Básico ($R_0$)')
    ax.grid(axis='x', linestyle=':', color='gray', zorder=0)
    ax.spines['left'].set_visible(False)
    ax.spines['bottom'].set_visible(False)
    ax.tick_params(axis='y', length=0)

    for bar in bars:
        width = bar.get_width()
        ax.text(width + 0.05, bar.get_y() + bar.get_height()/2, f'{width:.2f}',
                va='center', ha='left', fontsize=9)
    ax.set_xlim(right=max(r0s) * 1.15)

def plot_time_series_olas(ax, wave_params):
    t_eval = np.linspace(0, 200, 400)
    for i, p in enumerate(wave_params):
        y0 = [p['s0'] * POPULATION, 1e-4 * POPULATION]
        sol = solve_ivp(sir_model, [0, 200], y0, args=(p['R0'], p['gamma'],␣
 ↪POPULATION), t_eval=t_eval)
        ax.plot(sol.t, sol.y[1] / POPULATION, color=cmap(i),␣
 ↪label=f"{p['name']}")
    ax.set(xlabel='Tiempo (días)', ylabel=r'Fracción de Infectados ($I/N$)')
    ax.legend(loc='upper right')
    ax.set_xlim(0, 200)
    ax.set_ylim(bottom=0)

def plot_phase_portrait_olas(ax, wave_params):
    for i, p in enumerate(wave_params):
        y0 = [p['s0'] * POPULATION, 1e-4 * POPULATION]
        sol = solve_ivp(sir_model, [0, 300], y0, args=(p['R0'], p['gamma'],␣
 ↪POPULATION), dense_output=True)
        t = np.linspace(0, sol.t[-1], 300)
        s_path, i_path = sol.sol(t)
        ax.plot(s_path / POPULATION, i_path / POPULATION, color=cmap(i),␣
 ↪label=f"{p['name']}")
    ax.set(xlabel=r'Fracción de Susceptibles ($S/N$)', ylabel=r'Fracción de␣
 ↪Infectados ($I/N$)')
    ax.legend(loc='best')
    ax.set_xlim(0, 1.02)
    ax.set_ylim(bottom=0)

def plot_sensitivity_map_olas(ax, wave_params):
    S_grid, R0_grid = np.meshgrid(np.linspace(0.01, 1.0, 100), np.linspace(0.5,␣
 ↪6.0, 100))
    gamma_avg = np.mean([p['gamma'] for p in wave_params])
    sensitivity = gamma_avg * S_grid
```

```
    cont = ax.contourf(S_grid, R0_grid, sensitivity, levels=20, cmap='cividis')
    cbar = plt.colorbar(cont, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label(r'Sensibilidad $\partial\lambda_2 / \partial R_0$')

    for i, p in enumerate(wave_params):
        ax.plot(p['s0'], p['R0'], 'o', color='red', markersize=9,
                markeredgecolor='white', markeredgewidth=1.5)
        text_x_pos = p['s0'] - 0.04 if p['s0'] > 0.98 else p['s0']
        ax.text(text_x_pos, p['R0'] + 0.2, str(i + 1), color='white',␣
↪ha='center',
                va='bottom', fontsize=9, fontweight='bold')

    ax.set(xlabel=r'Fracción de Susceptibles ($S/N$)', ylabel=r'Número␣
↪Reproductivo ($R_0$)')
    ax.set_xlim(0, 1.02)
    ax.set_ylim(0.5, 6.0)

def create_table_image(df, filename='Tabla_Resultados_Simulacion.png'):
    """Genera y guarda una imagen PNG de un DataFrame de pandas."""
    df_display = df.copy()
    for col in df_display.columns:
        if df_display[col].dtype == 'float':
            df_display[col] = df_display[col].apply(lambda x: f'{x:.3f}')

    fig, ax = plt.subplots(figsize=(10, 3))
    ax.axis('tight'); ax.axis('off')
    table = ax.table(cellText=df_display.values, colLabels=df_display.columns,␣
↪loc='center',
                     cellLoc='center', colWidths=[0.1, 0.1, 0.1, 0.15, 0.1, 0.
↪2, 0.25])
    table.scale(1, 1.8)
    table.auto_set_font_size(False); table.set_fontsize(10)

    for (i, j), cell in table.get_celld().items():
        cell.set_edgecolor('#dddddd')
        if i == 0:
            cell.set_text_props(weight='bold', color='white'); cell.
↪set_facecolor('#2c3e50')
        elif i % 2 == 0: cell.set_facecolor('#ecf0f1')
        else: cell.set_facecolor('#ffffff')

    fig.suptitle('Tabla de Parámetros y Resultados de Simulación por Ola',␣
↪fontsize=16, y=1.05)
    plt.savefig(filename, dpi=300, bbox_inches='tight', pad_inches=0.5)
    plt.close(fig) # Cierra solo la figura de la tabla
    print(f"  Imagen de la tabla guardada como '{filename}'.")
```

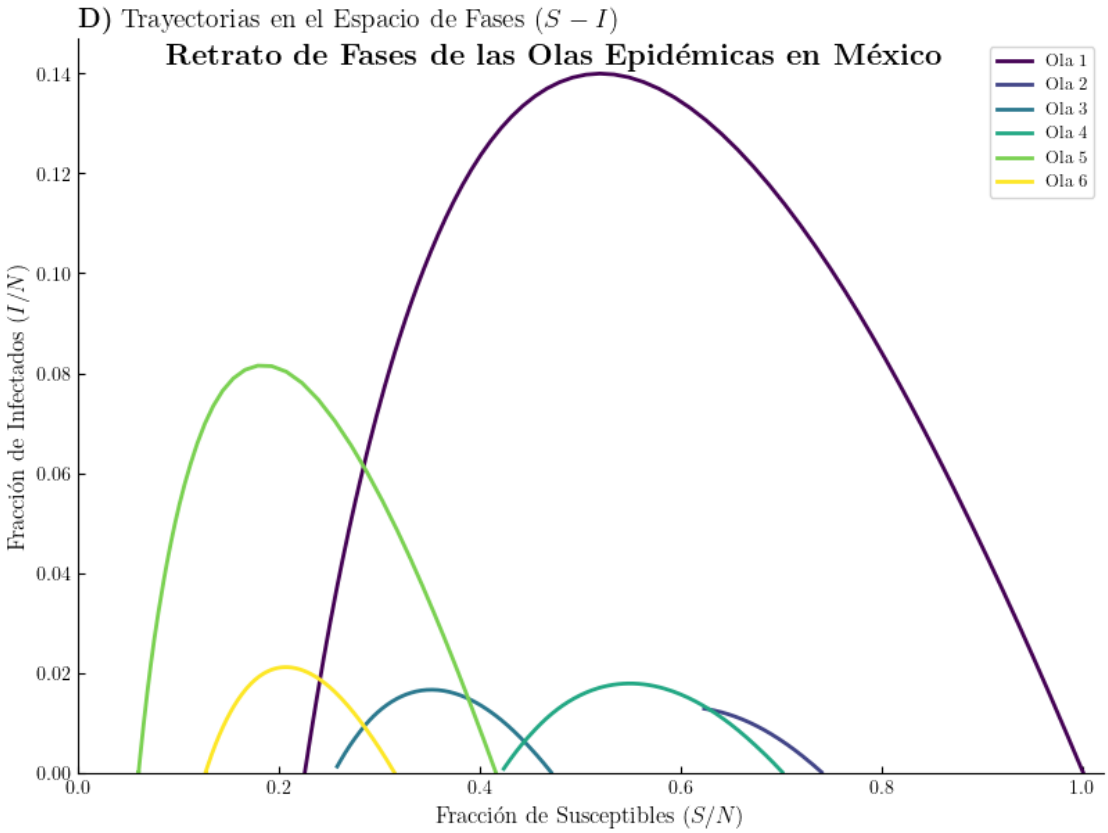
```
# ======================================================================
# 4. SCRIPT PRINCIPAL DE EJECUCIÓN
# ======================================================================
# --- Generar Datos y Tabla ---
simulation_results = []
for p in WAVE_PARAMETERS:
    y0 = [p['s0'] * POPULATION, 1e-6 * POPULATION]
    sol = solve_ivp(sir_model, [0, 500], y0, args=(p['R0'], p['gamma'],␣
 ↪POPULATION))
    i_max = np.max(sol.y[1]); s_final = sol.y[0][-1]
    epidemic_size = p['s0'] - (s_final / POPULATION)
    simulation_results.append({
        'Ola': p['name'], '$R_0$': p['R0'], '$s_0$': p['s0'], 'D (días)':␣
 ↪p['D'],
        r'$\gamma$': p['gamma'], '$I_{pico} / N$ (%)': i_max / POPULATION * 100,
        'Tamaño Total Epidemia (%)': epidemic_size * 100})
df_results = pd.DataFrame(simulation_results)

# --- Activar Estilo y Crear Gráficos ---
setup_publication_style()

# --- FIGURA 1: Estabilidad y R0 (A y B) ---
fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
fig1.suptitle(r'\textbf{Análisis de Estabilidad y Transmisibilidad por Ola}',␣
 ↪y=1.03)
plot_stability_olas(ax1, WAVE_PARAMETERS)
ax1.set_title(r'\textbf{A)} Estabilidad Lineal por Ola', loc='left')
plot_R0_comparativa(ax2, WAVE_PARAMETERS)
ax2.set_title(r'\textbf{B)} Comparativa de $R_0$ Estimado', loc='left')
fig1.savefig('Figura_1_Estabilidad_y_R0.png', dpi=300, bbox_inches='tight')
print("\n Figura 1 (Estabilidad y R0) guardada exitosamente.")

# --- FIGURA 2: Curvas y Sensibilidad (C y E) ---
fig2, (ax3, ax5) = plt.subplots(1, 2, figsize=(12, 5))
fig2.suptitle(r'\textbf{Dinámica Temporal y Sensibilidad del Modelo}', y=1.03)
plot_time_series_olas(ax3, WAVE_PARAMETERS)
ax3.set_title(r'\textbf{C)} Curvas Epidémicas Simuladas', loc='left')
plot_sensitivity_map_olas(ax5, WAVE_PARAMETERS)
ax5.set_title(r'\textbf{E)} Sensibilidad y Condiciones Iniciales', loc='left')
fig2.savefig('Figura_2_Curvas_y_Sensibilidad.png', dpi=300, bbox_inches='tight')
print(" Figura 2 (Curvas y Sensibilidad) guardada exitosamente.")

# --- FIGURA 3: Retrato de Fases (D) ---
fig3, ax4 = plt.subplots(figsize=(8, 6))
fig3.suptitle(r'\textbf{Retrato de Fases de las Olas Epidémicas en México}',␣
 ↪y=0.95)
```

```
plot_phase_portrait_olas(ax4, WAVE_PARAMETERS)
ax4.set_title(r'\textbf{D)} Trayectorias en el Espacio de Fases ($S-I$)',␣
 ↪loc='left')
fig3.savefig('Figura_3_Retrato_de_Fases.png', dpi=300, bbox_inches='tight')
print("  Figura 3 (Retrato de Fases) guardada exitosamente.")

# --- Crear Imagen de Tabla ---
create_table_image(df_results)

# --- Imprimir Tabla en Consola ---
print("\n" + "="*80)
print("Tabla de Parámetros y Resultados de Simulación por Ola")
print("="*80)
print(df_results.to_string(index=False, float_format="%.3f"))

# --- Mostrar todas las figuras generadas ---
plt.show()
```

```
  Figura 1 (Estabilidad y R0) guardada exitosamente.
  Figura 2 (Curvas y Sensibilidad) guardada exitosamente.
  Figura 3 (Retrato de Fases) guardada exitosamente.
  Imagen de la tabla guardada como 'Tabla_Resultados_Simulacion.png'.

================================================================================
Tabla de Parámetros y Resultados de Simulación por Ola
================================================================================
  Ola  $R_0$  $s_0$  D (días)  $\gamma$  $I_{pico} / N$ (%)  Tamaño Total
Epidemia (%)
Ola 1  1.920  1.000        10     0.100              13.679
77.278
Ola 2  1.650  0.740        11     0.091               1.006
7.233
Ola 3  2.840  0.471         9     0.111               1.540
21.554
Ola 4  1.820  0.701         7     0.143               1.737
27.938
Ola 5  5.410  0.416         6     0.167               7.604
35.523
Ola 6  4.820  0.315         6     0.167               2.094
18.744
```


Análisis de Estabilidad y Transmisibilidad por Ola
A) Estabilidad Lineal por Ola
Condición de la Ola
Ola 1 ($R_0 = 1.92$)
Ola 2 ($R_0 = 1.65$)
Ola 3 ($R_0 = 2.84$)
Ola 4 ($R_0 = 1.82$)
Ola 5 ($R_0 = 5.41$)
Ola 6 ($R_0 = 4.82$)
Eigenvalor (λ_2)
Fracción Susceptible (S/N)
B) Comparativa de R_0 Estimado
Ola 1
Ola 2
Ola 3
Ola 4
Ola 5
Ola 6
1.92
1.65
2.84
1.82
5.41
4.82
Número Reproductivo Básico (R_0)

Dinámica Temporal y Sensibilidad del Modelo
C) Curvas Epidémicas Simuladas
Ola 1
Ola 2
Ola 3
Ola 4
Ola 5
Ola 6
Fracción de Infectados (I/N)
Tiempo (días)
E) Sensibilidad y Condiciones Iniciales
Número Reproductivo (R_0)
Fracción de Susceptibles (S/N)
Sensibilidad $\partial\lambda_2/\partial R_0$



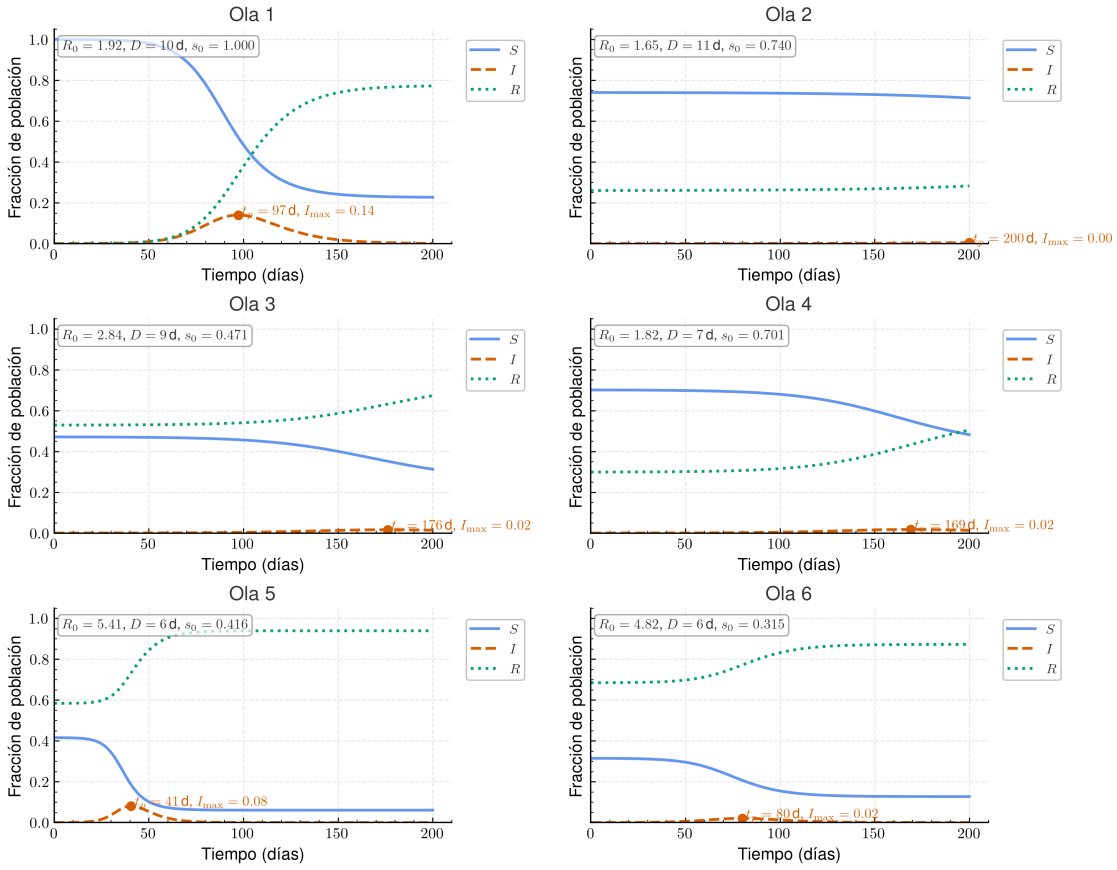
[28]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ============================================================================
# 1. PALETA Y ESTILO ULTRA-PROFESIONAL
# ============================================================================
PALETA = {
    'S': '#6495ED',   # Susceptibles
    'I': '#D55E00',   # Infectados
    'R': '#009E73',   # Recuperados
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
}
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['Arial', 'Helvetica', 'DejaVu Sans'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
    'font.size': 12,
```

```
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'figure.titlesize': 16,
    'text.color': PALETA['Texto'],
    'figure.dpi': 300,
    'savefig.format': 'pdf',
    'savefig.bbox': 'tight',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.linewidth': 1.0,
    'grid.linestyle': '--',
    'grid.color': '#dddddd',
    'grid.alpha': 0.7,
    'figure.facecolor': PALETA['Fondo'],
    'axes.facecolor': PALETA['Fondo'],
    'figure.constrained_layout.use': False,
})

# ============================================================================
# 2. PARÁMETROS DE LAS OLAS
# ============================================================================
WAVES = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']
N = 1.0

# ============================================================================
# 3. MODELO SIR
# ============================================================================
def sir(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    return [-beta*S*I/N, beta*S*I/N - gamma*I]

# ============================================================================
# 4. CÁLCULO DEL RANGO COMÚN HASTA CRUCE S=R
# ============================================================================
t = np.linspace(0, 200, 1000)
t_int_list = []
```

```
for w in WAVES:
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S, I = sol.y
    R = 1 - (S + I)
    diff = S - R
    # encontramos el primer índice donde S   R
    idx = np.where(diff <= 0)[0]
    t_int_list.append(t[idx[0]] if idx.size else t[-1])

# tomamos el máximo de esas intersecciones, le damos un 5% extra
xmax = max(t_int_list) * 1.05

# ============================================================================
# 5. SIMULACIÓN FINAL Y GRÁFICOS
# ============================================================================
fig, axes = plt.subplots(3, 2, figsize=(12, 10), sharey=True,␣
 ↪constrained_layout=False)
axes = axes.flatten()

for ax, w in zip(axes, WAVES):
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S, I = sol.y
    R = 1 - (S + I)

    # Dibujamos con líneas gruesas
    lw = 2.0
    ax.plot(t, S,  label=r'$S$', color=PALETA['S'], linewidth=lw)
    ax.plot(t, I, '--',label=r'$I$', color=PALETA['I'], linewidth=lw)
    ax.plot(t, R, ':', label=r'$R$', color=PALETA['R'], linewidth=lw)

    # Pico de infectados
    idx_peak = np.argmax(I)
    ax.plot(t[idx_peak], I[idx_peak], 'o', color=PALETA['I'], markersize=6)
    ax.text(t[idx_peak]+2, I[idx_peak],
            rf'$t_p={t[idx_peak]:.0f}\,$d, $I_{{\max}}={I[idx_peak]:.2f}$',
            color=PALETA['I'], fontsize=10)

    # Parámetros en recuadro
    txt = (rf"$R_0={w['R0']:.2f}$, "
           rf"$D={w['D']}\,$d, "
           rf"$s_0={w['s0']:.3f}$")
    ax.text(0.02, 0.95, txt, transform=ax.transAxes,
            ha='left', va='top', fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3",
                      facecolor='white', edgecolor='#888888', alpha=0.7))
```

```
    # Leyenda fuera, semitransparente
    leg = ax.legend(loc='upper left', bbox_to_anchor=(1.02, 0.98),
                    frameon=True, framealpha=0.7, borderpad=0.3, fontsize=10)
    leg.get_frame().set_edgecolor('#aaaaaa')

    # Ajuste fijo de ejes para todas
    ax.set_xlim(0, xmax)
    ax.set_ylim(0, 1.05)
    ax.set_title(w['name'], fontweight='bold')
    ax.set_xlabel('Tiempo (días)')
    ax.set_ylabel('Fracción de población')
    ax.grid(True)
    ax.minorticks_on()

# Título general y márgenes finales
fig.suptitle(r'\textbf{Figura 4.3(a-d): Evolución Temporal SIR por Ola}', y=0.
  ↪97, fontsize=16)
fig.subplots_adjust(top=0.90, hspace=0.35, wspace=0.35)
fig.savefig('Figura_4_3_SIR_Evolucion_Elegante.pdf')
plt.show()
```

Figura 4.3(a–d): Evolución Temporal SIR por Ola

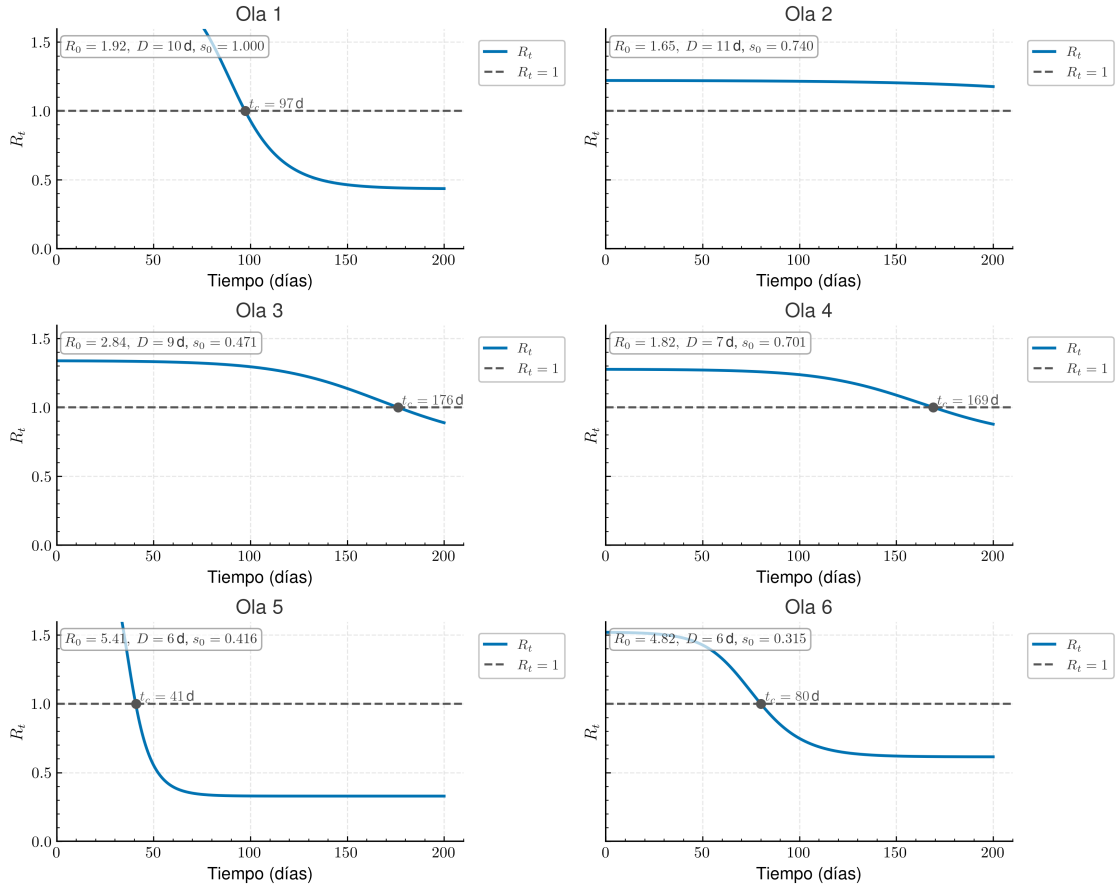


[29]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ============================================================================
# 1. PALETA Y ESTILO ULTRA-PROFESIONAL (igual que antes)
# ============================================================================
PALETA = {
    'Rt': '#0072B2',      # Azul intenso para Rt
    'Umbral1': '#555555',# Gris para línea umbral
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
}
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['Arial','Helvetica','DejaVu Sans'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
    'font.size': 12,
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'figure.titlesize': 16,
    'text.color': PALETA['Texto'],
    'figure.dpi': 300,
    'savefig.format': 'pdf',
    'savefig.bbox': 'tight',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.linewidth': 1.0,
    'grid.linestyle': '--',
    'grid.color': '#dddddd',
    'grid.alpha': 0.7,
    'figure.facecolor': PALETA['Fondo'],
    'axes.facecolor': PALETA['Fondo'],
    'figure.constrained_layout.use': False,
})

# ============================================================================
# 2. PARÁMETROS DE LAS OLAS
# ============================================================================
WAVES = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
```

```
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']
N = 1.0

# ============================================================================
# 3. MODELO SIR (usado para obtener S(t))
# ============================================================================
def sir(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    return [-beta*S*I/N, beta*S*I/N - gamma*I]

# ============================================================================
# 4. CALCULAR LÍMITE X PARA MOSTRAR HASTA R_t=1
# ============================================================================
t = np.linspace(0, 200, 1000)
t_lim_list = []
for w in WAVES:
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S = sol.y[0]
    Rt = w['R0'] * S / N
    # buscamos primer índice donde Rt  1
    idx = np.where(Rt <= 1)[0]
    t_lim_list.append(t[idx[0]] if idx.size else t[-1])
xmax = max(t_lim_list) * 1.05

# ============================================================================
# 5. GRÁFICOS Rt POR OLA
# ============================================================================
fig, axes = plt.subplots(3, 2, figsize=(12, 10), sharey=True,␣
 ↪constrained_layout=False)
axes = axes.flatten()

for ax, w in zip(axes, WAVES):
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S = sol.y[0]
    Rt = w['R0'] * S / N

    # Línea Rt
    ax.plot(t, Rt, '-', label=r'$R_t$', color=PALETA['Rt'], linewidth=2.0)
    # Línea umbral Rt=1
```

```
    ax.axhline(1.0, linestyle='--', color=PALETA['Umbral1'], linewidth=1.5,
               label=r'$R_t=1$')

    # Marcador del cruce exacto Rt=1
    idx = np.where(Rt <= 1)[0]
    if idx.size:
        t1 = t[idx[0]]
        ax.plot(t1, 1.0, 'o', color=PALETA['Umbral1'], markersize=6)
        ax.text(t1+2, 1.02, rf'$t_c={t1:.0f}\,$d',
                color=PALETA['Umbral1'], fontsize=10)

    # Parámetros en recuadro semitransparente
    txt = rf"$R_0={w['R0']:.2f},\ D={w['D']}\,$d, $s_0={w['s0']:.3f}$"
    ax.text(0.02, 0.95, txt, transform=ax.transAxes,
            ha='left', va='top', fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3",
                      facecolor='white', edgecolor='#888888', alpha=0.7))

    # Leyenda fuera
    leg = ax.legend(loc='upper left', bbox_to_anchor=(1.02, 0.98),
                    frameon=True, framealpha=0.7, fontsize=10, handlelength=1.5)
    leg.get_frame().set_edgecolor('#aaaaaa')

    # Ejes y grilla
    ax.set_xlim(0, xmax)
    ax.set_ylim(0, max(1.2, np.nanmax(Rt)*1.05))
    ax.set_title(w['name'], fontweight='bold')
    ax.set_xlabel('Tiempo (días)')
    ax.set_ylabel('$R_t$')
    ax.grid(True, which='major')
    ax.minorticks_on()

# Título general y ajustes de márgenes
fig.suptitle(r'\textbf{Figura 4.5(a-d): Evolución del Número Reproductivo␣
 ↪Efectivo $R_t$ por Ola}', y=0.97, fontsize=16)
fig.subplots_adjust(top=0.90, hspace=0.35, wspace=0.35)
fig.savefig('Figura_4_5_Rt_Evolucion_Elegante.pdf')
plt.show()
```

Figura 4.5(a–d): Evolución del Número Reproductivo Efectivo R_t por Ola



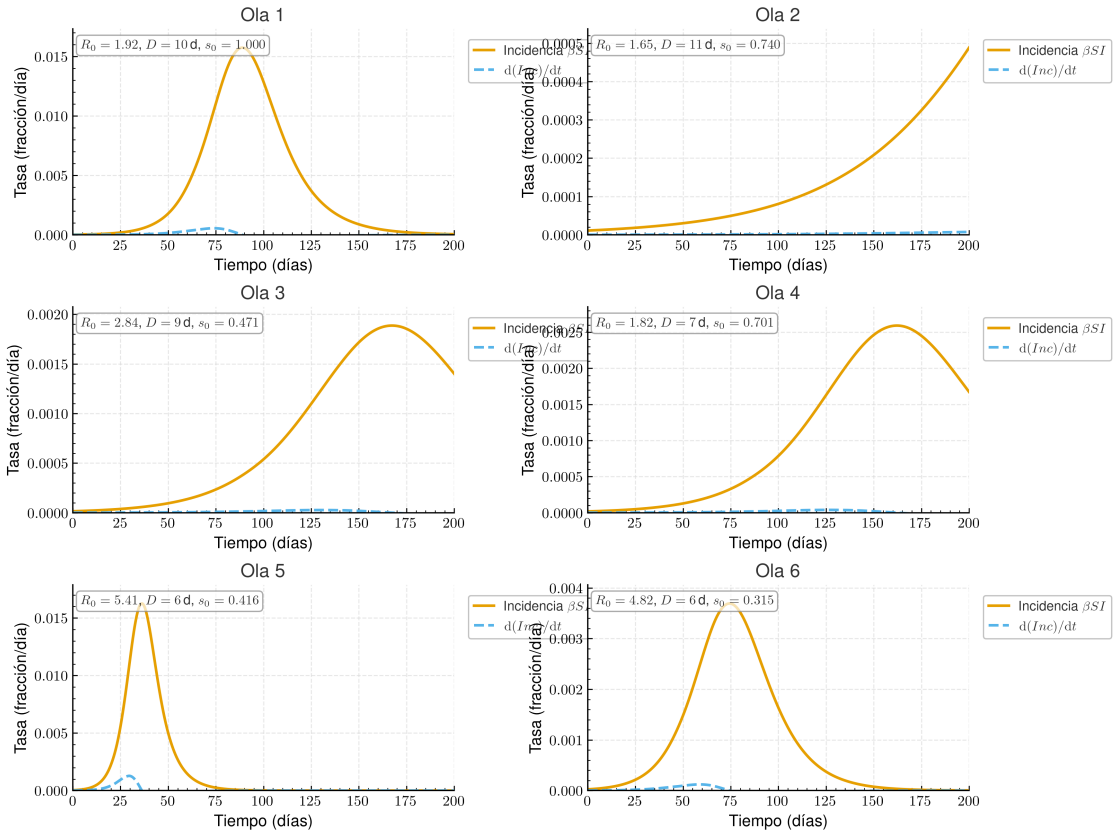
[30]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ============================================================================
# 1. PALETA Y ESTILO (mismo perfil profesional)
# ============================================================================
PALETA = {
    'Inc': '#E69F00',     # Dorado para incidencia
    'dInc': '#56B4E9',    # Celeste para su cambio
    'Umbral': '#555555',
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
}
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['Arial','Helvetica','DejaVu Sans'],
    'text.usetex': True,
```

```
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
    'font.size': 12,
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'figure.titlesize': 16,
    'text.color': PALETA['Texto'],
    'figure.dpi': 300,
    'savefig.format': 'pdf',
    'savefig.bbox': 'tight',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.linewidth': 1.0,
    'grid.linestyle': '--',
    'grid.color': '#dddddd',
    'grid.alpha': 0.7,
    'figure.facecolor': PALETA['Fondo'],
    'axes.facecolor': PALETA['Fondo'],
    'figure.constrained_layout.use': False,
})

# ============================================================================
# 2. PARÁMETROS DE LAS OLAS
# ============================================================================
WAVES = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']
N = 1.0

# ============================================================================
# 3. MODELO SIR
# ============================================================================
def sir(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    return [-beta*S*I/N, beta*S*I/N - gamma*I]

# ============================================================================
# 4. Simulación y cálculo de incidencia y su cambio
# ============================================================================
t = np.linspace(0, 200, 1000)
```

```
# Precalc rango X igual para todos
xmax = 200

fig, axes = plt.subplots(3, 2, figsize=(12, 10), sharey=False,␣
 ↪constrained_layout=False)
axes = axes.flatten()

for ax, w in zip(axes, WAVES):
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S, I = sol.y
    beta = w['R0']*w['gamma']
    # incidencia (nuevos casos por unidad de tiempo)
    Inc = beta * S * I
    # tasa de cambio de incidencia (derivada numérica)
    dInc = np.gradient(Inc, t)

    # graficar incidencia
    ax.plot(t, Inc, '-',  label=r'Incidencia $\beta SI$', color=PALETA['Inc'],␣
 ↪linewidth=2.0)
    # graficar tasa de cambio
    ax.plot(t, dInc, '--', label=r'$\mathrm{d}(Inc)/\mathrm{d}t$',␣
 ↪color=PALETA['dInc'], linewidth=2.0)

    # recuadro parámetros
    txt = (rf"$R_0={w['R0']:.2f}$, "
           rf"$D={w['D']}\,$d, "
           rf"$s_0={w['s0']:.3f}$")
    ax.text(0.02, 0.95, txt, transform=ax.transAxes,
            ha='left', va='top', fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3",
                      facecolor='white', edgecolor='#888888', alpha=0.7))

    # leyenda fuera
    leg = ax.legend(loc='upper left', bbox_to_anchor=(1.02, 0.98),
                    frameon=True, framealpha=0.7, fontsize=10, handlelength=1.5)
    leg.get_frame().set_edgecolor('#aaaaaa')

    # ejes
    ax.set_xlim(0, xmax)
    ax.set_ylim(0, max(Inc.max(), dInc.max())*1.1)
    ax.set_title(w['name'], fontweight='bold')
    ax.set_xlabel('Tiempo (días)')
    ax.set_ylabel('Tasa (fracción/día)')
    ax.grid(True)
    ax.minorticks_on()
```

```
# título general y ajustes
fig.suptitle(r'\textbf{Figura 4.6(a-d): Incidencia y Tasa de Cambio por Ola}',␣
 ↪y=0.97, fontsize=16)
fig.subplots_adjust(top=0.90, hspace=0.35, wspace=0.35)
fig.savefig('Figura_4_6_Incidencia_dInc_Elegante.pdf')
plt.show()
```

Figura 4.6(a–d): Incidencia y Tasa de Cambio por Ola



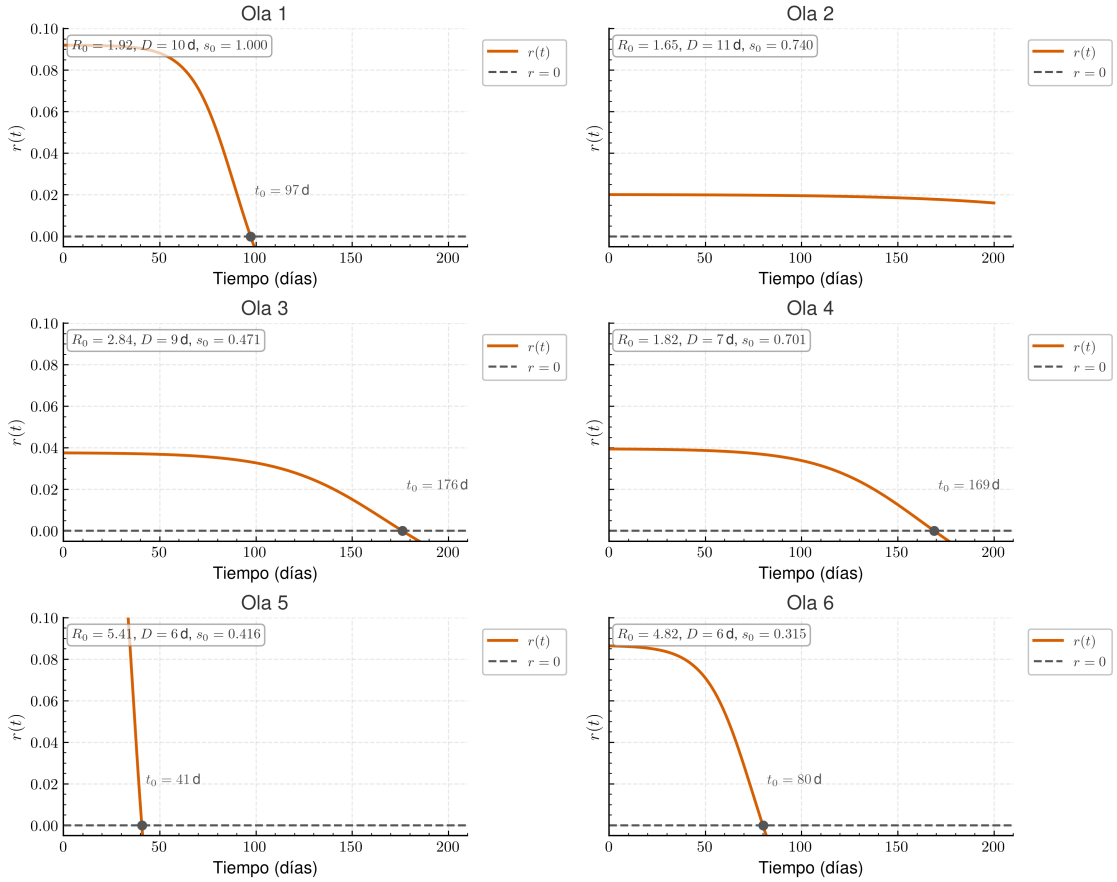
[31]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ============================================================================
# 1. PALETA Y ESTILO (mismo perfil ultra-profesional)
# ============================================================================
PALETA = {
    'r': '#D55E00',       # Naranja quemado para tasa de crecimiento
    'Umbral0': '#555555',# Gris para r=0
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
```

```
}
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['Arial','Helvetica','DejaVu Sans'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
    'font.size': 12,
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'figure.titlesize': 16,
    'text.color': PALETA['Texto'],
    'figure.dpi': 300,
    'savefig.format': 'pdf',
    'savefig.bbox': 'tight',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.linewidth': 1.0,
    'grid.linestyle': '--',
    'grid.color': '#dddddd',
    'grid.alpha': 0.7,
    'figure.facecolor': PALETA['Fondo'],
    'axes.facecolor': PALETA['Fondo'],
    'figure.constrained_layout.use': False,
})

# ============================================================================
# 2. PARÁMETROS DE LAS OLAS
# ============================================================================
WAVES = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']
N = 1.0

# ============================================================================
# 3. MODELO SIR (para obtener S(t), I(t))
# ============================================================================
def sir(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    return [-beta * S * I / N,
```

```
            beta * S * I / N - gamma * I]

# ============================================================================
# 4. CÁLCULO DE r(t) = (1/I) dI/dt =  S/N -
# ============================================================================
t = np.linspace(0, 200, 1000)
# pre-calcular máximo t hasta r cruza cero
t_zero_list = []
for w in WAVES:
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S, I = sol.y
    r = w['R0'] * w['gamma'] * S / N - w['gamma']
    idx = np.where(r <= 0)[0]
    t_zero_list.append(t[idx[0]] if idx.size else t[-1])
t_max = max(t_zero_list) * 1.05

# ============================================================================
# 5. GRÁFICO: Tasa de Crecimiento Epidémico por Ola
# ============================================================================
fig, axes = plt.subplots(3, 2, figsize=(12, 10), sharey=True,␣
 ↪constrained_layout=False)
axes = axes.flatten()

for ax, w in zip(axes, WAVES):
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S, I = sol.y
    r = w['R0'] * w['gamma'] * S / N - w['gamma']

    # Línea de r(t)
    ax.plot(t, r, '-', color=PALETA['r'], linewidth=2.0, label=r'$r(t)$')
    # Umbral r=0
    ax.axhline(0, linestyle='--', color=PALETA['Umbral0'], linewidth=1.5,␣
 ↪label=r'$r=0$')

    # Punto de cruce r=0
    idx0 = np.where(r <= 0)[0]
    if idx0.size:
        t0 = t[idx0[0]]
        ax.plot(t0, 0, 'o', color=PALETA['Umbral0'], markersize=6)
        ax.text(t0+2, 0.02, rf'$t_0={t0:.0f}\,$d',
                color=PALETA['Umbral0'], fontsize=10)

    # Recuadro de parámetros
    txt = (rf"$R_0={w['R0']:.2f}$, "
           rf"$D={w['D']}\,$d, "
```

```
            rf"$s_0={w['s0']:.3f}$")
    ax.text(0.02, 0.95, txt, transform=ax.transAxes,
            ha='left', va='top', fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3",
                      facecolor='white', edgecolor='#888888', alpha=0.7))

    # Leyenda fuera
    leg = ax.legend(loc='upper left', bbox_to_anchor=(1.02, 0.98),
                    frameon=True, framealpha=0.7, fontsize=10)
    leg.get_frame().set_edgecolor('#aaaaaa')

    # Ejes y grilla
    ax.set_xlim(0, t_max)
    ylim = max(np.max(r)*1.05, 0.1)
    ax.set_ylim(-ylim*0.05, ylim)
    ax.set_title(w['name'], fontweight='bold')
    ax.set_xlabel('Tiempo (días)')
    ax.set_ylabel(r'$r(t)$')
    ax.grid(True)
    ax.minorticks_on()

# Título general y ajustes finales
fig.suptitle(r'\textbf{Figura 4.7(a-d): Tasa de Crecimiento Epidémico $r(t)$␣
 ↪por Ola}', y=0.97, fontsize=16)
fig.subplots_adjust(top=0.90, hspace=0.35, wspace=0.35)
fig.savefig('Figura_4_7_Crecimiento_Epidemico.pdf')
plt.show()
```

Figura 4.7(a–d): Tasa de Crecimiento Epidémico $r(t)$ por Ola



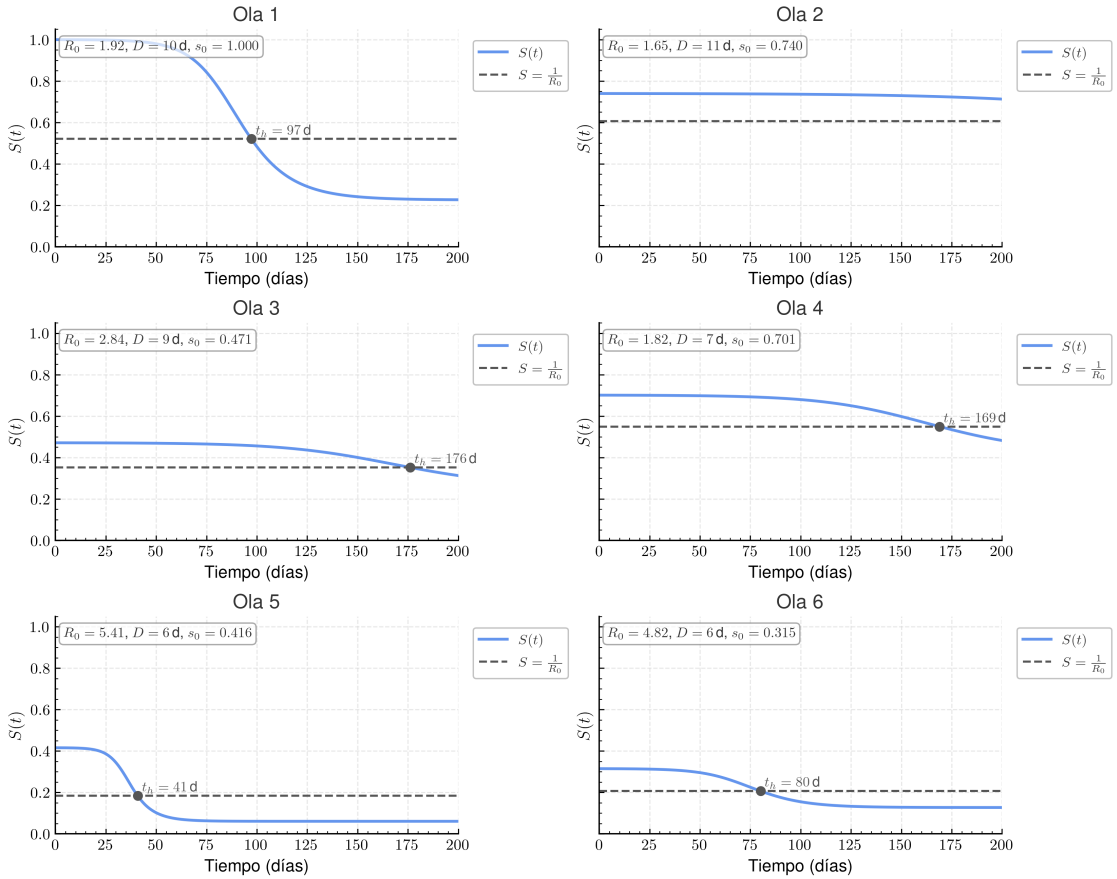
[35]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ============================================================================
# 1. PALETA Y ESTILO PROFESIONAL
# ============================================================================
PALETA = {
    'S': '#6495ED',        # Azul claro UAM
    'Umbral': '#555555',   # Gris para línea umbral
    'Texto': '#333333',
    'Fondo': '#FFFFFF',
}
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['Arial','Helvetica','DejaVu Sans'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage[dvipsnames]{xcolor}',
```

```
    'font.size': 12,
    'axes.labelsize': 12,
    'axes.titlesize': 14,
    'figure.titlesize': 16,
    'text.color': PALETA['Texto'],
    'figure.dpi': 300,
    'savefig.format': 'pdf',
    'savefig.bbox': 'tight',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.linewidth': 1.0,
    'grid.linestyle': '--',
    'grid.color': '#dddddd',
    'grid.alpha': 0.7,
    'figure.facecolor': PALETA['Fondo'],
    'axes.facecolor': PALETA['Fondo'],
    'figure.constrained_layout.use': False,
})

# ============================================================================
# 2. PARÁMETROS DE LAS OLAS
# ============================================================================
WAVES = [
    {'name': 'Ola 1', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2', 'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3', 'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4', 'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5', 'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']
N = 1.0

# ============================================================================
# 3. MODELO SIR (para obtener S(t))
# ============================================================================
def sir(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    return [-beta * S * I / N,
             beta * S * I / N - gamma * I]

# ============================================================================
# 4. SIMULACIÓN Y CÁLCULO DE S(t) Y UMBRAL
# ============================================================================
t = np.linspace(0, 200, 1000)
```

```
S_curves = []
threshold_times = []

for w in WAVES:
    sol = solve_ivp(lambda tt, yy: sir(tt, yy, w['R0'], w['gamma']),
                    [0, 200], [w['s0'], 1e-4], t_eval=t)
    S = sol.y[0]
    S_curves.append(S)
    # encontrar cruce de S = 1/R0 (umbral de inmunidad)
    idx = np.where(S <= 1.0 / w['R0'])[0]
    threshold_times.append(t[idx[0]] if idx.size else np.nan)

# ============================================================================
# 5. GRÁFICA: S(t) POR OLA
# ============================================================================
fig, axes = plt.subplots(3, 2, figsize=(12, 10), sharey=True,␣
 ↪constrained_layout=False)
axes = axes.flatten()

for ax, w, S, t_thr in zip(axes, WAVES, S_curves, threshold_times):
    # Graficar S(t)
    ax.plot(t, S, '-', color=PALETA['S'], linewidth=2.0, label=r'$S(t)$')
    # Línea umbral S=1/R0
    ax.axhline(1.0/w['R0'], linestyle='--', color=PALETA['Umbral'], linewidth=1.
 ↪5,
               label=rf'$S=\frac{{1}}{{R_0}}$')
    # Punto de cruce
    if not np.isnan(t_thr):
        ax.plot(t_thr, 1.0/w['R0'], 'o', color=PALETA['Umbral'], markersize=6)
        ax.text(t_thr+2, 1.0/w['R0']+0.02,
                rf'$t_h={t_thr:.0f}\,$d',
                color=PALETA['Umbral'], fontsize=10)

    # Recuadro de parámetros
    params_txt = (rf"$R_0={w['R0']:.2f}$, "
                  rf"$D={w['D']}\,$d, "
                  rf"$s_0={w['s0']:.3f}$")
    ax.text(0.02, 0.95, params_txt, transform=ax.transAxes,
            ha='left', va='top', fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3",
                      facecolor='white', edgecolor='#888888', alpha=0.7))

    # Leyenda fuera
    leg = ax.legend(loc='upper left', bbox_to_anchor=(1.02, 0.98),
                    frameon=True, framealpha=0.7, fontsize=10)
    leg.get_frame().set_edgecolor('#aaaaaa')
```

```
    # Ejes y grilla
    ax.set_xlim(0, 200)
    ax.set_ylim(0, 1.05)
    ax.set_title(w['name'], fontweight='bold')
    ax.set_xlabel('Tiempo (días)')
    ax.set_ylabel(r'$S(t)$')
    ax.grid(True)
    ax.minorticks_on()

# Título general y ajustes
fig.suptitle(r'\textbf{Figura 4.9(a-d): Fracción de Susceptibles $S(t)$ por␣
 ↪Ola}', y=0.97, fontsize=16)
fig.subplots_adjust(top=0.90, hspace=0.35, wspace=0.35)
fig.savefig('Figura_4_9_Susceptibles_Evolucion.pdf')
plt.show()
```

Figura 4.9(a–d): Fracción de Susceptibles $S(t)$ por Ola



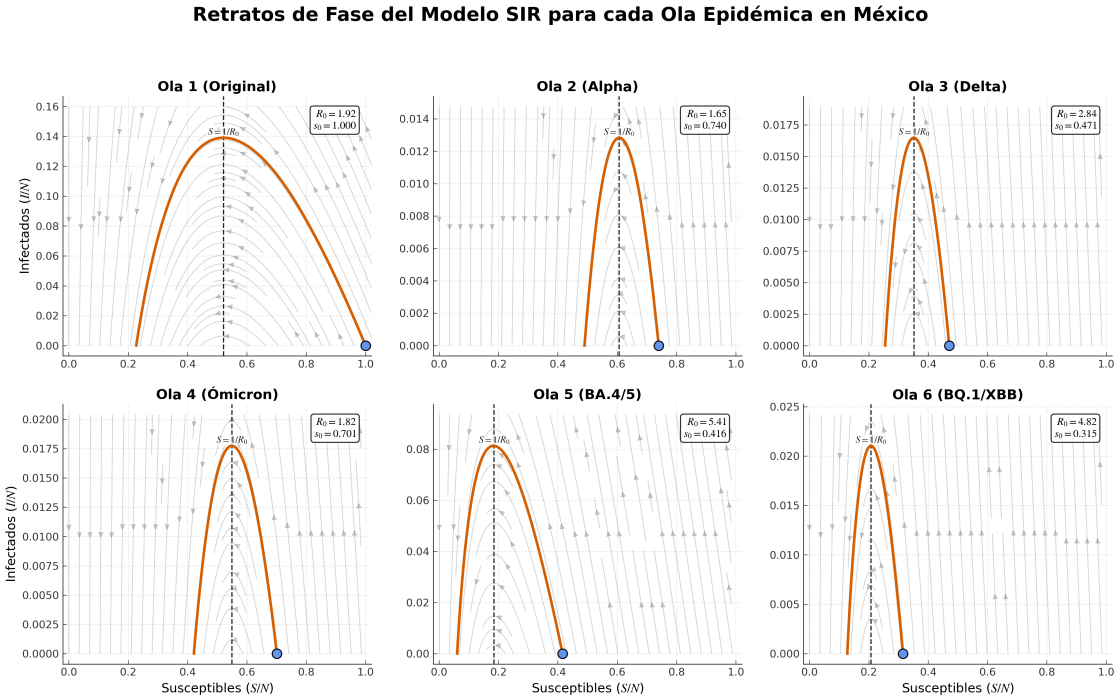
[50]:
```
import os
import re
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from matplotlib.colors import to_rgba

# ==============================================================================
# 1. ESTILO Y PALETA DE COLORES REFINADA
# ==============================================================================
PALETTE = {
    'trajectory': '#D55E00', # Naranja quemado para la curva principal
    'start_point': '#6495ED', # Azul claro para el punto inicial
    'start_point_edge': '#000000',
    'nullcline': '#222222',
    'vector_field': to_rgba('#555555', 0.25), # Más sutil
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

def apply_publication_style():
    """Aplica un estilo limpio y profesional para publicaciones científicas."""
    plt.rcParams.update({
        'font.family': 'sans-serif',
        'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
        'font.size': 11,
        'axes.titlesize': 13,
        'axes.labelsize': 12,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
        'legend.fontsize': 10,
        'axes.linewidth': 0.8,
        'lines.linewidth': 2.0,
        'grid.linestyle': '--',
        'grid.color': PALETTE['grid'],
        'figure.facecolor': PALETTE['background'],
        'axes.facecolor': PALETTE['background'],
        'axes.spines.top': False,
        'axes.spines.right': False,
        'figure.dpi': 300,
        'savefig.bbox': 'tight'
    })

# ==============================================================================
# 2. CONFIGURACIÓN DE LAS 6 OLAS
# ==============================================================================
WAVES = [
```

```
    {'name': 'Ola 1 (Original)', 'R0': 1.92, 's0': 1.000, 'D': 10},
    {'name': 'Ola 2 (Alpha)',    'R0': 1.65, 's0': 0.740, 'D': 11},
    {'name': 'Ola 3 (Delta)',    'R0': 2.84, 's0': 0.471, 'D': 9},
    {'name': 'Ola 4 (Ómicron)',  'R0': 1.82, 's0': 0.701, 'D': 7},
    {'name': 'Ola 5 (BA.4/5)',   'R0': 5.41, 's0': 0.416, 'D': 6},
    {'name': 'Ola 6 (BQ.1/XBB)', 'R0': 4.82, 's0': 0.315, 'D': 6},
]
for w in WAVES:
    w['gamma'] = 1.0 / w['D']

# ============================================================================
# 3. MODELO SIR 2D
# ============================================================================
def sir_rhs(t, y, R0, gamma):
    """
    Función del lado derecho del sistema de EDOs del modelo SIR.
    """
    S, I = y
    beta = R0 * gamma
    dS_dt = -beta * S * I
    dI_dt = beta * S * I - gamma * I
    return [dS_dt, dI_dt]

# ============================================================================
# 4. SIMULACIÓN (CORRECCIÓN FINAL)
# ============================================================================
def simulate_trajectory(R0, gamma, s0):
    """Simula la trayectoria asegurando que termine en I=0."""
    # CORRECCIÓN FINAL: Tiempo extendido a 1200 días para asegurar que la Ola 2␣
 ↪complete su ciclo
    t_span = [0, 1200]
    t_eval = np.linspace(t_span[0], t_span[1], 3000)

    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-6], # I0 muy pequeño pero no cero
        args=(R0, gamma),
        t_eval=t_eval,
        dense_output=True
    )

    # Asegurar que la curva termine exactamente en I=0 en la gráfica
    s_traj, i_traj = sol.y
    i_traj[i_traj < 1e-5] = 0 # Limpieza de valores residuales de I
    return s_traj, i_traj
```

```python
# ============================================================================
# 5. FUNCIÓN DE GRAFICADO PARA UN SOLO EJE
# ============================================================================
def plot_on_axis(ax, wave):
    """Dibuja el retrato de fases para una ola en un eje (ax) determinado."""
    name, R0, s0, gamma = wave['name'], wave['R0'], wave['s0'], wave['gamma']

    # --- Simulación ---
    S_traj, I_traj = simulate_trajectory(R0, gamma, s0)
    i_max = np.max(I_traj)

    # --- Campo Vectorial (Streamplot) ---
    s_grid = np.linspace(0, 1.02, 25)
    i_grid = np.linspace(0, i_max * 1.15 if i_max > 0 else 0.1, 25)
    Sg, Ig = np.meshgrid(s_grid, i_grid)
    dS, dI = sir_rhs(None, (Sg, Ig), R0, gamma)

    ax.streamplot(
        Sg, Ig, dS, dI,
        density=1.0,
        color=PALETTE['vector_field'],
        linewidth=0.8
    )

    # --- Curva de Trayectoria ---
    ax.plot(S_traj, I_traj, color=PALETTE['trajectory'], lw=2.5, zorder=4)

    # --- Punto Inicial ---
    ax.scatter(
        [s0], [I_traj[0]],
        color=PALETTE['start_point'],
        edgecolor=PALETTE['start_point_edge'],
        s=80,
        zorder=5,
        alpha=1.0
    )

    # --- Nulclina S = 1/R0 ---
    if R0 > 1:
        ax.axvline(1.0/R0, ls='--', color=PALETTE['nullcline'], lw=1.2,␣
↪zorder=3)
        ax.text(
            1.0/R0, i_max * 1.05 if i_max > 0 else 0.08, r'$S=1/R_0$',
            ha='center', va='top', fontsize=9, color=PALETTE['nullcline'],
            bbox=dict(facecolor='white', alpha=0.6, edgecolor='none', pad=1)
        )
```

```
    # --- Títulos y Formato de Ejes ---
    ax.set_title(name, fontweight='bold', pad=5)
    ax.set_xlim(-0.02, 1.02)
    ax.set_ylim(-i_max * 0.05, i_max * 1.2 if i_max > 0 else 0.1)

    # --- Cuadro de Parámetros ---
    param_text = f"$R_0={R0:.2f}$\n$s_0={s0:.3f}$"
    ax.text(
        0.95, 0.95, param_text, transform=ax.transAxes,
        ha='right', va='top', fontsize=10,
        bbox=dict(boxstyle="round,pad=0.3", facecolor=to_rgba('white', 0.8),
                  edgecolor=PALETTE['nullcline'])
    )

    ax.grid(True)

# ==============================================================================
# 6. EJECUCIÓN PRINCIPAL PARA CREAR FIGURA COMPUESTA
# ==============================================================================
if __name__ == "__main__":
    apply_publication_style()

    fig, axes = plt.subplots(2, 3, figsize=(15, 9))

    for i, wave in enumerate(WAVES):
        ax = axes.flat[i]
        plot_on_axis(ax, wave)

    # --- Limpieza de etiquetas ---
    for i, ax in enumerate(axes.flat):
        if i % 3 != 0:
            ax.set_ylabel('')
        else:
            ax.set_ylabel('Infectados ($I/N$)')

        if i < 3:
            ax.set_xlabel('')
        else:
            ax.set_xlabel('Susceptibles ($S/N$)')

    fig.suptitle('Retratos de Fase del Modelo SIR para cada Ola Epidémica en␣
 ↪México',
                 fontsize=18, fontweight='bold', y=1.03)

    plt.tight_layout(pad=1.5)

    output_dir = 'figuras_mejoradas'
```

```
    os.makedirs(output_dir, exist_ok=True)

    final_path = os.path.join(output_dir,␣
↪'Retrato_de_Fases_SIR_Olas_Definitivo')
    fig.savefig(f"{final_path}.png", dpi=300)
    fig.savefig(f"{final_path}.pdf")

    plt.show()
    plt.close(fig)

    print(f"Figura definitiva guardada en:\n{final_path}.png\n{final_path}.pdf")
```




```
Figura definitiva guardada en:
figuras_mejoradas/Retrato_de_Fases_SIR_Olas_Definitivo.png
figuras_mejoradas/Retrato_de_Fases_SIR_Olas_Definitivo.pdf
```

[58]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import find_peaks
from matplotlib.lines import Line2D

# ================================================================================
```

```python
# 1. CONFIGURACIÓN GENERAL Y ESTILO
# ==============================================================================
# Paleta de colores optimizada para claridad y publicación
PALETTE = {
    'model': '#D55E00',       # Naranja quemado (distintivo y claro)
    'data': '#0072B2',        # Azul zafiro (contrasta bien con naranja)
    'annotations': '#333333', # Gris oscuro para texto
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

# Estilo de publicación (basado en seaborn-whitegrid con ajustes)
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 13,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 12,
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.5,
    'figure.facecolor': PALETTE['background'],
    'axes.facecolor': PALETTE['background'],
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 300,
    'savefig.bbox': 'tight',
    'axes.titleweight': 'bold',
    'axes.titlepad': 15,
})

# ==============================================================================
# 2. PARÁMETROS Y DATOS (Valores de tu proyecto)
# ==============================================================================
WAVE_PARAMETERS_MODEL = [
    {'name': 'Ola 1 (Original)', 'R0': 1.92, 'D': 10, 's0_theoretical': 1.000},
    {'name': 'Ola 2 (Alpha)',    'R0': 1.65, 'D': 11, 's0_theoretical': 0.740},
    {'name': 'Ola 3 (Delta)',    'R0': 2.84, 'D': 9,  's0_theoretical': 0.471},
    {'name': 'Ola 4 (Ómicron)',  'R0': 1.82, 'D': 7,  's0_theoretical': 0.701},
    {'name': 'Ola 5 (BA.4/5)',   'R0': 5.41, 'D': 6,  's0_theoretical': 0.416},
    {'name': 'Ola 6 (BQ.1/XBB)', 'R0': 4.82, 'D': 6,  's0_theoretical': 0.315},
]
for w in WAVE_PARAMETERS_MODEL:
    w['gamma'] = 1.0 / w['D']
```

```python
# Factores de corrección por subregistro
DETECTION_FACTORS = {
    'Ola 1 (Original)': 0.091,
    'Ola 2 (Alpha)': 0.091,
    'Ola 3 (Delta)': 0.111,
    'Ola 4 (Ómicron)': 0.143,
    'Ola 5 (BA.4/5)': 0.167,
    'Ola 6 (BQ.1/XBB)': 0.167
}

# ==============================================================================
# 3. PROCESAMIENTO DE DATOS REALES (Tu función robusta)
# ==============================================================================
def process_real_data(filepath, wave_params, detection_factors):
    df_raw = pd.read_csv(filepath)
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()

    poblacion_mx = df_nacional['poblacion'].iloc[0]
    casos_diarios = df_nacional.iloc[0, 3:].astype(float)
    df_data = pd.DataFrame(
        {'casos_diarios': casos_diarios.values},
        index=pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
    )

    casos_suavizados_7d = df_data['casos_diarios'].rolling(7, center=True,␣
 ↪min_periods=1).mean()
    indices_valles, _ = find_peaks(-casos_suavizados_7d, distance=120,␣
 ↪prominence=1000)
    fronteras = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(df_data)-1])))

    df_data['ola_num'] = 0
    for i in range(len(fronteras) - 1):
        if i < len(wave_params):
            df_data.iloc[fronteras[i]:fronteras[i+1], df_data.columns.
 ↪get_loc('ola_num')] = i + 1

    df_data['casos_suavizados_14d'] = df_data['casos_diarios'].rolling(14,␣
 ↪center=True, min_periods=1).mean()

    df_data['casos_ajustados'] = 0.0
    for i, p in enumerate(wave_params):
        mask = df_data['ola_num'] == (i + 1)
        factor = detection_factors.get(p['name'], 1.0)
        df_data.loc[mask, 'casos_ajustados'] = df_data.loc[mask,␣
 ↪'casos_suavizados_14d'] / factor
```

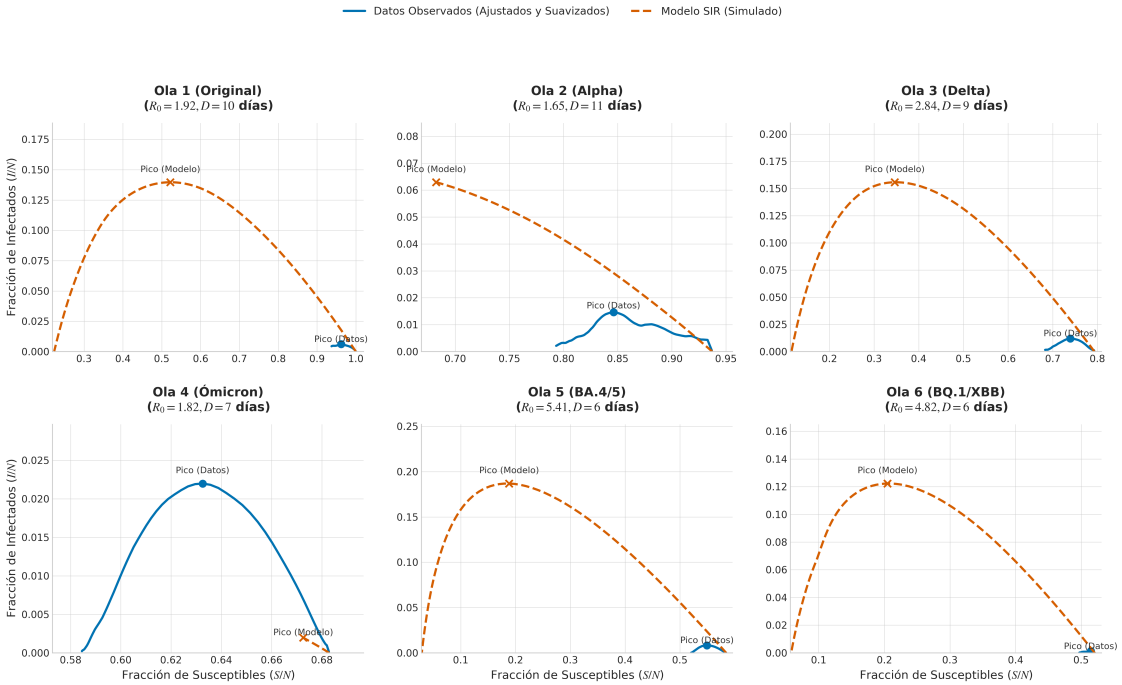
```
    all_waves_data = []
    casos_acum_global = 0
    for i, p in enumerate(wave_params):
        wave_num = i + 1
        df_wave = df_data[df_data['ola_num'] == wave_num].copy()
        if df_wave.empty:
            continue

        infectados_activos = df_wave['casos_ajustados'].rolling(window=p['D'],␣
 ↪min_periods=1).sum()
        df_wave['I_frac_datos'] = infectados_activos / poblacion_mx

        s0_real_ola = (poblacion_mx - casos_acum_global) / poblacion_mx
        df_wave['s0_real'] = s0_real_ola

        casos_acum_ola = df_wave['casos_ajustados'].cumsum()
        fraccion_infectada_ola = casos_acum_ola / poblacion_mx
        df_wave['S_frac_datos'] = s0_real_ola - fraccion_infectada_ola

        all_waves_data.append(df_wave)
        casos_acum_global += df_wave['casos_ajustados'].sum()

    return pd.concat(all_waves_data)

# ============================================================================
# 4. SIMULACIÓN DEL MODELO SIR
# ============================================================================
def sir_rhs(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    return [dSdt, dIdt]

def simulate_trajectory(R0, gamma, s0):
    t_span = [0, 300]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)
    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-7],
        args=(R0, gamma),
        t_eval=t_eval,
        method='RK45'
    )
    s_traj, i_traj = sol.y
```

```
    i_traj[i_traj < 0] = 0
    return s_traj, i_traj

# ==============================================================================
# 5. EJECUCIÓN Y GRAFICACIÓN
# ==============================================================================
csv_filepath = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
output_dir = 'figuras_tesis'
os.makedirs(output_dir, exist_ok=True)

df_datos_proc = process_real_data(
    csv_filepath, WAVE_PARAMETERS_MODEL, DETECTION_FACTORS
)

fig, axes = plt.subplots(2, 3, figsize=(18, 11))

for i, (ax, params) in enumerate(zip(axes.flat, WAVE_PARAMETERS_MODEL)):
    wave_num = i + 1
    datos_ola = df_datos_proc[df_datos_proc['ola_num'] == wave_num]

    if not datos_ola.empty:
        ax.plot(datos_ola['S_frac_datos'], datos_ola['I_frac_datos'],
                color=PALETTE['data'], ls='-', label='Datos Observados')

        idx_pico_datos = datos_ola['I_frac_datos'].idxmax()
        s_pico_datos = datos_ola.loc[idx_pico_datos, 'S_frac_datos']
        i_pico_datos = datos_ola.loc[idx_pico_datos, 'I_frac_datos']
        ax.plot(s_pico_datos, i_pico_datos, 'o', color=PALETTE['data'],␣
 ↪markersize=8)
        ax.text(s_pico_datos, i_pico_datos * 1.05, 'Pico (Datos)',
                color=PALETTE['annotations'], ha='center', va='bottom',␣
 ↪fontsize=10)

    s0_para_simulacion = datos_ola['s0_real'].iloc[0] if not datos_ola.empty␣
 ↪else params['s0_theoretical']
    S_model, I_model = simulate_trajectory(params['R0'], params['gamma'],␣
 ↪s0_para_simulacion)
    ax.plot(S_model, I_model, color=PALETTE['model'], ls='--', label='Modelo␣
 ↪SIR')

    idx_pico_modelo = np.argmax(I_model)
    s_pico_modelo = S_model[idx_pico_modelo]
    i_pico_modelo = I_model[idx_pico_modelo]
    ax.plot(s_pico_modelo, i_pico_modelo, 'x', color=PALETTE['model'],␣
 ↪markersize=8, markeredgewidth=2)
    ax.text(s_pico_modelo, i_pico_modelo * 1.05, 'Pico (Modelo)',
```

```
            color=PALETTE['annotations'], ha='center', va='bottom', fontsize=10)

    ax.set_title(f"{params['name']}\n($R_0 = {params['R0']:.2f}, D =␣
 ↪{params['D']}$ días)")

    i_max_total = max(datos_ola['I_frac_datos'].max() if not datos_ola.empty␣
 ↪else 0, I_model.max())
    ax.set_ylim(bottom=0, top=i_max_total * 1.35)

    s_min_total = min(datos_ola['S_frac_datos'].min() if not datos_ola.empty␣
 ↪else 1, S_model.min())
    s_max_total = max(datos_ola['S_frac_datos'].max() if not datos_ola.empty␣
 ↪else 1, S_model.max())
    ax.set_xlim(left=s_min_total * 0.98, right=s_max_total * 1.02)

    if i % 3 == 0:
        ax.set_ylabel('Fracción de Infectados ($I/N$)')
    if i >= 3:
        ax.set_xlabel('Fracción de Susceptibles ($S/N$)')

legend_elements = [
    Line2D([0], [0], color=PALETTE['data'], lw=2.5, ls='-', label='Datos␣
 ↪Observados (Ajustados y Suavizados)'),
    Line2D([0], [0], color=PALETTE['model'], lw=2.5, ls='--', label='Modelo SIR␣
 ↪(Simulado)')
]
fig.legend(handles=legend_elements, loc='upper center', bbox_to_anchor=(0.5, 1.
 ↪0), ncol=2, frameon=False)
fig.suptitle('Figura 4.6: Comparativa de Trayectorias en el Espacio de Fase␣
 ↪(Modelo vs. Datos)',
             fontsize=20, fontweight='bold', y=1.06)

plt.tight_layout(pad=2.0, rect=[0, 0, 1, 0.96])

final_path = os.path.join(output_dir, 'Figura_4_6_Validacion_Fase')
fig.savefig(f"{final_path}.png", dpi=300)
fig.savefig(f"{final_path}.pdf")

print(f"Figuras de validación guardadas con éxito en la carpeta '{output_dir}'.
 ↪")
```

```
Figuras de validación guardadas con éxito en la carpeta 'figuras_tesis'.
```

Figura 4.6: Comparativa de Trayectorias en el Espacio de Fase (Modelo vs. Datos)



[72]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import find_peaks
from matplotlib.lines import Line2D

# ==============================================================================
# 1. CONFIGURACIÓN GENERAL Y ESTILO
# ==============================================================================
# Paleta de colores optimizada para claridad y publicación
PALETTE = {
    'model': '#D55E00',       # Naranja quemado (distintivo y claro)
    'data': '#0072B2',        # Azul zafiro (contrasta bien con naranja)
    'annotations': '#333333',# Gris oscuro para texto
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

# Estilo de publicación (basado en seaborn-whitegrid con ajustes)
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
```

```
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 13,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 12,
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.5,
    'figure.facecolor': PALETTE['background'],
    'axes.facecolor': PALETTE['background'],
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 300,
    'savefig.bbox': 'tight',
    'axes.titleweight': 'bold',
    'axes.titlepad': 15,
})

# ==============================================================================
# 2. PARÁMETROS Y DATOS (Valores del Modelo NLS)
# ==============================================================================
# --- AJUSTE: Se usan los R0 del modelo de Mínimos Cuadrados No Lineales ---
WAVE_PARAMETERS_MODEL = [
    {'name': 'Ola 1 (Original)',    'R0': 1.733, 'D': 10, 's0_theoretical': 1.
↪000},
    {'name': 'Ola 2 (Alpha)',       'R0': 1.626, 'D': 11, 's0_theoretical': 0.
↪740},
    {'name': 'Ola 3 (Delta)',       'R0': 1.807, 'D': 9,  's0_theoretical': 0.
↪471},
    {'name': 'Ola 4 (Ómicron)',     'R0': 1.201, 'D': 7,  's0_theoretical': 0.
↪701},
    {'name': 'Ola 5 (Ómicron Subs.)', 'R0': 1.328, 'D': 6,  's0_theoretical': 0.
↪416},
    {'name': 'Ola 6',               'R0': 2.008, 'D': 6,  's0_theoretical': 0.
↪315},
]
for w in WAVE_PARAMETERS_MODEL:
    w['gamma'] = 1.0 / w['D']

# Factores de corrección por subregistro (se mantienen)
DETECTION_FACTORS = {
    'Ola 1 (Original)': 0.091,
    'Ola 2 (Alpha)': 0.091,
    'Ola 3 (Delta)': 0.111,
```

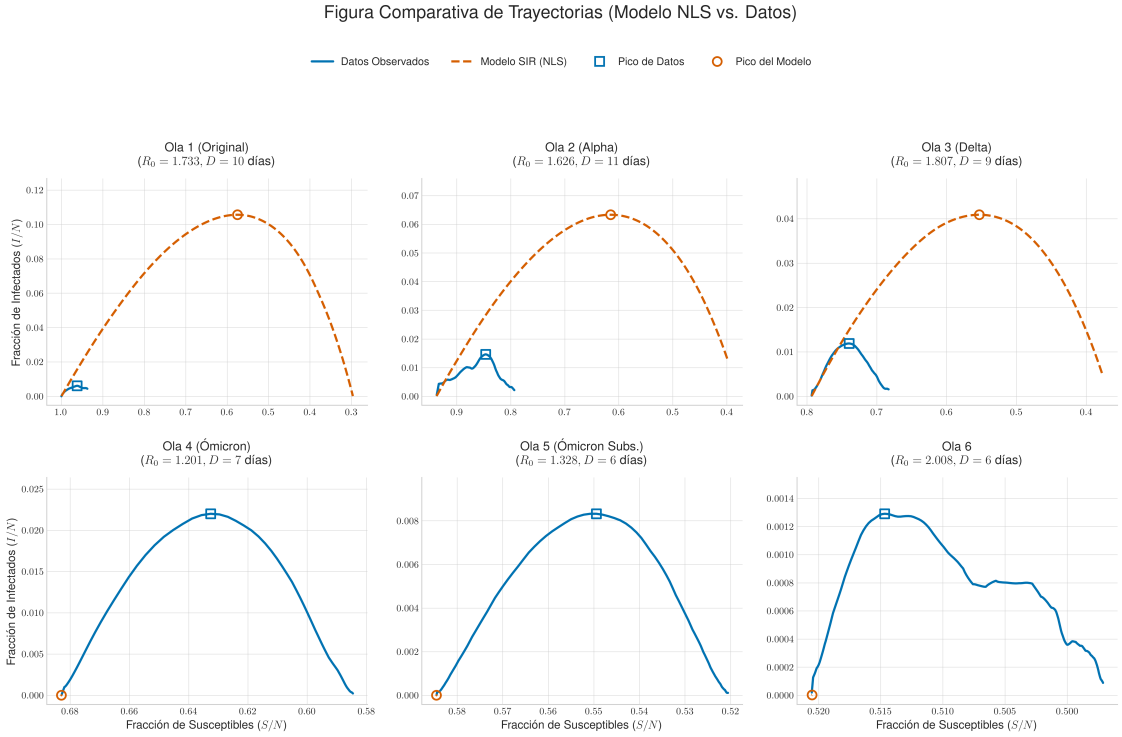
```
    'Ola 4 (Ómicron)': 0.143,
    'Ola 5 (Ómicron Subs.)': 0.167,
    'Ola 6': 0.167
}

# ==============================================================================
# 3. PROCESAMIENTO DE DATOS REALES
# ==============================================================================
def process_real_data(filepath, wave_params, detection_factors):
    df_raw = pd.read_csv(filepath)
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()

    poblacion_mx = df_nacional['poblacion'].iloc[0]
    casos_diarios = df_nacional.iloc[0, 3:].astype(float)
    df_data = pd.DataFrame(
        {'casos_diarios': casos_diarios.values},
        index=pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
    )

    casos_suavizados_7d = df_data['casos_diarios'].rolling(7, center=True,␣
 ↪min_periods=1).mean()
    indices_valles, _ = find_peaks(-casos_suavizados_7d, distance=120,␣
 ↪prominence=1000)
    fronteras = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(df_data)-1])))

    df_data['ola_num'] = 0
    # Usar los nombres de la lista de parámetros para asignar números
    wave_names = [p['name'] for p in wave_params]
    for i in range(len(fronteras) - 1):
        if i < len(wave_params):
            df_data.iloc[fronteras[i]:fronteras[i+1], df_data.columns.
 ↪get_loc('ola_num')] = i + 1

    df_data['casos_suavizados_14d'] = df_data['casos_diarios'].rolling(14,␣
 ↪center=True, min_periods=1).mean()

    df_data['casos_ajustados'] = 0.0
    for i, p in enumerate(wave_params):
        mask = df_data['ola_num'] == (i + 1)
        factor = detection_factors.get(p['name'], 1.0)
        df_data.loc[mask, 'casos_ajustados'] = df_data.loc[mask,␣
 ↪'casos_suavizados_14d'] / factor

    all_waves_data = []
    casos_acum_global = 0
```

```
    for i, p in enumerate(wave_params):
        wave_num = i + 1
        df_wave = df_data[df_data['ola_num'] == wave_num].copy()
        if df_wave.empty:
            continue

        infectados_activos = df_wave['casos_ajustados'].rolling(window=p['D'],␣
 ↪min_periods=1).sum()
        df_wave['I_frac_datos'] = infectados_activos / poblacion_mx

        s0_real_ola = (poblacion_mx - casos_acum_global) / poblacion_mx
        df_wave['s0_real'] = s0_real_ola

        casos_acum_ola = df_wave['casos_ajustados'].cumsum()
        fraccion_infectada_ola = casos_acum_ola / poblacion_mx
        df_wave['S_frac_datos'] = s0_real_ola - fraccion_infectada_ola

        all_waves_data.append(df_wave)
        casos_acum_global += df_wave['casos_ajustados'].sum()

    return pd.concat(all_waves_data)

# ============================================================================
# 4. SIMULACIÓN DEL MODELO SIR
# ============================================================================
def sir_rhs(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    return [dSdt, dIdt]

def simulate_trajectory(R0, gamma, s0, duration=400):
    """
    Simula una trayectoria SIR para una duración específica.
    Se usa una duración larga por defecto para asegurar trayectorias completas.
    """
    t_span = [0, duration]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)
    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-7],
        args=(R0, gamma),
        t_eval=t_eval,
        method='RK45'
    )
```

```
    s_traj, i_traj = sol.y
    i_traj[i_traj < 0] = 0
    return s_traj, i_traj

# ==============================================================================
# 5. EJECUCIÓN Y GRAFICACIÓN
# ==============================================================================
csv_filepath = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
output_dir = 'figuras_tesis'
os.makedirs(output_dir, exist_ok=True)

df_datos_proc = process_real_data(
    csv_filepath, WAVE_PARAMETERS_MODEL, DETECTION_FACTORS
)

fig, axes = plt.subplots(2, 3, figsize=(18, 11))

for i, (ax, params) in enumerate(zip(axes.flat, WAVE_PARAMETERS_MODEL)):
    wave_num = i + 1
    datos_ola = df_datos_proc[df_datos_proc['ola_num'] == wave_num]

    # --- Simulación con los parámetros NLS ---
    s0_para_simulacion = datos_ola['s0_real'].iloc[0] if not datos_ola.empty␣
 ↪else params['s0_theoretical']
    S_model, I_model = simulate_trajectory(params['R0'], params['gamma'],␣
 ↪s0_para_simulacion)

    # --- Graficar Curvas Principales ---
    if not datos_ola.empty:
        ax.plot(datos_ola['S_frac_datos'], datos_ola['I_frac_datos'],
                color=PALETTE['data'], ls='-', label='Datos Observados',␣
 ↪zorder=2)

    visible_model_mask = I_model > (I_model.max() * 0.001)
    ax.plot(S_model[visible_model_mask], I_model[visible_model_mask],
            color=PALETTE['model'], ls='--', label='Modelo SIR', zorder=3)

    # --- Marcar Picos ---
    idx_pico_modelo = np.argmax(I_model)
    ax.scatter(S_model[idx_pico_modelo], I_model[idx_pico_modelo],
               marker='o', s=100, facecolors='none',␣
 ↪edgecolors=PALETTE['model'],
               linewidth=2, zorder=4)

    if not datos_ola.empty:
        idx_pico_datos = datos_ola['I_frac_datos'].idxmax()
```

```
        ax.scatter(datos_ola.loc[idx_pico_datos, 'S_frac_datos'], datos_ola.
↪loc[idx_pico_datos, 'I_frac_datos'],
                   marker='s', s=100, facecolors='none',␣
↪edgecolors=PALETTE['data'],
                   linewidth=2, zorder=4)

    # --- Configuración de Ejes ---
    ax.set_title(f"{params['name']}\n($R_0 = {params['R0']:.3f}, D =␣
↪{params['D']}$ días)")

    i_max_total = max(datos_ola['I_frac_datos'].max() if not datos_ola.empty␣
↪else 0, I_model.max())
    s_min_total = min(datos_ola['S_frac_datos'].min() if not datos_ola.empty␣
↪else 1, S_model[visible_model_mask].min())
    s_max_total = max(datos_ola['S_frac_datos'].max() if not datos_ola.empty␣
↪else 1, S_model.max())

    s_range = s_max_total - s_min_total
    s_padding = s_range * 0.05

    ax.set_ylim(bottom=-i_max_total*0.05, top=i_max_total * 1.2)
    ax.set_xlim(left=s_max_total + s_padding, right=s_min_total - s_padding)

    if i % 3 == 0:
        ax.set_ylabel('Fracción de Infectados ($I/N$)')
    if i >= 3:
        ax.set_xlabel('Fracción de Susceptibles ($S/N$)')

# --- Leyenda Unificada ---
legend_elements = [
    Line2D([0], [0], color=PALETTE['data'], lw=2.5, ls='-', label='Datos␣
↪Observados'),
    Line2D([0], [0], color=PALETTE['model'], lw=2.5, ls='--', label='Modelo SIR␣
↪(NLS)'),
    Line2D([0], [0], marker='s', color='w', label='Pico de Datos',
           markerfacecolor='none', markeredgecolor=PALETTE['data'],␣
↪markersize=10, markeredgewidth=2),
    Line2D([0], [0], marker='o', color='w', label='Pico del Modelo',
           markerfacecolor='none', markeredgecolor=PALETTE['model'],␣
↪markersize=10, markeredgewidth=2)
]
fig.legend(handles=legend_elements, loc='upper center', bbox_to_anchor=(0.5, 1.
↪0), ncol=4, frameon=False)
fig.suptitle('Figura Comparativa de Trayectorias (Modelo NLS vs. Datos)',
             fontsize=20, fontweight='bold', y=1.06)
```

```
plt.tight_layout(pad=2.0, rect=[0, 0, 1, 0.95])

# --- Guardado de Archivos ---
final_path = os.path.join(output_dir, 'Figura_Validacion_Fase_NLS')
fig.savefig(f"{final_path}.png", dpi=300)
fig.savefig(f"{final_path}.pdf")

plt.show()

print(f"Figuras de validación con parámetros NLS guardadas en la carpeta␣
  ↪'{output_dir}'.")
```




```
Figuras de validación con parámetros NLS guardadas en la carpeta
'figuras_tesis'.
```

[73]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import find_peaks
from matplotlib.lines import Line2D

# ============================================================================
```

```python
# 1. CONFIGURACIÓN GENERAL Y ESTILO
# ==============================================================================
# Paleta de colores optimizada para claridad y publicación
PALETTE = {
    'model': '#D55E00',      # Naranja quemado (distintivo y claro)
    'data': '#0072B2',       # Azul zafiro (contrasta bien con naranja)
    'annotations': '#333333',# Gris oscuro para texto
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

# Estilo de publicación (basado en seaborn-whitegrid con ajustes)
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 13,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 12,
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.5,
    'figure.facecolor': PALETTE['background'],
    'axes.facecolor': PALETTE['background'],
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 300,
    'savefig.bbox': 'tight',
    'axes.titleweight': 'bold',
    'axes.titlepad': 15,
})

# ==============================================================================
# 2. PARÁMETROS Y DATOS (Valores del Modelo MLE)
# ==============================================================================
# --- AJUSTE: Se usan los R0 del modelo de Máxima Verosimilitud ---
WAVE_PARAMETERS_MODEL = [
    {'name': 'Ola 1 (Original)',    'R0': 1.739, 'D': 10, 's0_theoretical': 1.
↪000},
    {'name': 'Ola 2 (Alpha)',       'R0': 1.626, 'D': 11, 's0_theoretical': 0.
↪740},
    {'name': 'Ola 3 (Delta)',       'R0': 1.809, 'D': 9,  's0_theoretical': 0.
↪471},
    {'name': 'Ola 4 (Ómicron)',     'R0': 1.200, 'D': 7,  's0_theoretical': 0.
↪701},
```

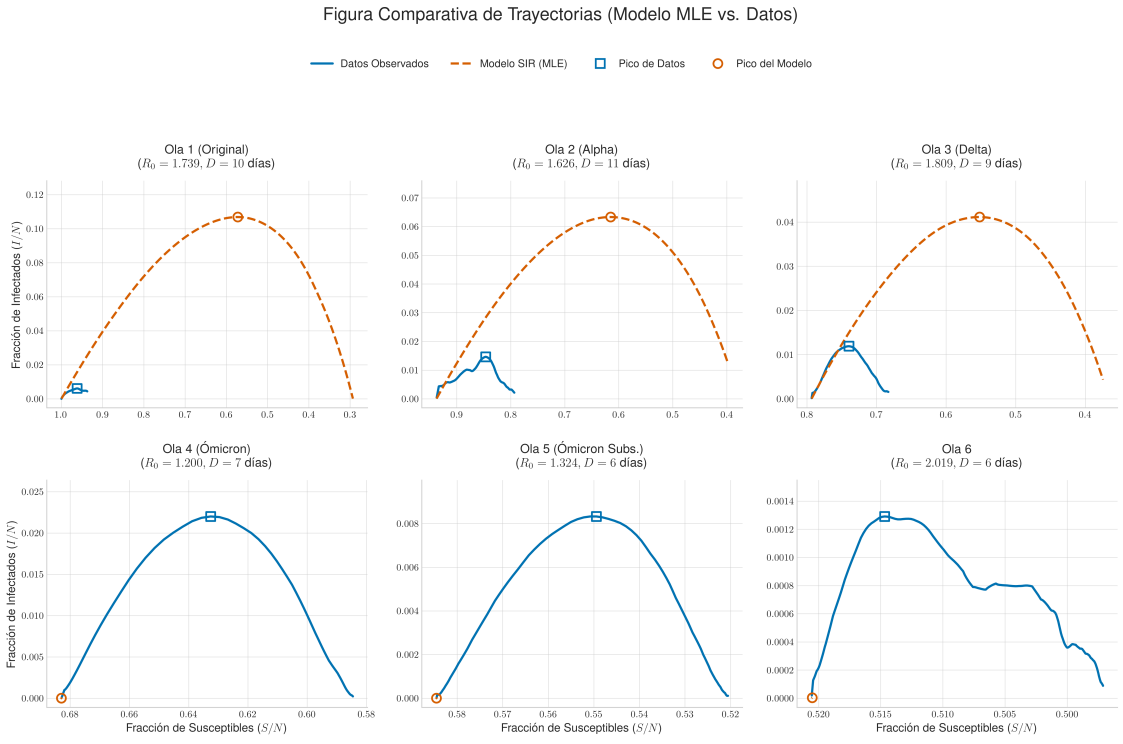
```
    {'name': 'Ola 5 (Ómicron Subs.)', 'R0': 1.324, 'D': 6,  's0_theoretical': 0.
↪416},
    {'name': 'Ola 6',               'R0': 2.019, 'D': 6,  's0_theoretical': 0.
↪315},
]
for w in WAVE_PARAMETERS_MODEL:
    w['gamma'] = 1.0 / w['D']

# Factores de corrección por subregistro (se mantienen)
DETECTION_FACTORS = {
    'Ola 1 (Original)': 0.091,
    'Ola 2 (Alpha)': 0.091,
    'Ola 3 (Delta)': 0.111,
    'Ola 4 (Ómicron)': 0.143,
    'Ola 5 (Ómicron Subs.)': 0.167,
    'Ola 6': 0.167
}

# ==============================================================================
# 3. PROCESAMIENTO DE DATOS REALES
# ==============================================================================
def process_real_data(filepath, wave_params, detection_factors):
    df_raw = pd.read_csv(filepath)
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()

    poblacion_mx = df_nacional['poblacion'].iloc[0]
    casos_diarios = df_nacional.iloc[0, 3:].astype(float)
    df_data = pd.DataFrame(
        {'casos_diarios': casos_diarios.values},
        index=pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
    )

    casos_suavizados_7d = df_data['casos_diarios'].rolling(7, center=True,␣
↪min_periods=1).mean()
    indices_valles, _ = find_peaks(-casos_suavizados_7d, distance=120,␣
↪prominence=1000)
    fronteras = np.unique(np.concatenate(([0], indices_valles,␣
↪[len(df_data)-1])))

    df_data['ola_num'] = 0
    # Usar los nombres de la lista de parámetros para asignar números
    wave_names = [p['name'] for p in wave_params]
    for i in range(len(fronteras) - 1):
        if i < len(wave_params):
            df_data.iloc[fronteras[i]:fronteras[i+1], df_data.columns.
↪get_loc('ola_num')] = i + 1
```

```
    df_data['casos_suavizados_14d'] = df_data['casos_diarios'].rolling(14,␣
 ↪center=True, min_periods=1).mean()

    df_data['casos_ajustados'] = 0.0
    for i, p in enumerate(wave_params):
        mask = df_data['ola_num'] == (i + 1)
        factor = detection_factors.get(p['name'], 1.0)
        df_data.loc[mask, 'casos_ajustados'] = df_data.loc[mask,␣
 ↪'casos_suavizados_14d'] / factor

    all_waves_data = []
    casos_acum_global = 0
    for i, p in enumerate(wave_params):
        wave_num = i + 1
        df_wave = df_data[df_data['ola_num'] == wave_num].copy()
        if df_wave.empty:
            continue

        infectados_activos = df_wave['casos_ajustados'].rolling(window=p['D'],␣
 ↪min_periods=1).sum()
        df_wave['I_frac_datos'] = infectados_activos / poblacion_mx

        s0_real_ola = (poblacion_mx - casos_acum_global) / poblacion_mx
        df_wave['s0_real'] = s0_real_ola

        casos_acum_ola = df_wave['casos_ajustados'].cumsum()
        fraccion_infectada_ola = casos_acum_ola / poblacion_mx
        df_wave['S_frac_datos'] = s0_real_ola - fraccion_infectada_ola

        all_waves_data.append(df_wave)
        casos_acum_global += df_wave['casos_ajustados'].sum()

    return pd.concat(all_waves_data)

# ============================================================================
# 4. SIMULACIÓN DEL MODELO SIR
# ============================================================================
def sir_rhs(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    return [dSdt, dIdt]

def simulate_trajectory(R0, gamma, s0, duration=400):
    """
```

```
    Simula una trayectoria SIR para una duración específica.
    Se usa una duración larga por defecto para asegurar trayectorias completas.
    """
    t_span = [0, duration]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)
    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-7],
        args=(R0, gamma),
        t_eval=t_eval,
        method='RK45'
    )
    s_traj, i_traj = sol.y
    i_traj[i_traj < 0] = 0
    return s_traj, i_traj

# ==============================================================================
# 5. EJECUCIÓN Y GRAFICACIÓN
# ==============================================================================
csv_filepath = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
output_dir = 'figuras_tesis'
os.makedirs(output_dir, exist_ok=True)

df_datos_proc = process_real_data(
    csv_filepath, WAVE_PARAMETERS_MODEL, DETECTION_FACTORS
)

fig, axes = plt.subplots(2, 3, figsize=(18, 11))

for i, (ax, params) in enumerate(zip(axes.flat, WAVE_PARAMETERS_MODEL)):
    wave_num = i + 1
    datos_ola = df_datos_proc[df_datos_proc['ola_num'] == wave_num]

    # --- Simulación con los parámetros MLE ---
    s0_para_simulacion = datos_ola['s0_real'].iloc[0] if not datos_ola.empty␣
 ↪else params['s0_theoretical']
    S_model, I_model = simulate_trajectory(params['R0'], params['gamma'],␣
 ↪s0_para_simulacion)

    # --- Graficar Curvas Principales ---
    if not datos_ola.empty:
        ax.plot(datos_ola['S_frac_datos'], datos_ola['I_frac_datos'],
                color=PALETTE['data'], ls='-', label='Datos Observados',␣
 ↪zorder=2)

    visible_model_mask = I_model > (I_model.max() * 0.001)
```

```
    ax.plot(S_model[visible_model_mask], I_model[visible_model_mask],
            color=PALETTE['model'], ls='--', label='Modelo SIR', zorder=3)

    # --- Marcar Picos ---
    idx_pico_modelo = np.argmax(I_model)
    ax.scatter(S_model[idx_pico_modelo], I_model[idx_pico_modelo],
               marker='o', s=100, facecolors='none',␣
↪edgecolors=PALETTE['model'],
               linewidth=2, zorder=4)

    if not datos_ola.empty:
        idx_pico_datos = datos_ola['I_frac_datos'].idxmax()
        ax.scatter(datos_ola.loc[idx_pico_datos, 'S_frac_datos'], datos_ola.
↪loc[idx_pico_datos, 'I_frac_datos'],
                   marker='s', s=100, facecolors='none',␣
↪edgecolors=PALETTE['data'],
                   linewidth=2, zorder=4)

    # --- Configuración de Ejes ---
    ax.set_title(f"{params['name']}\n($R_0 = {params['R0']:.3f}, D =␣
↪{params['D']}$ días)")

    i_max_total = max(datos_ola['I_frac_datos'].max() if not datos_ola.empty␣
↪else 0, I_model.max())
    s_min_total = min(datos_ola['S_frac_datos'].min() if not datos_ola.empty␣
↪else 1, S_model[visible_model_mask].min())
    s_max_total = max(datos_ola['S_frac_datos'].max() if not datos_ola.empty␣
↪else 1, S_model.max())

    s_range = s_max_total - s_min_total
    s_padding = s_range * 0.05

    ax.set_ylim(bottom=-i_max_total*0.05, top=i_max_total * 1.2)
    ax.set_xlim(left=s_max_total + s_padding, right=s_min_total - s_padding)

    if i % 3 == 0:
        ax.set_ylabel('Fracción de Infectados ($I/N$)')
    if i >= 3:
        ax.set_xlabel('Fracción de Susceptibles ($S/N$)')

# --- Leyenda Unificada ---
legend_elements = [
    Line2D([0], [0], color=PALETTE['data'], lw=2.5, ls='-', label='Datos␣
↪Observados'),
    Line2D([0], [0], color=PALETTE['model'], lw=2.5, ls='--', label='Modelo SIR␣
↪(MLE)'),
```

```
    Line2D([0], [0], marker='s', color='w', label='Pico de Datos',
           markerfacecolor='none', markeredgecolor=PALETTE['data'],␣
 ↪markersize=10, markeredgewidth=2),
    Line2D([0], [0], marker='o', color='w', label='Pico del Modelo',
           markerfacecolor='none', markeredgecolor=PALETTE['model'],␣
 ↪markersize=10, markeredgewidth=2)
]
fig.legend(handles=legend_elements, loc='upper center', bbox_to_anchor=(0.5, 1.
 ↪0), ncol=4, frameon=False)
fig.suptitle('Figura Comparativa de Trayectorias (Modelo MLE vs. Datos)',
             fontsize=20, fontweight='bold', y=1.06)

plt.tight_layout(pad=2.0, rect=[0, 0, 1, 0.95])

# --- Guardado de Archivos ---
final_path = os.path.join(output_dir, 'Figura_Validacion_Fase_MLE')
fig.savefig(f"{final_path}.png", dpi=300)
fig.savefig(f"{final_path}.pdf")

plt.show()

print(f"Figuras de validación con parámetros MLE guardadas en la carpeta␣
 ↪'{output_dir}'.")
```



Figuras de validación con parámetros MLE guardadas en la carpeta 'figuras_tesis'.

[74]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import find_peaks
from matplotlib.lines import Line2D

# ==============================================================================
# 1. CONFIGURACIÓN GENERAL Y ESTILO
# ==============================================================================
# Paleta de colores optimizada para claridad y publicación
PALETTE = {
    'model': '#D55E00',      # Naranja quemado (distintivo y claro)
    'data': '#0072B2',       # Azul zafiro (contrasta bien con naranja)
    'annotations': '#333333',# Gris oscuro para texto
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

# Estilo de publicación (basado en seaborn-whitegrid con ajustes)
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 13,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 12,
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.5,
    'figure.facecolor': PALETTE['background'],
    'axes.facecolor': PALETTE['background'],
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 300,
    'savefig.bbox': 'tight',
    'axes.titleweight': 'bold',
    'axes.titlepad': 15,
})

# ==============================================================================
```

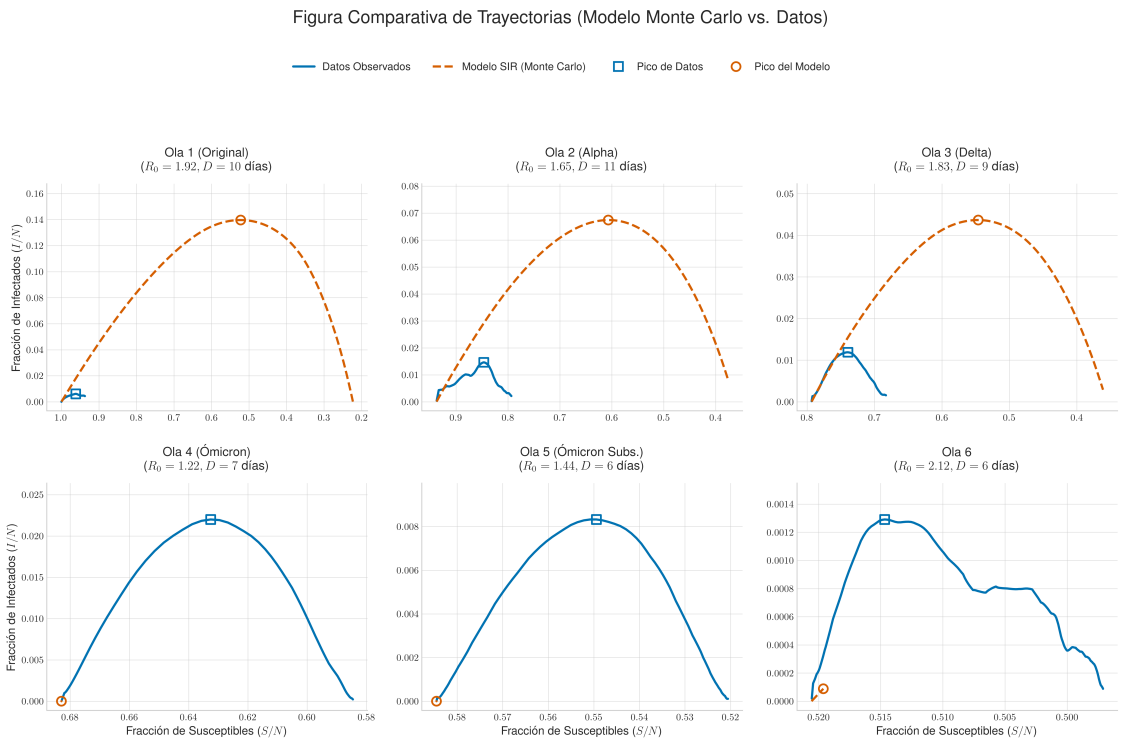
```
# 2. PARÁMETROS Y DATOS (Valores del Modelo Monte Carlo)
# ==============================================================================
# --- AJUSTE: Se usan los R0 Medios del modelo de Monte Carlo ---
WAVE_PARAMETERS_MODEL = [
    {'name': 'Ola 1 (Original)',    'R0': 1.92, 'D': 10, 's0_theoretical': 1.
 ↪000},
    {'name': 'Ola 2 (Alpha)',       'R0': 1.65, 'D': 11, 's0_theoretical': 0.
 ↪740},
    {'name': 'Ola 3 (Delta)',       'R0': 1.83, 'D': 9,  's0_theoretical': 0.
 ↪471},
    {'name': 'Ola 4 (Ómicron)',     'R0': 1.22, 'D': 7,  's0_theoretical': 0.
 ↪701},
    {'name': 'Ola 5 (Ómicron Subs.)', 'R0': 1.44, 'D': 6,  's0_theoretical': 0.
 ↪416},
    {'name': 'Ola 6',               'R0': 2.12, 'D': 6,  's0_theoretical': 0.
 ↪315},
]
for w in WAVE_PARAMETERS_MODEL:
    w['gamma'] = 1.0 / w['D']

# Factores de corrección por subregistro (se mantienen)
DETECTION_FACTORS = {
    'Ola 1 (Original)': 0.091,
    'Ola 2 (Alpha)': 0.091,
    'Ola 3 (Delta)': 0.111,
    'Ola 4 (Ómicron)': 0.143,
    'Ola 5 (Ómicron Subs.)': 0.167,
    'Ola 6': 0.167
}

# ==============================================================================
# 3. PROCESAMIENTO DE DATOS REALES
# ==============================================================================
def process_real_data(filepath, wave_params, detection_factors):
    df_raw = pd.read_csv(filepath)
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()

    poblacion_mx = df_nacional['poblacion'].iloc[0]
    casos_diarios = df_nacional.iloc[0, 3:].astype(float)
    df_data = pd.DataFrame(
        {'casos_diarios': casos_diarios.values},
        index=pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
    )

    casos_suavizados_7d = df_data['casos_diarios'].rolling(7, center=True,␣
 ↪min_periods=1).mean()
```

```
    indices_valles, _ = find_peaks(-casos_suavizados_7d, distance=120,␣
↪prominence=1000)
    fronteras = np.unique(np.concatenate(([0], indices_valles,␣
↪[len(df_data)-1])))

    df_data['ola_num'] = 0
    # Usar los nombres de la lista de parámetros para asignar números
    wave_names = [p['name'] for p in wave_params]
    for i in range(len(fronteras) - 1):
        if i < len(wave_params):
            df_data.iloc[fronteras[i]:fronteras[i+1], df_data.columns.
↪get_loc('ola_num')] = i + 1

    df_data['casos_suavizados_14d'] = df_data['casos_diarios'].rolling(14,␣
↪center=True, min_periods=1).mean()

    df_data['casos_ajustados'] = 0.0
    for i, p in enumerate(wave_params):
        mask = df_data['ola_num'] == (i + 1)
        factor = detection_factors.get(p['name'], 1.0)
        df_data.loc[mask, 'casos_ajustados'] = df_data.loc[mask,␣
↪'casos_suavizados_14d'] / factor

    all_waves_data = []
    casos_acum_global = 0
    for i, p in enumerate(wave_params):
        wave_num = i + 1
        df_wave = df_data[df_data['ola_num'] == wave_num].copy()
        if df_wave.empty:
            continue

        infectados_activos = df_wave['casos_ajustados'].rolling(window=p['D'],␣
↪min_periods=1).sum()
        df_wave['I_frac_datos'] = infectados_activos / poblacion_mx

        s0_real_ola = (poblacion_mx - casos_acum_global) / poblacion_mx
        df_wave['s0_real'] = s0_real_ola

        casos_acum_ola = df_wave['casos_ajustados'].cumsum()
        fraccion_infectada_ola = casos_acum_ola / poblacion_mx
        df_wave['S_frac_datos'] = s0_real_ola - fraccion_infectada_ola

        all_waves_data.append(df_wave)
        casos_acum_global += df_wave['casos_ajustados'].sum()

    return pd.concat(all_waves_data)
```

```
# ============================================================================
# 4. SIMULACIÓN DEL MODELO SIR
# ============================================================================
def sir_rhs(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    return [dSdt, dIdt]

def simulate_trajectory(R0, gamma, s0, duration=400):
    """
    Simula una trayectoria SIR para una duración específica.
    Se usa una duración larga por defecto para asegurar trayectorias completas.
    """
    t_span = [0, duration]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)
    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-7],
        args=(R0, gamma),
        t_eval=t_eval,
        method='RK45'
    )
    s_traj, i_traj = sol.y
    i_traj[i_traj < 0] = 0
    return s_traj, i_traj

# ============================================================================
# 5. EJECUCIÓN Y GRAFICACIÓN
# ============================================================================
csv_filepath = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
output_dir = 'figuras_tesis'
os.makedirs(output_dir, exist_ok=True)

df_datos_proc = process_real_data(
    csv_filepath, WAVE_PARAMETERS_MODEL, DETECTION_FACTORS
)

fig, axes = plt.subplots(2, 3, figsize=(18, 11))

for i, (ax, params) in enumerate(zip(axes.flat, WAVE_PARAMETERS_MODEL)):
    wave_num = i + 1
    datos_ola = df_datos_proc[df_datos_proc['ola_num'] == wave_num]
```

```
    # --- Simulación con los parámetros de Monte Carlo ---
    s0_para_simulacion = datos_ola['s0_real'].iloc[0] if not datos_ola.empty␣
↪else params['s0_theoretical']
    S_model, I_model = simulate_trajectory(params['R0'], params['gamma'],␣
↪s0_para_simulacion)

    # --- Graficar Curvas Principales ---
    if not datos_ola.empty:
        ax.plot(datos_ola['S_frac_datos'], datos_ola['I_frac_datos'],
                color=PALETTE['data'], ls='-', label='Datos Observados',␣
↪zorder=2)

    visible_model_mask = I_model > (I_model.max() * 0.001)
    ax.plot(S_model[visible_model_mask], I_model[visible_model_mask],
            color=PALETTE['model'], ls='--', label='Modelo SIR', zorder=3)

    # --- Marcar Picos ---
    idx_pico_modelo = np.argmax(I_model)
    ax.scatter(S_model[idx_pico_modelo], I_model[idx_pico_modelo],
               marker='o', s=100, facecolors='none',␣
↪edgecolors=PALETTE['model'],
               linewidth=2, zorder=4)

    if not datos_ola.empty:
        idx_pico_datos = datos_ola['I_frac_datos'].idxmax()
        ax.scatter(datos_ola.loc[idx_pico_datos, 'S_frac_datos'], datos_ola.
↪loc[idx_pico_datos, 'I_frac_datos'],
                   marker='s', s=100, facecolors='none',␣
↪edgecolors=PALETTE['data'],
                   linewidth=2, zorder=4)

    # --- Configuración de Ejes ---
    ax.set_title(f"{params['name']}\n($R_0 = {params['R0']:.2f}, D =␣
↪{params['D']}$ días)")

    i_max_total = max(datos_ola['I_frac_datos'].max() if not datos_ola.empty␣
↪else 0, I_model.max())
    s_min_total = min(datos_ola['S_frac_datos'].min() if not datos_ola.empty␣
↪else 1, S_model[visible_model_mask].min())
    s_max_total = max(datos_ola['S_frac_datos'].max() if not datos_ola.empty␣
↪else 1, S_model.max())

    s_range = s_max_total - s_min_total
    s_padding = s_range * 0.05

    ax.set_ylim(bottom=-i_max_total*0.05, top=i_max_total * 1.2)
```

```
    ax.set_xlim(left=s_max_total + s_padding, right=s_min_total - s_padding)

    if i % 3 == 0:
        ax.set_ylabel('Fracción de Infectados ($I/N$)')
    if i >= 3:
        ax.set_xlabel('Fracción de Susceptibles ($S/N$)')

# --- Leyenda Unificada ---
legend_elements = [
    Line2D([0], [0], color=PALETTE['data'], lw=2.5, ls='-', label='Datos␣
 ↪Observados'),
    Line2D([0], [0], color=PALETTE['model'], lw=2.5, ls='--', label='Modelo SIR␣
 ↪(Monte Carlo)'),
    Line2D([0], [0], marker='s', color='w', label='Pico de Datos',
           markerfacecolor='none', markeredgecolor=PALETTE['data'],␣
 ↪markersize=10, markeredgewidth=2),
    Line2D([0], [0], marker='o', color='w', label='Pico del Modelo',
           markerfacecolor='none', markeredgecolor=PALETTE['model'],␣
 ↪markersize=10, markeredgewidth=2)
]
fig.legend(handles=legend_elements, loc='upper center', bbox_to_anchor=(0.5, 1.
 ↪0), ncol=4, frameon=False)
fig.suptitle('Figura Comparativa de Trayectorias (Modelo Monte Carlo vs.␣
 ↪Datos)',
             fontsize=20, fontweight='bold', y=1.06)

plt.tight_layout(pad=2.0, rect=[0, 0, 1, 0.95])

# --- Guardado de Archivos ---
final_path = os.path.join(output_dir, 'Figura_Validacion_Fase_MonteCarlo')
fig.savefig(f"{final_path}.png", dpi=300)
fig.savefig(f"{final_path}.pdf")

plt.show()

print(f"Figuras de validación con parámetros de Monte Carlo guardadas en la␣
 ↪carpeta '{output_dir}'.")
```



Figuras de validación con parámetros de Monte Carlo guardadas en la carpeta 'figuras_tesis'.

[70]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import find_peaks
from matplotlib.lines import Line2D

# ==============================================================================
# 1. CONFIGURACIÓN GENERAL Y ESTILO
# ==============================================================================
# Paleta de colores optimizada para claridad y publicación
PALETTE = {
    'model': '#D55E00',       # Naranja quemado (distintivo y claro)
    'data': '#0072B2',        # Azul zafiro (contrasta bien con naranja)
    'annotations': '#333333',# Gris oscuro para texto
    'background': '#FFFFFF',
    'grid': '#DDDDDD'
}

# Estilo de publicación (basado en seaborn-whitegrid con ajustes)
```

```
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.sans-serif': ['DejaVu Sans', 'Arial', 'Helvetica'],
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 13,
    'xtick.labelsize': 11,
    'ytick.labelsize': 11,
    'legend.fontsize': 12,
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.5,
    'figure.facecolor': PALETTE['background'],
    'axes.facecolor': PALETTE['background'],
    'axes.spines.top': False,
    'axes.spines.right': False,
    'figure.dpi': 300,
    'savefig.bbox': 'tight',
    'axes.titleweight': 'bold',
    'axes.titlepad': 15,
})

# ============================================================================
# 2. PARÁMETROS Y DATOS (Valores de tu proyecto)
# ============================================================================
WAVE_PARAMETERS_MODEL = [
    {'name': 'Ola 1 (Original)', 'R0': 1.92, 'D': 10, 's0_theoretical': 1.000},
    {'name': 'Ola 2 (Alpha)',    'R0': 1.65, 'D': 11, 's0_theoretical': 0.740},
    {'name': 'Ola 3 (Delta)',    'R0': 2.84, 'D': 9,  's0_theoretical': 0.471},
    {'name': 'Ola 4 (Ómicron)',  'R0': 1.82, 'D': 7,  's0_theoretical': 0.701},
    {'name': 'Ola 5 (BA.4/5)',   'R0': 5.41, 'D': 6,  's0_theoretical': 0.416},
    {'name': 'Ola 6 (BQ.1/XBB)', 'R0': 4.82, 'D': 6,  's0_theoretical': 0.315},
]
for w in WAVE_PARAMETERS_MODEL:
    w['gamma'] = 1.0 / w['D']

# Factores de corrección por subregistro
DETECTION_FACTORS = {
    'Ola 1 (Original)': 0.091,
    'Ola 2 (Alpha)': 0.091,
    'Ola 3 (Delta)': 0.111,
    'Ola 4 (Ómicron)': 0.143,
    'Ola 5 (BA.4/5)': 0.167,
    'Ola 6 (BQ.1/XBB)': 0.167
}

# ============================================================================
```

```python
# 3. PROCESAMIENTO DE DATOS REALES (Tu función robusta)
# ==============================================================================
def process_real_data(filepath, wave_params, detection_factors):
    df_raw = pd.read_csv(filepath)
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()

    poblacion_mx = df_nacional['poblacion'].iloc[0]
    casos_diarios = df_nacional.iloc[0, 3:].astype(float)
    df_data = pd.DataFrame(
        {'casos_diarios': casos_diarios.values},
        index=pd.to_datetime(casos_diarios.index, format='%d-%m-%Y')
    )

    casos_suavizados_7d = df_data['casos_diarios'].rolling(7, center=True,␣
 ↪min_periods=1).mean()
    indices_valles, _ = find_peaks(-casos_suavizados_7d, distance=120,␣
 ↪prominence=1000)
    fronteras = np.unique(np.concatenate(([0], indices_valles,␣
 ↪[len(df_data)-1])))

    df_data['ola_num'] = 0
    for i in range(len(fronteras) - 1):
        if i < len(wave_params):
            df_data.iloc[fronteras[i]:fronteras[i+1], df_data.columns.
 ↪get_loc('ola_num')] = i + 1

    df_data['casos_suavizados_14d'] = df_data['casos_diarios'].rolling(14,␣
 ↪center=True, min_periods=1).mean()

    df_data['casos_ajustados'] = 0.0
    for i, p in enumerate(wave_params):
        mask = df_data['ola_num'] == (i + 1)
        factor = detection_factors.get(p['name'], 1.0)
        df_data.loc[mask, 'casos_ajustados'] = df_data.loc[mask,␣
 ↪'casos_suavizados_14d'] / factor

    all_waves_data = []
    casos_acum_global = 0
    for i, p in enumerate(wave_params):
        wave_num = i + 1
        df_wave = df_data[df_data['ola_num'] == wave_num].copy()
        if df_wave.empty:
            continue

        infectados_activos = df_wave['casos_ajustados'].rolling(window=p['D'],␣
 ↪min_periods=1).sum()
```

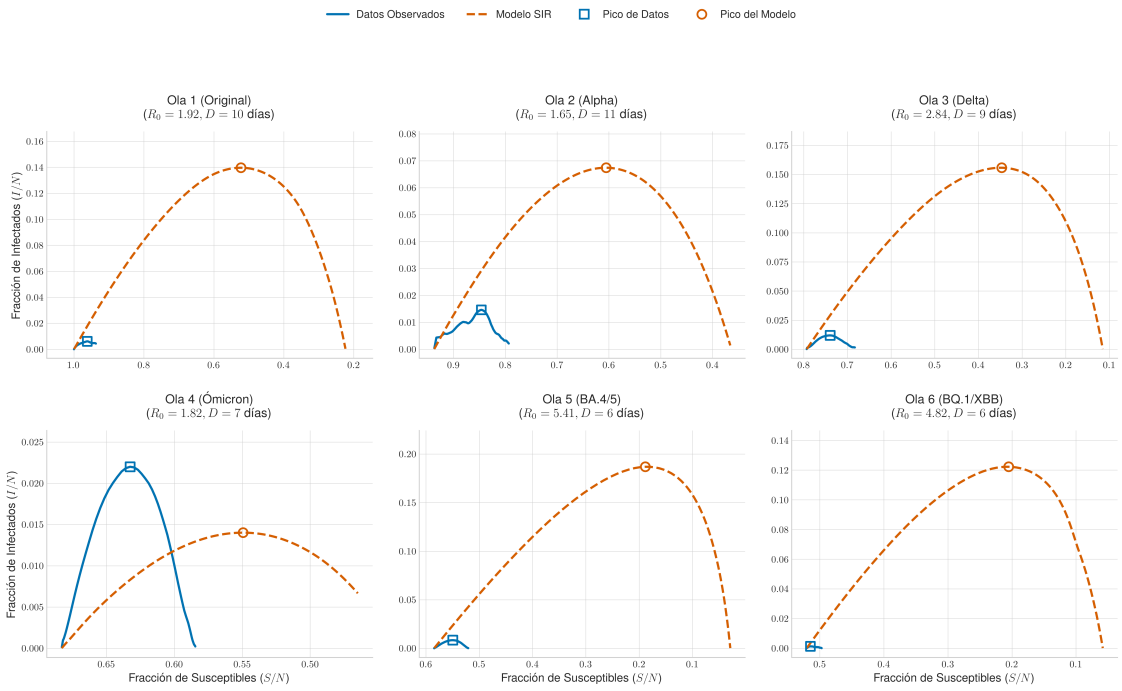
```
        df_wave['I_frac_datos'] = infectados_activos / poblacion_mx

        s0_real_ola = (poblacion_mx - casos_acum_global) / poblacion_mx
        df_wave['s0_real'] = s0_real_ola

        casos_acum_ola = df_wave['casos_ajustados'].cumsum()
        fraccion_infectada_ola = casos_acum_ola / poblacion_mx
        df_wave['S_frac_datos'] = s0_real_ola - fraccion_infectada_ola

        all_waves_data.append(df_wave)
        casos_acum_global += df_wave['casos_ajustados'].sum()

    return pd.concat(all_waves_data)

# ==============================================================================
# 4. SIMULACIÓN DEL MODELO SIR
# ==============================================================================
def sir_rhs(t, y, R0, gamma):
    S, I = y
    beta = R0 * gamma
    dSdt = -beta * S * I
    dIdt = beta * S * I - gamma * I
    return [dSdt, dIdt]

# --- AJUSTE: La función ahora acepta una duración para la simulación ---
def simulate_trajectory(R0, gamma, s0, duration=300):
    """
    Simula una trayectoria SIR para una duración específica.
    """
    t_span = [0, duration]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)
    sol = solve_ivp(
        fun=sir_rhs,
        t_span=t_span,
        y0=[s0, 1e-7],
        args=(R0, gamma),
        t_eval=t_eval,
        method='RK45'
    )
    s_traj, i_traj = sol.y
    i_traj[i_traj < 0] = 0
    return s_traj, i_traj

# ==============================================================================
# 5. EJECUCIÓN Y GRAFICACIÓN (CON AJUSTES FINALES)
# ==============================================================================
csv_filepath = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
```

```
output_dir = 'figuras_tesis'
os.makedirs(output_dir, exist_ok=True)

df_datos_proc = process_real_data(
    csv_filepath, WAVE_PARAMETERS_MODEL, DETECTION_FACTORS
)

fig, axes = plt.subplots(2, 3, figsize=(18, 11))

for i, (ax, params) in enumerate(zip(axes.flat, WAVE_PARAMETERS_MODEL)):
    wave_num = i + 1
    datos_ola = df_datos_proc[df_datos_proc['ola_num'] == wave_num]

    # --- AJUSTE: Duración de simulación específica por ola ---
    sim_duration = 300  # Duración por defecto
    if wave_num in [2, 4]:
        sim_duration = 450  # Duración extendida para Olas 2 y 4 para que se␣
 ↪vean completas

    # --- Simulación ---
    s0_para_simulacion = datos_ola['s0_real'].iloc[0] if not datos_ola.empty␣
 ↪else params['s0_theoretical']
    S_model, I_model = simulate_trajectory(params['R0'], params['gamma'],␣
 ↪s0_para_simulacion, duration=sim_duration)

    # --- Graficar Curvas Principales ---
    if not datos_ola.empty:
        ax.plot(datos_ola['S_frac_datos'], datos_ola['I_frac_datos'],
                color=PALETTE['data'], ls='-', label='Datos Observados',␣
 ↪zorder=2)

    visible_model_mask = I_model > (I_model.max() * 0.001)
    ax.plot(S_model[visible_model_mask], I_model[visible_model_mask],
            color=PALETTE['model'], ls='--', label='Modelo SIR', zorder=3)

    # --- Marcar Picos ---
    idx_pico_modelo = np.argmax(I_model)
    ax.scatter(S_model[idx_pico_modelo], I_model[idx_pico_modelo],
               marker='o', s=100, facecolors='none',␣
 ↪edgecolors=PALETTE['model'],
               linewidth=2, zorder=4)

    if not datos_ola.empty:
        idx_pico_datos = datos_ola['I_frac_datos'].idxmax()
        ax.scatter(datos_ola.loc[idx_pico_datos, 'S_frac_datos'], datos_ola.
 ↪loc[idx_pico_datos, 'I_frac_datos'],
```

```
                    marker='s', s=100, facecolors='none',␣
 ↪edgecolors=PALETTE['data'],
                    linewidth=2, zorder=4)

    # --- Configuración de Ejes ---
    ax.set_title(f"{params['name']}\n($R_0 = {params['R0']:.2f}, D =␣
 ↪{params['D']}$ días)")

    i_max_total = max(datos_ola['I_frac_datos'].max() if not datos_ola.empty␣
 ↪else 0, I_model.max())
    s_min_total = min(datos_ola['S_frac_datos'].min() if not datos_ola.empty␣
 ↪else 1, S_model[visible_model_mask].min())
    s_max_total = max(datos_ola['S_frac_datos'].max() if not datos_ola.empty␣
 ↪else 1, S_model.max())

    s_range = s_max_total - s_min_total
    s_padding = s_range * 0.05

    # --- AJUSTE: Más "aire" o relleno para la Ola 1 ---
    if wave_num == 1:
        s_padding = s_range * 0.1

    ax.set_ylim(bottom=-i_max_total*0.05, top=i_max_total * 1.2)
    ax.set_xlim(left=s_max_total + s_padding, right=s_min_total - s_padding)

    if i % 3 == 0:
        ax.set_ylabel('Fracción de Infectados ($I/N$)')
    if i >= 3:
        ax.set_xlabel('Fracción de Susceptibles ($S/N$)')

# --- Leyenda Unificada ---
legend_elements = [
    Line2D([0], [0], color=PALETTE['data'], lw=2.5, ls='-', label='Datos␣
 ↪Observados'),
    Line2D([0], [0], color=PALETTE['model'], lw=2.5, ls='--', label='Modelo␣
 ↪SIR'),
    Line2D([0], [0], marker='s', color='w', label='Pico de Datos',
           markerfacecolor='none', markeredgecolor=PALETTE['data'],␣
 ↪markersize=10, markeredgewidth=2),
    Line2D([0], [0], marker='o', color='w', label='Pico del Modelo',
           markerfacecolor='none', markeredgecolor=PALETTE['model'],␣
 ↪markersize=10, markeredgewidth=2)
]
fig.legend(handles=legend_elements, loc='upper center', bbox_to_anchor=(0.5, 1.
 ↪0), ncol=4, frameon=False)
```

```
fig.suptitle('Figura 4.6: Comparativa de Trayectorias en el Espacio de Fase␣
 ↪(Modelo vs. Datos)',
             fontsize=20, fontweight='bold', y=1.06)

plt.tight_layout(pad=2.0, rect=[0, 0, 1, 0.95])

# --- Guardado de Archivos ---
final_path = os.path.join(output_dir, 'Figura_4_6_Validacion_Fase_AjusteFinal')
fig.savefig(f"{final_path}.png", dpi=300)
fig.savefig(f"{final_path}.pdf")

plt.show()

print(f"Figuras de validación con ajustes finales guardadas con éxito en la␣
 ↪carpeta '{output_dir}'.")
```

Figura 4.6: Comparativa de Trayectorias en el Espacio de Fase (Modelo vs. Datos)




```
Figuras de validación con ajustes finales guardadas con éxito en la carpeta
'figuras_tesis'.
```

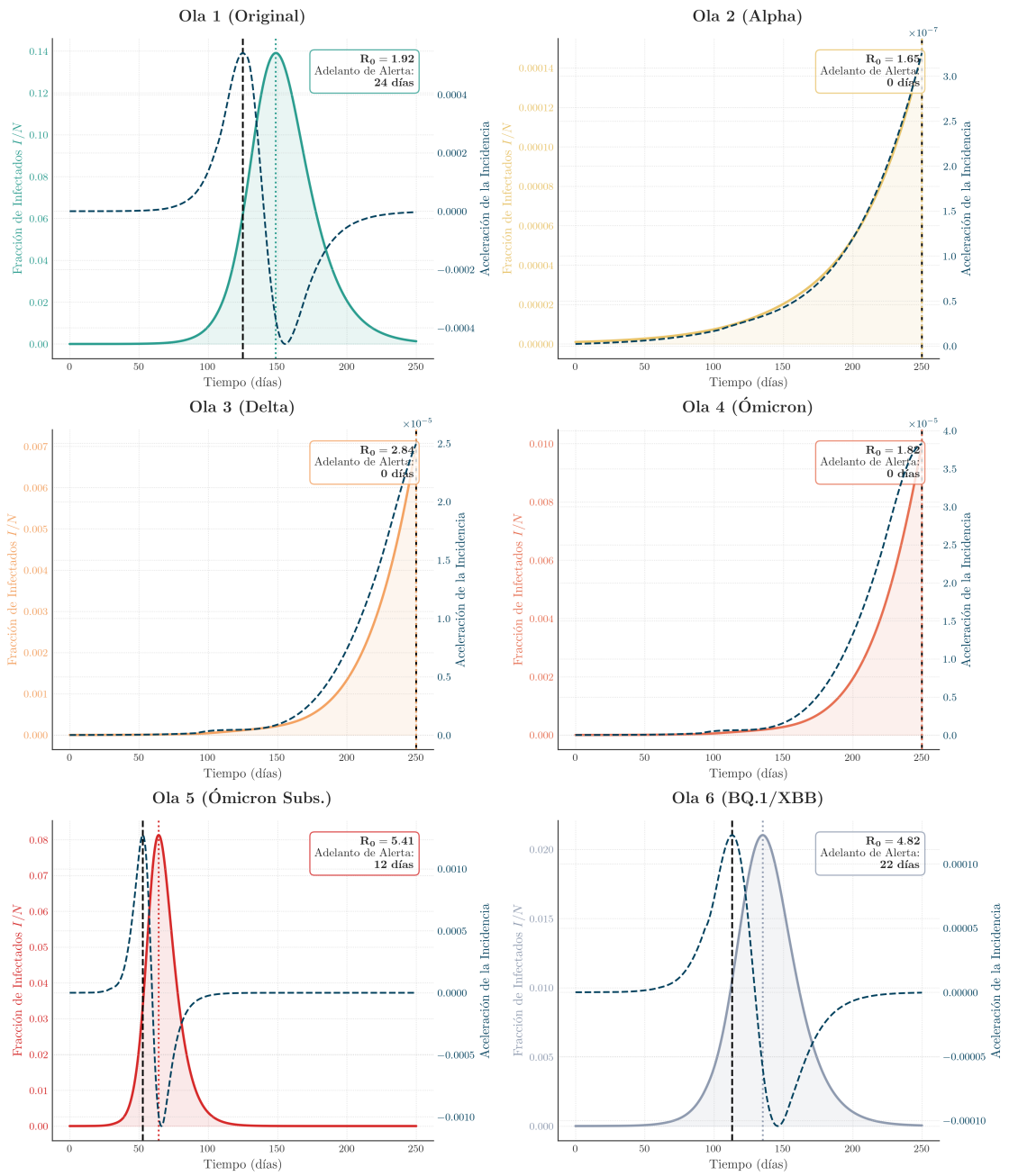
[65]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

def setup_publication_style():
```

```
    """Configura el estilo de Matplotlib para una estética profesional y␣
 ↪académica."""
    plt.rcParams.update({
        'font.family': 'serif',
        'font.serif': ['Computer Modern Roman', 'Times New Roman'],
        'text.usetex': True,
        'text.latex.preamble': r'\usepackage{amsmath} \usepackage{amssymb}',
        'font.size': 10,
        'axes.labelsize': 11,
        'axes.titlesize': 13,
        'xtick.labelsize': 9,
        'ytick.labelsize': 9,
        'legend.fontsize': 9,
        'figure.titlesize': 16,
        'lines.linewidth': 2.0,
        'axes.linewidth': 0.8,
        'grid.color': '#cccccc',
        'grid.linestyle': ':',
        'axes.spines.right': False,
        'axes.spines.top': False,
        'savefig.dpi': 300,
        'savefig.format': 'pdf',
        'savefig.bbox': 'tight'
    })

def sir_model(t, y, R0, gamma):
    """Sistema de Ecuaciones Diferenciales Ordinarias (EDOs) para el modelo SIR.
 ↪"""
    S, I = y
    beta = R0 * gamma
    dS_dt = -beta * S * I
    dI_dt = beta * S * I - gamma * I
    return [dS_dt, dI_dt]

# --- PARÁMETROS ROBUSTOS VALIDADOS (de la tabla de resultados del script␣
 ↪anterior) ---
WAVE_PARAMETERS = [
    {'name': 'Ola 1 (Original)', 'R0': 1.92, 's0': 1.000, 'D': 10, 'color':␣
 ↪'#2a9d8f'},
    {'name': 'Ola 2 (Alpha)',    'R0': 1.65, 's0': 0.740, 'D': 11, 'color':␣
 ↪'#e9c46a'},
    {'name': 'Ola 3 (Delta)',    'R0': 2.84, 's0': 0.471, 'D': 9,  'color':␣
 ↪'#f4a261'},
    {'name': 'Ola 4 (Ómicron)',  'R0': 1.82, 's0': 0.701, 'D': 7,  'color':␣
 ↪'#e76f51'},
```

```
    {'name': 'Ola 5 (Ómicron Subs.)', 'R0': 5.41, 's0': 0.416, 'D': 6,  'color':
 ↪ '#d62828'},
    {'name': 'Ola 6 (BQ.1/XBB)', 'R0': 4.82, 's0': 0.315, 'D': 6,  'color':␣
 ↪'#8e9aaf'}
]

# --- EJECUCIÓN DEL ANÁLISIS Y VISUALIZACIÓN ---
setup_publication_style()
fig, axes = plt.subplots(3, 2, figsize=(12, 14), constrained_layout=True)
axes = axes.flatten()

print("Analizando y visualizando el sistema de alerta para cada ola...")

for i, wave in enumerate(WAVE_PARAMETERS):
    ax = axes[i]

    # 1. Simulación del Modelo SIR
    gamma = 1.0 / wave['D']
    t_span = [0, 250]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)

    sol = solve_ivp(
        fun=sir_model,
        t_span=t_span,
        y0=[wave['s0'], 1e-6], # Iniciar con una pequeña fracción de infectados
        args=(wave['R0'], gamma),
        t_eval=t_eval
    )
    S_t, I_t = sol.y

    # 2. Cálculo de la Aceleración de la Incidencia
    beta = wave['R0'] * gamma
    incidencia = beta * S_t * I_t
    aceleracion = np.gradient(incidencia, sol.t)

    # 3. Identificación de Puntos Críticos
    t_peak_I = sol.t[np.argmax(I_t)]
    # Buscar el pico de aceleración solo antes del pico de infecciones
    idx_ascenso = np.where(sol.t <= t_peak_I)
    t_peak_accel = sol.t[idx_ascenso][np.argmax(aceleracion[idx_ascenso])]

    # 4. Cuantificación del Adelanto
    adelanto_dias = t_peak_I - t_peak_accel

    # 5. Visualización
    ax.set_title(r'\textbf{' + wave['name'] + '}', fontsize=14)
```

```
    # Eje primario: Infectados
    color_I = wave['color']
    ax.plot(sol.t, I_t, color=color_I, label='Fracción de Infectados $I(t)$')
    ax.fill_between(sol.t, I_t, color=color_I, alpha=0.1)
    ax.set_ylabel('Fracción de Infectados $I/N$', color=color_I)
    ax.tick_params(axis='y', labelcolor=color_I)
    ax.grid(True, linestyle=':')

    # Líneas verticales de alerta
    ax.axvline(t_peak_accel, color='black', linestyle='--', lw=1.5,
               label=f'Alerta Temprana (Día {t_peak_accel:.0f})')
    ax.axvline(t_peak_I, color=color_I, linestyle=':', lw=1.5,
               label=f'Pico de la Ola (Día {t_peak_I:.0f})')

    # Eje secundario: Aceleración
    ax2 = ax.twinx()
    color_accel = '#003f5c'
    ax2.plot(sol.t, aceleracion, color=color_accel, ls='--', lw=1.5,
             label='Aceleración de la Epidemia')
    ax2.set_ylabel('Aceleración de la Incidencia', color=color_accel)
    ax2.tick_params(axis='y', labelcolor=color_accel)

    ax.set_xlabel('Tiempo (días)')

    # Añadir cuadro de texto con el resultado clave
    info_text = (f'$\\mathbf{{R_0 = {wave["R0"]:.2f}}}$\n'
                 f'Adelanto de Alerta:\n'
                 f'$\\mathbf{{{adelanto_dias:.0f}}}$ \\textbf{{días}}')

    ax.text(0.95, 0.95, info_text, transform=ax.transAxes, ha='right',␣
 ↪va='top', fontsize=10,
            bbox=dict(boxstyle='round,pad=0.4', fc='white', alpha=0.8,␣
 ↪ec=wave['color']))

fig.suptitle(r'\textbf{Diseño de Alerta Temprana Basado en la Aceleración de la␣
 ↪Epidemia por Ola}', y=1.03)
fig.savefig('Figura_Alertas_Tempranas_6_Olas.pdf')
plt.show()

print("\nAnálisis completo. Gráfico 'Figura_Alertas_Tempranas_6_Olas.pdf'␣
 ↪guardado.")
```

```
Analizando y visualizando el sistema de alerta para cada ola…
```




```
Análisis completo. Gráfico 'Figura_Alertas_Tempranas_6_Olas.pdf' guardado.
```

[71]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from matplotlib.colors import ListedColormap
```

```
from matplotlib.patches import Patch

def setup_publication_style():
    """Configura el estilo de Matplotlib para una estética profesional y académica."""
    plt.rcParams.update({
        'font.family': 'serif',
        'font.serif': ['Computer Modern Roman', 'Times New Roman'],
        'text.usetex': True,
        'text.latex.preamble': r'\usepackage{amsmath} \usepackage{amssymb}',
        'font.size': 10,
        'axes.labelsize': 11,
        'axes.titlesize': 13,
        'xtick.labelsize': 9,
        'ytick.labelsize': 9,
        'legend.fontsize': 9,
        'figure.titlesize': 16,
        'lines.linewidth': 2.0,
        'axes.linewidth': 0.8,
        'grid.color': '#cccccc',
        'grid.linestyle': ':',
        'axes.spines.right': False,
        'axes.spines.top': False,
        'savefig.dpi': 300,
        'savefig.format': 'pdf',
        'savefig.bbox': 'tight'
    })

def sir_model(t, y, R0, gamma):
    """Sistema de Ecuaciones Diferenciales Ordinarias (EDOs) para el modelo SIR."""
    S, I = y
    beta = R0 * gamma
    dS_dt = -beta * S * I
    dI_dt = beta * S * I - gamma * I
    return [dS_dt, dI_dt]

# --- PARÁMETROS ROBUSTOS VALIDADOS POR OLA ---
WAVE_PARAMETERS = [
    {'name': 'Ola 1 (Original)', 'R0': 1.92, 's0': 1.000, 'D': 10, 'color': '#2a9d8f'},
    {'name': 'Ola 2 (Alpha)',    'R0': 1.65, 's0': 0.740, 'D': 11, 'color': '#e9c46a'},
    {'name': 'Ola 3 (Delta)',    'R0': 2.84, 's0': 0.471, 'D': 9,  'color': '#f4a261'},
    {'name': 'Ola 4 (Ómicron)',  'R0': 1.82, 's0': 0.701, 'D': 7,  'color': '#e76f51'},
```

```
    {'name': 'Ola 5 (Ómicron Subs.)', 'R0': 5.41, 's0': 0.416, 'D': 6,  'color':
 ↪ '#d62828'},
    {'name': 'Ola 6 (BQ.1/XBB)', 'R0': 4.82, 's0': 0.315, 'D': 6,  'color':␣
 ↪'#8e9aaf'}
]

# --- LÓGICA PRINCIPAL DE SIMULACIÓN Y ANÁLISIS ---
print("Ejecutando simulación y análisis avanzado para las 6 olas...")
simulation_results = {}
for wave in WAVE_PARAMETERS:
    gamma = 1.0 / wave['D']
    t_span = [0, 250]
    t_eval = np.linspace(t_span[0], t_span[1], 1000)

    sol = solve_ivp(
        fun=sir_model, t_span=t_span, y0=[wave['s0'], 1e-6],
        args=(wave['R0'], gamma), t_eval=t_eval
    )

    S_t, I_t = sol.y
    R_t = wave['R0'] * S_t
    v_t = np.gradient(R_t, sol.t)
    a_t = np.gradient(v_t, sol.t)

    idx_ascenso = np.where((R_t > 1) & (v_t > 0))[0]
    tau_v = np.percentile(v_t[idx_ascenso], 50) if len(idx_ascenso) > 0 else 0
    tau_a = np.percentile(a_t[idx_ascenso], 75) if len(idx_ascenso) > 0 else 0

    # === NUEVA LÓGICA SECUENCIAL Y ESCALONADA ===
    alert_levels = np.zeros_like(sol.t)
    # Nivel inicial
    alert_levels[0] = 1 if R_t[0] < 1 else 2
    # Iterar para aplicar transiciones
    for i in range(1, len(sol.t)):
        prev_level = alert_levels[i-1]
        current_level = prev_level

        if prev_level == 1: # VERDE
            if R_t[i] > 1: current_level = 2 # Escalar a Amarillo
        elif prev_level == 2: # AMARILLO
            if v_t[i] > tau_v: current_level = 3 # Escalar a Naranja
            elif R_t[i] < 1: current_level = 1 # Bajar a Verde
        elif prev_level == 3: # NARANJA
            if a_t[i] > tau_a: current_level = 4 # Escalar a Rojo
            elif v_t[i] < tau_v: current_level = 2 # Bajar a Amarillo
        elif prev_level == 4: # ROJO
            if a_t[i] < tau_a: current_level = 3 # Bajar a Naranja
```

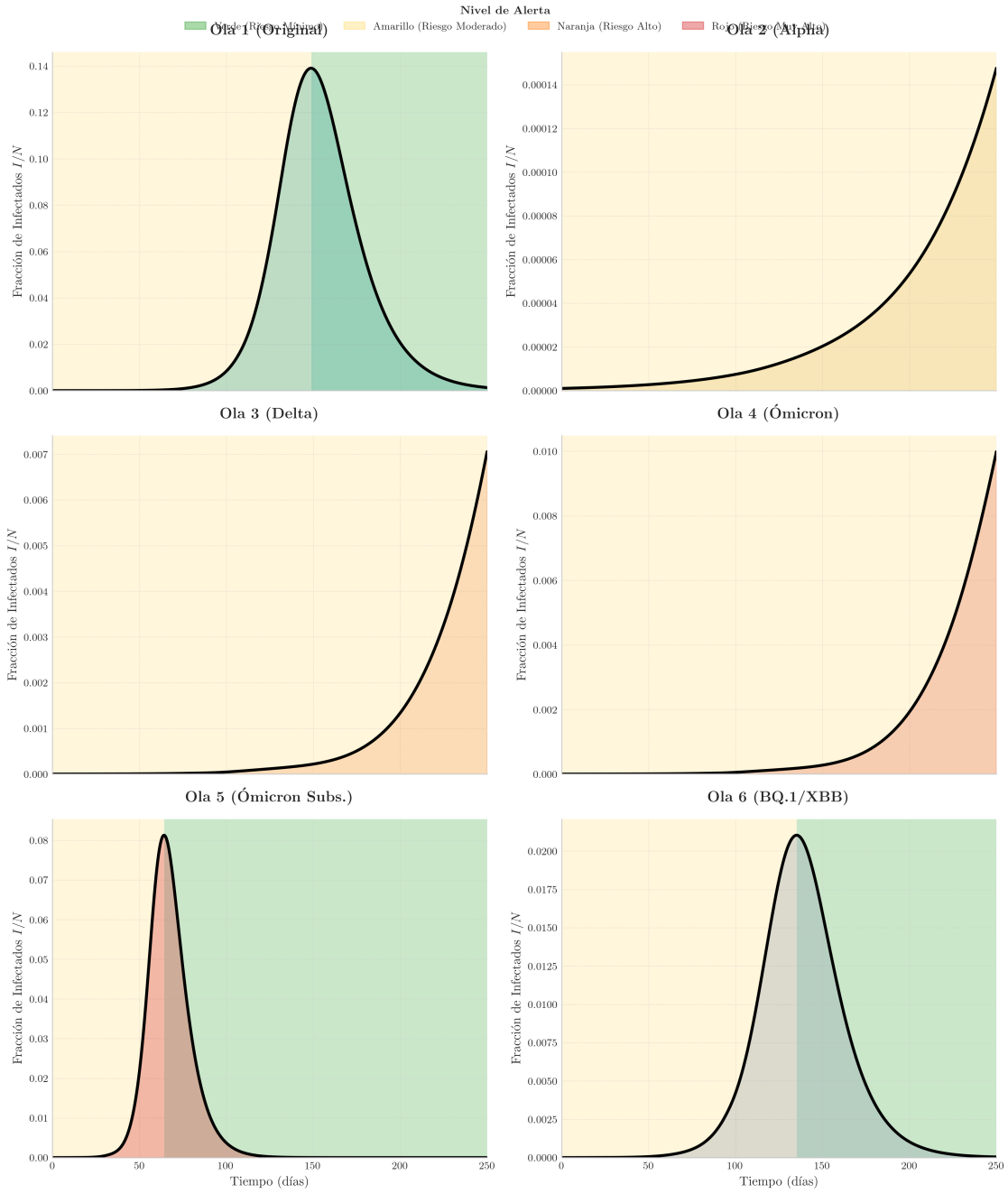
```
        alert_levels[i] = current_level

    t_peak_I = sol.t[np.argmax(I_t)]
    I_max = np.max(I_t)
    R_final = 1 - S_t[-1] - I_t[-1]

    simulation_results[wave['name']] = {
        'sol': sol, 'I_t': I_t, 'alert_levels': alert_levels, 'color':␣
 ↪wave['color'],
        't_peak_I': t_peak_I, 'I_max': I_max, 'R_final': R_final
    }
print("Análisis completado.")

# --- VISUALIZACIÓN ---
setup_publication_style()

#  Gráfico 1: Líneas de Tiempo del Sistema de Alerta (Transiciones)
print("Generando Gráfico 1: Líneas de Tiempo de Alertas (Lógica Secuencial)...")
fig1, axes = plt.subplots(3, 2, figsize=(12, 14), sharex=True,␣
 ↪constrained_layout=True)
axes = axes.flatten()
alert_colors = {'Verde (Riesgo Mínimo)': '#2ca02c', 'Amarillo (Riesgo␣
 ↪Moderado)': '#ffdd71', 'Naranja (Riesgo Alto)': '#ff7f0e', 'Rojo (Riesgo Muy␣
 ↪Alto)': '#d62728'}
alert_cmap = ListedColormap(list(alert_colors.values()))

for i, wave_name in enumerate(simulation_results.keys()):
    res = simulation_results[wave_name]
    ax = axes[i]
    ax.set_title(r'\textbf{' + wave_name + '}')

    ax.plot(res['sol'].t, res['I_t'], color='black', lw=2.5, label='Infectados␣
 ↪$I(t)$')
    ax.fill_between(res['sol'].t, res['I_t'], color=res['color'], alpha=0.3)
    ax.set_ylabel('Fracción de Infectados $I/N$')

    t = res['sol'].t
    levels = res['alert_levels']
    change_points = np.where(np.diff(levels) != 0)[0] + 1
    all_points = np.unique(np.concatenate(([0], change_points, [len(t)-1])))

    for j in range(len(all_points) - 1):
        start_idx, end_idx = all_points[j], all_points[j+1]
        start_time, end_time = t[start_idx], t[end_idx]
        alert_level = levels[start_idx]
        color = alert_cmap(int(alert_level) - 1)
```

```
        ax.axvspan(start_time, end_time, color=color, alpha=0.25, ec='none',␣
 ↪zorder=-1)

    if i >= 4: ax.set_xlabel('Tiempo (días)')
    ax.set_ylim(bottom=0)
    ax.set_xlim(0, 250)

legend_patches = [Patch(color=color, label=label, alpha=0.4) for label, color␣
 ↪in alert_colors.items()]
fig1.legend(handles=legend_patches, title=r'\textbf{Nivel de Alerta}',␣
 ↪loc='upper center',
           bbox_to_anchor=(0.5, 1.02), ncol=4)
fig1.suptitle(r'\textbf{Figura 1: Transición Escalonada de Niveles de Alerta␣
 ↪por Ola}', y=1.06)
plt.savefig('Figura_1_Transicion_Alertas_Secuencial.pdf')
plt.show()

#  Gráfico 2: Retrato de Fases Combinado
print("Generando Gráfico 2: Retrato de Fases Combinado...")
fig2, ax = plt.subplots(figsize=(10, 8))
for wave_name, res in simulation_results.items():
    S_t = res['sol'].y[0]
    I_t = res['I_t']
    ax.plot(S_t, I_t, color=res['color'], lw=2.5, label=wave_name)

ax.set_xlabel(r'Fracción de Susceptibles ($S/N$)')
ax.set_ylabel(r'Fracción de Infectados ($I/N$)')
ax.set_title(r'\textbf{Figura 2: Retrato de Fases Combinado de las 6 Olas}')
ax.legend(title='Ola Epidémica', loc='upper right')
ax.grid(True)
ax.set_xlim(0, 1)
ax.set_ylim(bottom=0)
plt.savefig('Figura_2_Retrato_Fases_Combinado.pdf')
plt.show()

#  Gráfico 3: Resumen Comparativo de Métricas
print("Generando Gráfico 3: Resumen Comparativo de Métricas...")
fig3, axes = plt.subplots(1, 3, figsize=(15, 5), constrained_layout=True)
titles = [r'Pico Máximo de Infecciones ($I_{max}$)', r'Tamaño Total de la␣
 ↪Epidemia ($R_{\infty}$)', r'Tiempo hasta el Pico (días)']
y_labels = ['Fracción de la Población', 'Fracción de la Población', 'Días']
colors = [res['color'] for res in simulation_results.values()]
wave_names = list(simulation_results.keys())
metrics = ['I_max', 'R_final', 't_peak_I']

for i, metric in enumerate(metrics):
    ax = axes[i]
```

```
    values = [res[metric] for res in simulation_results.values()]
    bars = ax.bar(wave_names, values, color=colors, edgecolor='black', lw=0.8)
    ax.set_title(titles[i])
    ax.set_ylabel(y_labels[i])
    ax.tick_params(axis='x', labelrotation=45, labelsize=8)
    ax.grid(axis='y')
    ax.bar_label(bars, fmt='%.2f', fontsize=8)

fig3.suptitle(r'\textbf{Figura 3: Comparativa de Métricas de Impacto por Ola}',␣
 ↪y=1.05)
plt.savefig('Figura_3_Resumen_Metricas.pdf')
plt.show()

print("\nAnálisis avanzado completado. Todas las figuras han sido guardadas.")
```

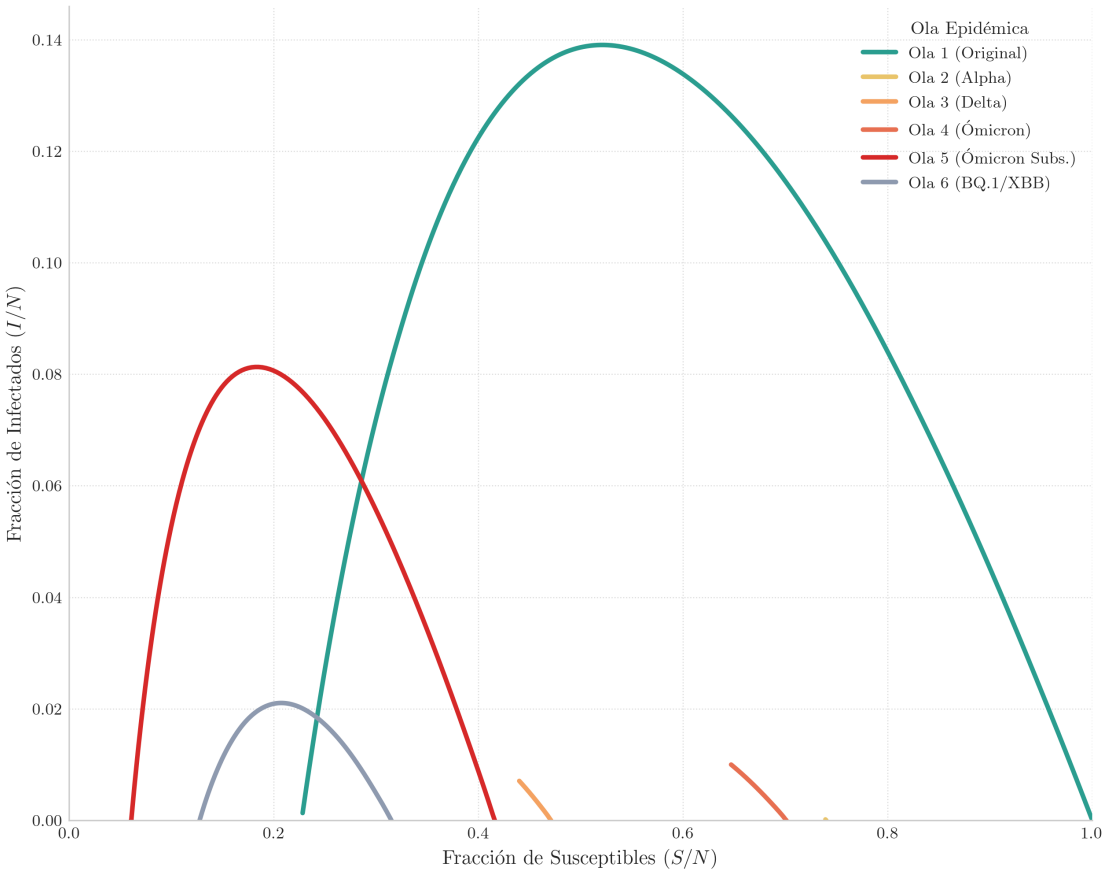
```
Ejecutando simulación y análisis avanzado para las 6 olas…
Análisis completado.
Generando Gráfico 1: Líneas de Tiempo de Alertas (Lógica Secuencial)…
```

Figura 1: Transición Escalonada de Niveles de Alerta por Ola



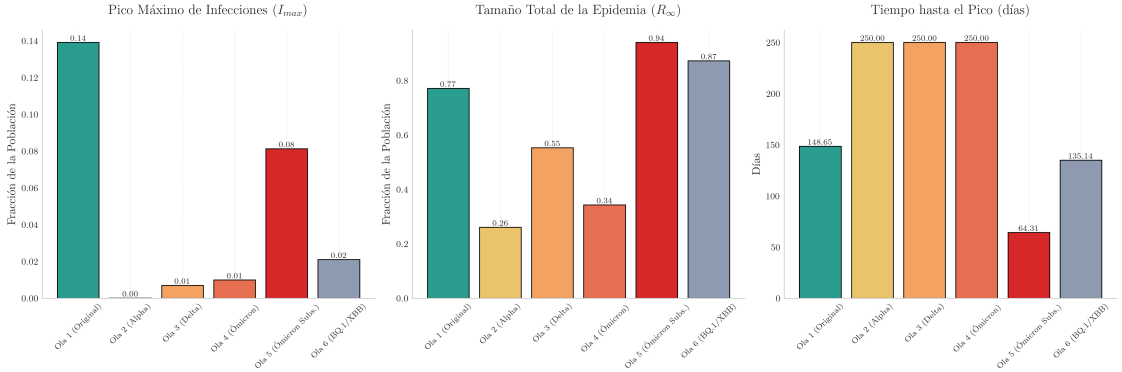
Generando Gráfico 2: Retrato de Fases Combinado…

Figura 2: Retrato de Fases Combinado de las 6 Olas



Generando Gráfico 3: Resumen Comparativo de Métricas…

Figura 3: Comparativa de Métricas de Impacto por Ola



Análisis avanzado completado. Todas las figuras han sido guardadas.

[ ]: